



UNIVERSIDAD AUTÓNOMA METROPOLITANA

Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería

Calibración de Hiper-Parámetros en Algoritmos
Metaheurísticos y Modelos de Lenguaje para la
Detección de Noticias Falsas

Idónea Comunicación de Resultados

que presenta el:

Ing. Gabriel Hurtado Avilés

para obtener el grado de:

Maestro en Ciencias de la Computación

Directores:

Dr. José Alejandro Reyes Ortiz

Dr. Román Anselmo Mora Gutiérrez

Ciudad de México

Octubre 2025

Resumen

Este proyecto aborda el desafío de la detección de noticias falsas en español mediante la aplicación y comparación de dos metodologías de inteligencia artificial. Aunque las técnicas desarrolladas podrían aplicarse a otros tipos de fraude digital, el enfoque específico se centra en la identificación de contenido noticioso falso o engañoso. La primera metodología explora el uso de algoritmos metaheurísticos, incluyendo Recocido Multiarranque (MSA), Búsqueda Dispersa (SS), Búsqueda en Vecindades Variables (VNS), Algoritmo Genético (GA) y Optimización por Enjambre de Partículas (PSO), sobre una representación TF-IDF. La segunda, adoptada tras los hallazgos iniciales, se basa en el ajuste fino (fine-tuning) de un modelo de lenguaje Transformer pre-entrenado (DistilBERT). Para el entrenamiento, se construyó un corpus unificando cuatro conjuntos de datos públicos en español y datos extraídos mediante extracción web del portal satírico “El Deforma”, resultando en más de 61,000 noticias. Se implementó un cuidadoso proceso de calibración de hiperparámetros para ambos enfoques, utilizando una división de datos estratificada de 70 % para entrenamiento, 10 % para validación y 20 % para pruebas. El rendimiento fue evaluado con métricas como Exactitud, Precisión, Exhaustividad y F1-Score. Finalmente, el modelo Transformer, que demostró una eficacia superior, fue integrado en una aplicación web funcional desarrollada con Flask y contenerizada con Docker, capaz de analizar URLs en tiempo real. Los resultados validan la metodología de ajuste fino como una solución de vanguardia para combatir la desinformación, superando a los enfoques metaheurísticos en esta tarea.

Palabras clave: Detección de noticias falsas, desinformación, modelos de lenguaje, Transformers, DistilBERT, algoritmos metaheurísticos, procesamiento de lenguaje natural.

Abstract

This project addresses the challenge of fake news detection in Spanish through the application and comparison of two artificial intelligence methodologies. Although the developed techniques could be applied to other types of digital fraud, the specific focus centers on identifying false or misleading news content. The first explores the use of metaheuristic algorithms, including Multi-Start Simulated Annealing (MSA), Scatter Search (SS), Variable Neighborhood Search (VNS), Genetic Algorithm (GA), and Particle Swarm Optimization (PSO), applied to a TF-IDF representation. The second, adopted following initial findings, is based on fine-tuning a pre-trained Transformer language model (DistilBERT). For training, a corpus was constructed by unifying four public Spanish datasets and data extracted through web scraping from the satirical portal “El Deforma”, resulting in over 61,000 news articles. A careful hyperparameter calibration process was implemented for both approaches, using a stratified data split of 70 % for training, 10 % for validation, and 20 % for testing. Performance was evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score. Finally, the Transformer model, which demonstrated superior efficacy, was integrated into a functional web application developed with Flask and containerized with Docker, capable of analyzing URLs in real-time. The results validate the fine-tuning methodology as a state-of-the-art solution for combating misinformation, outperforming metaheuristic approaches in this task.

Keywords: Fake news detection, misinformation, language models, Transformers, DistilBERT, metaheuristic algorithms, natural language processing.

Dedicatoria

// Esta sección se presenta en la versión final de su ICR. Son frases cuyo objetivo es otorgar una mención especial a las personas que te han motivado durante tu ICR.

Agradecimientos

// Esta sección se presenta en la versión final de su ICR. Son frases cuyo objetivo es plasmar el apoyo moral, físico, económico y/o emocional que recibió de las personas o instituciones durante la elaboración de todo su proyecto.

Índice general

Índice general	VII
Índice de figuras	XIII
Índice de tablas	XV
Glosario	XIX
1. Introducción	1
1.1. Planteamiento del problema	1
1.1.1. El desafío específico del español: enfoque multiregional	3
1.1.2. La complejidad técnica del problema	5
1.2. Motivación	5
1.2.1. Impacto personal y social observado	5
1.2.2. Vulnerabilidad de poblaciones específicas	6
1.2.3. Urgencia tecnológica	6
1.3. Justificación	7
1.3.1. Brecha tecnológica en recursos para el español	7
1.3.2. Novedad metodológica: enfoque evolutivo comparativo	7
1.3.3. Contribución científica multidimensional	8
1.4. Objetivos	9
1.5. Alcance y limitaciones	9
1.5.1. Alcance	9
1.5.2. Limitaciones	10
1.6. Contribuciones esperadas	11
1.6.1. Contribuciones teóricas	11
1.6.2. Contribuciones prácticas	11
1.6.3. Contribuciones sociales	12
1.7. Organización del documento	12
2. Marco Teórico	13
2.1. Detección de noticias falsas: fundamentos y extensibilidad	13
2.1.1. Impacto social y desafíos de la desinformación	14
2.1.2. El problema de las heurísticas cognitivas	14

2.1.3.	Diferenciación técnica y transferibilidad: noticias falsas vs. fraude digital	14
2.2.	Representación de texto: de TF-IDF a Embeddings Contextuales . . .	16
2.2.1.	Bolsa de palabras (Bag of Words): primer acercamiento metodológico	16
2.2.2.	TF-IDF (Term Frequency-Inverse Document Frequency) . . .	17
2.2.3.	Fundamentos matemáticos de TF-IDF	18
2.2.4.	Limitaciones de los métodos clásicos	18
2.3.	La revolución Transformer y los Modelos de Lenguaje modernos . . .	19
2.3.1.	La arquitectura Transformer: fundamentos	19
2.3.2.	BERT y la era del preentrenamiento bidireccional	19
2.3.3.	Aplicaciones en la detección de noticias falsas en español . . .	19
2.3.4.	Grandes Modelos de Lenguaje (LLMs) y nuevos paradigmas .	20
2.3.5.	El doble rol de los LLMs en detección	20
2.4.	Optimización metaheurística en la detección de fraude	21
2.4.1.	Fundamentos de las metaheurísticas	21
2.4.2.	Aplicaciones en la detección de noticias falsas	28
2.4.3.	Metaheurísticas para la detección de fraude financiero	28
2.4.4.	Algoritmos híbridos modernos	29
2.5.	Síntesis del marco teórico	29
3.	Estado del arte	31
3.1.	Metodología de búsqueda bibliográfica	31
3.2.	Panorama de la investigación en detección de noticias falsas y Modelos de Lenguaje	32
3.2.1.	Fundamentos metodológicos y construcción de corpus	33
3.2.2.	Evolución de los Modelos de Lenguaje en la detección de desinformación	34
3.2.3.	Técnicas de optimización metaheurística en la detección de desinformación	34
3.3.	Algoritmos metaheurísticos: fundamentos teóricos y aplicaciones prácticas	35
3.3.1.	Algoritmos Genéticos (GA): evolución artificial para la optimización	37
3.3.2.	Optimización por Enjambre de Partículas (PSO): inteligencia colectiva	37
3.3.3.	Recocido Simulado: física estadística para la optimización . . .	37
3.3.4.	Búsqueda Dispersa (Scatter Search): combinación determinística	37
3.3.5.	Búsqueda en Vecindades Variables (VNS): exploración sistemática	37
3.4.	Hiperparámetros en modelos de aprendizaje automático: fundamentos y optimización	38
3.4.1.	Naturaleza y definición de hiperparámetros	38
3.4.2.	Importancia crítica en la detección de noticias falsas	38

3.4.3.	Desafíos en la búsqueda de la configuración óptima	39
3.4.4.	Herramientas modernas: Keras Tuner y automatización	39
3.4.5.	Beneficios combinados: Modelos de Lenguaje y herramientas de optimización	39
3.5.	Análisis de la literatura relevante	40
3.5.1.	El desafío fundamental de los datos: creación y curación de corpus en español	40
3.5.2.	Revolución de los Modelos de Lenguaje y arquitecturas Transformer	40
3.5.3.	Optimización y metaheurísticas en la detección: enfoques innovadores	41
3.5.4.	Aplicación de metaheurísticas por tipo de detección	41
3.5.5.	Perspectivas interdisciplinarias y análisis social	41
3.5.6.	Detección de fraude: extensión más allá de las noticias	41
3.6.	Síntesis y perspectivas futuras	41
4.	Metodología	45
4.1.	Visión general del proceso metodológico	45
4.1.1.	Esquema visual del flujo de trabajo general del sistema	46
4.2.	Definición y distinción de conceptos fundamentales	47
4.2.1.	Distinción entre noticia falsa y bulo	47
4.2.2.	Taxonomía de la desinformación	48
4.2.3.	Justificación de la unificación terminológica	48
4.2.4.	Esquema de clasificación: enfoque binario	49
4.2.5.	Fronteras difusas en la clasificación de veracidad	50
4.3.	Construcción del corpus unificado	51
4.3.1.	Fuentes de datos académicas	51
4.3.2.	Análisis comparativo detallado de los corpus utilizados	53
4.3.3.	Proceso de unificación y estandarización	56
4.3.4.	Ampliación del corpus mediante extracción web	56
4.3.5.	Corpus final y estrategia de división	58
4.4.	Enfoque 1: detección mediante algoritmos metaheurísticos	63
4.4.1.	Preprocesamiento y representación textual	63
4.4.2.	Algoritmos metaheurísticos implementados	64
4.4.3.	Función de evaluación y clasificación	67
4.4.4.	Reducción de dimensionalidad	68
4.5.	Enfoque 2: detección mediante modelo Transformer	68
4.5.1.	Selección y justificación del modelo	68
4.5.2.	Infraestructura computacional	69
4.5.3.	Configuración experimental y optimización de hiperparámetros	69
4.6.	Metodología de evaluación comparativa	71
4.6.1.	Protocolo de evaluación	71
4.6.2.	Fundamentos de la matriz de confusión	71

4.6.3.	Marco de métricas de rendimiento	72
4.6.4.	Criterios de selección del mejor modelo	76
4.6.5.	Análisis de matrices de confusión	76
4.6.6.	Reporte de resultados	77
4.7.	Infraestructura computacional y herramientas	77
4.7.1.	Entorno de desarrollo para algoritmos metaheurísticos	77
4.7.2.	Stack tecnológico completo	77
4.8.	Consideraciones éticas y de privacidad	78
4.8.1.	Marco Ético de Desarrollo	78
4.8.2.	Limitaciones Declaradas y Uso Responsable	78
4.9.	Validación y Reproducibilidad	79
4.9.1.	Ecosistema de Artefactos de Reproducibilidad	79
4.9.2.	Solución Lista para Producción	79
5.	Resultados, Evaluación Comparativa e Implementación	81
5.1.	Validación del flujo de procesamiento y representaciones	81
5.1.1.	Características del corpus unificado final	81
5.1.2.	Implementación y configuración de algoritmos metaheurísticos	83
5.1.3.	Pseudocódigos de los algoritmos implementados	83
5.1.4.	Visualizaciones de resultados por algoritmo	88
5.1.5.	Contextualización con investigación publicada	96
5.1.6.	Análisis comparativo de resultados	97
5.1.7.	Limitaciones fundamentales y justificación para evolución . . .	98
5.1.8.	Síntesis y transición	99
5.2.	Resultados del enfoque Transformer: DistilBERT multilingüe	99
5.2.1.	Marco experimental y evolución del desarrollo	100
5.2.2.	Configuración del modelo DistilBERT optimizado	101
5.2.3.	Proceso de optimización y búsqueda de hiperparámetros . . .	101
5.2.4.	Análisis de convergencia y control de sobreajuste	102
5.2.5.	Evolución experimental: versiones de desarrollo	103
5.2.6.	Pseudocódigo del algoritmo de entrenamiento DistilBERT . .	107
5.2.7.	Resultados finales y comparación	109
5.3.	Evaluación comparativa integral: metaheurísticas vs. Transformers . .	110
5.3.1.	Comparación de rendimiento cuantitativo	110
5.3.2.	Análisis multidimensional de paradigmas	110
5.3.3.	Justificación de la selección del modelo final	111
5.3.4.	Valor científico del enfoque comparativo	112
5.4.	Análisis de errores y limitaciones del modelo final	112
5.4.1.	Marco teórico para el análisis de errores	112
5.4.2.	Metodología propuesta para análisis cualitativo	112
5.4.3.	Tipos de errores esperados según la literatura	113
5.4.4.	Limitaciones reconocidas del modelo	114
5.4.5.	Conclusiones sobre limitaciones y direcciones futuras	114

5.5. Implementación de prototipos funcionales y demostración práctica . .	115
5.5.1. Arquitectura general del sistema	115
5.5.2. Prototipo 1: Analizador basado en metaheurísticas	118
5.5.3. Prototipo 2: Analizador basado en Modelos Transformer (ver- sión final)	119
5.5.4. Interfaz de usuario y casos de uso	120
5.5.5. Impacto práctico y robustez del sistema	121
5.5.6. Consideraciones de despliegue en producción	123
5.5.7. Síntesis de implementación y viabilidad	124
6. Conclusiones y Trabajo Futuro	125
6.1. Limitaciones del trabajo	126
6.2. Trabajo futuro y líneas de investigación	126
A. Anexo 1	129
Bibliografía	130

Índice de figuras

1.1. Mapa Conceptual 1: Taxonomía del problema de desinformación, con enfoque en noticias falsas y su extensibilidad hacia el fraude digital. .	3
1.2. Mapa Conceptual 2: Estrategias tecnológicas para la detección de fraude digital.	6
2.1. Flujo del proceso de optimización para la detección de noticias falsas.	27
3.1. Mapa Conceptual 3: Artículos relacionados que incorporan Modelos de Lenguaje.	34
3.2. Mapa Conceptual 4: Métodos de optimización y metaheurísticas aplicadas.	36
3.3. Mapa Conceptual 5: Clasificación de artículos por enfoque metodológico.	40
3.4. Mapa Conceptual 6: Artículos que son revisiones o están relacionados al análisis de contenido y detección de fraude digital.	43
4.1. Metodología propuesta que aborda la problemática combinando Algoritmos Metaheurísticos y Modelos de Lenguaje.	46
4.2. Arquitectura general del sistema de detección de noticias falsas, mostrando el flujo completo desde la construcción del corpus hasta la implementación de la aplicación web.	47
4.3. Representación gráfica del proceso de balanceo del corpus: comparación entre distribución inicial desbalanceada y distribución final equilibrada. La imagen muestra cómo la extracción web de 9,000 noticias falsas permitió alcanzar un equilibrio óptimo de 49.8 % noticias falsas vs 50.2 % noticias reales.	59
5.1. Evolución de la convergencia del algoritmo MSA mostrando el progreso gradual a través de los 31 niveles de temperatura desde 1000 hasta 1.24.	89
5.2. Matriz de confusión para MSA en el conjunto de pruebas, evidenciando la baja especificidad (33 %) y el sesgo hacia la clasificación como noticias reales.	90
5.3. Convergencia eficiente del algoritmo SS en solo 10 iteraciones, mostrando mejoras progresivas en las iteraciones 4, 5 y 7 hasta estabilizarse en 0.6630.	91

5.4. Matriz de confusión para SS demostrando mejor balance que MSA con especificidad del 40 % y excelente generalización.	91
5.5. Evolución darwiniana del algoritmo GA a lo largo de 20 generaciones, evidenciando progreso sostenido desde 0.6198 hasta 0.7090 con logros evolutivos significativos.	92
5.6. Matriz de confusión para GA mostrando el mejor balance global con especificidad líder del 48 % y rendimiento sólido en ambas clases. . . .	93
5.7. Progreso sistemático del algoritmo VNS a través de 20 iteraciones con cambios efectivos de vecindario, mostrando saltos significativos en las iteraciones 6 y 14.	94
5.8. Matriz de confusión para VNS destacando la excelente exhaustividad del 89 % para detección de noticias reales con especificidad competitiva del 41 %.	94
5.9. Convergencia problemática del algoritmo PSO evidenciando estancamiento prematuro en la iteración 7-8 y exploración insuficiente del espacio de búsqueda.	95
5.10. Matriz de confusión para PSO revelando el comportamiento extremo problemático con especificidad crítica del 15 % y sesgo severo hacia la clase mayoritaria.	96
5.11. Evolución de la exactitud y pérdida durante el entrenamiento del modelo DistilBERT final. Las líneas azul y roja muestran la convergencia en entrenamiento y validación respectivamente. La estrella dorada marca la mejor época (13), después de la cual se observa el inicio del sobreajuste con una separación creciente entre las curvas.	102
5.12. Evolución de exactitud a través de las versiones experimentales. . . .	106
5.13. Diagrama de componentes del sistema mostrando la arquitectura modular de cuatro capas implementada para ambos prototipos de detección de noticias falsas.	116
5.14. Diagrama de secuencia mostrando el flujo temporal de procesamiento desde la entrada del usuario hasta la respuesta de clasificación del sistema. . . .	117
5.15. Captura de pantalla de la aplicación analizando una noticia real. . . .	120
5.16. Captura de pantalla de la aplicación detectando una noticia falsa basada en su contenido.	121
5.17. Captura de pantalla de la aplicación detectando una página de fraude digital.	122

Índice de tablas

2.1. Diferencias técnicas entre dominios de desinformación	15
2.2. Comparación de algoritmos metaheurísticos implementados	26
3.1. Contribuciones fundamentales en metodología y construcción de corpus para español.	33
3.2. Evolución de modelos de lenguaje aplicados a la detección de desinformación.	35
3.3. Contribuciones metodológicas en optimización metaheurística para detección.	36
3.4. Algoritmos metaheurísticos: características fundamentales y aplicaciones en detección de noticias falsas.	38
3.5. Aplicación de metaheurísticas por tipo de detección en la literatura revisada.	42
4.1. Corpus académicos utilizados para la construcción del conjunto de datos unificado.	52
4.2. Comparación exhaustiva de características de los corpus utilizados en la construcción del conjunto de datos unificado.	53
4.3. Fases del proceso de extracción web implementado para “El Deforma”.	57
4.4. Referencias bibliográficas completas de los corpus académicos utilizados.	57
4.5. Composición final del corpus unificado después del procesamiento completo.	58
4.6. División estratificada del corpus para entrenamiento y evaluación.	62
4.7. Algoritmos metaheurísticos implementados y sus fundamentos conceptuales.	64
4.8. Configuración de parámetros del algoritmo MSA.	65
4.9. Configuración de parámetros del algoritmo SS.	65
4.10. Configuración de parámetros del algoritmo GA.	66
4.11. Configuración de parámetros del algoritmo VNS.	66
4.12. Configuración de parámetros del algoritmo PSO.	67
4.13. Configuración del proceso de reducción de dimensionalidad.	68
4.14. Comparación de modelos BERT optimizados para la tarea de clasificación.	68

4.15. Especificaciones del hardware utilizado para entrenamiento de DistilBERT.	69
4.16. Configuración de parámetros base para el entrenamiento de DistilBERT.	70
4.17. Estrategias de regularización implementadas para controlar overfitting.	70
4.18. Protocolo de evaluación implementado para ambos paradigmas.	71
4.19. Definición de categorías de la matriz de confusión en el contexto de detección de noticias falsas.	71
4.20. Estructura de la matriz de confusión para clasificación binaria.	72
4.21. Marco de métricas de evaluación para clasificación binaria de noticias falsas.	73
4.22. Criterios multi-dimensionales para selección del modelo óptimo.	76
4.23. Estructura del reporte de resultados comparativo.	77
4.24. Especificaciones del entorno de desarrollo para algoritmos metaheurísticos.	77
4.25. Stack tecnológico completo utilizado en el desarrollo del proyecto.	78
4.26. Marco ético implementado para el desarrollo responsable de la herramienta.	78
4.27. Artefactos generados para facilitar la reproducibilidad completa del estudio.	79
5.1. Corpus académicos utilizados para la construcción del conjunto de datos unificado.	82
5.2. Configuración detallada de parámetros para los cinco algoritmos metaheurísticos implementados.	83
5.3. Función de evaluación común utilizada por todos los algoritmos metaheurísticos.	84
5.4. Pseudocódigo completo del algoritmo Multi-Start Simulated Annealing (MSA).	85
5.5. Pseudocódigo completo del algoritmo Scatter Search (SS).	86
5.6. Pseudocódigo completo del Algoritmo Genético (GA).	86
5.7. Pseudocódigo completo del algoritmo Variable Neighborhood Search (VNS).	87
5.8. Pseudocódigo completo del algoritmo Particle Swarm Optimization (PSO).	88
5.9. Resultados comparativos finales de los cinco algoritmos metaheurísticos implementados usando métricas macro promedio.	97
5.10. Análisis de fortalezas y debilidades de cada algoritmo metaheurístico basado en las métricas obtenidas.	97
5.11. Comparación de rendimiento entre enfoques metaheurísticos y Modelos de Lenguaje usando métricas macro.	98
5.12. Composición del corpus expandido utilizado para el entrenamiento de DistilBERT.	100
5.13. Configuración de la arquitectura del modelo DistilBERT implementado.	101
5.14. Técnicas de regularización implementadas en la configuración final.	101

5.15. Resumen de versiones experimentales de DistilBERT con evolución de estrategias y resultados reales del desarrollo.	104
5.16. Visualización comparativa de convergencia y métricas para las versiones experimentales más significativas con DistilBERT.	105
5.17. Evolución de configuraciones y resultados entre versiones experimentales.	105
5.18. Pseudocódigo simplificado del algoritmo DistilBERT final (V7). . . .	108
5.19. Métricas finales del modelo DistilBERT optimizado.	109
5.20. Comparación DistilBERT vs. mejor algoritmo metaheurístico.	109
5.21. Comparación cuantitativa final entre el mejor algoritmo metaheurístico y el modelo Transformer optimizado.	110
5.22. Estrategias propuestas para abordar limitaciones identificadas.	114

Glosario

API Application Programming Interface

Atención (Attention) Mecanismo que permite a los modelos enfocarse en partes específicas de la entrada al procesar cada elemento. En PLN, permite que el modelo “atienda” a palabras relevantes al procesar una palabra específica.

AUC Area Under the Curve

AUC-ROC Área bajo la curva ROC (AUC de ROC). Mide la capacidad del modelo para distinguir entre clases. Un valor de 1.0 indica un clasificador perfecto, mientras que 0.5 indica rendimiento aleatorio.

BERT Bidirectional Encoder Representations from Transformers

BETO Bidirectional Encoder Representations from Transformers for Spanish

Bolsa de Palabras (Bag of Words) Modelo de representación de texto que describe la ocurrencia de palabras en un documento, ignorando el orden y la estructura gramatical. Cada documento se representa como un vector donde cada dimensión corresponde a una palabra del vocabulario y su valor indica la frecuencia de aparición de esa palabra. Es importante para técnicas como TF-IDF y constituye la base para muchos algoritmos de clasificación de texto tradicionales.

Bulo Información falsa que circula ampliamente, especialmente en redes sociales y medios digitales, con el propósito de engañar a la audiencia. A diferencia de las noticias falsas, los bulos pueden no tener formato periodístico y suelen propagarse de manera viral. Incluye rumores, teorías conspirativas, información médica falsa, y contenido que apela más a las emociones que a los hechos verificables.

CNN Redes Neuronales Convolucionales

Convergencia Proceso por el cual un algoritmo de optimización se acerca progresivamente a una solución óptima o cerca del óptimo. Se evalúa observando la estabilización de la función objetivo a lo largo de las iteraciones.

Corpus Colección grande y estructurada de textos utilizados para investigación lingüística o entrenamiento de modelos de PLN. En este contexto, se refiere al conjunto unificado de noticias en español.

CPU Central Processing Unit

CSS Cascading Style Sheets

CSV Comma-Separated Values

Dataset Conjunto estructurado de datos utilizado para entrenar, validar y probar modelos de machine learning. En esta investigación, se refiere al corpus de noticias etiquetadas como verdaderas o falsas.

Deepfake Contenido multimedia (video, audio, imágenes) generado o manipulado usando inteligencia artificial, especialmente técnicas de aprendizaje profundo, para hacer que parezca que alguien dijo o hizo algo que nunca ocurrió realmente.

Desinformación Difusión intencional de información falsa o engañosa con el propósito específico de manipular la opinión pública, influir en decisiones políticas o sociales, o causar daño. Se caracteriza por ser un proceso activo y deliberado de creación y distribución de contenido falso.

Desinformación (Misinformation) Información incorrecta que se comparte sin intención maliciosa. A diferencia de la desinformación, quienes comparten misinformación no tienen conocimiento de que la información es falsa y actúan de buena fe.

DistilBERT Distilled Bidirectional Encoder Representations from Transformers

DL Deep Learning

Docker Plataforma para contenerización que empaqueta una aplicación y sus dependencias en un entorno aislado y reproducible.

Dropout Una técnica de regularización utilizada en redes neuronales para prevenir el sobreajuste. Consiste en desactivar aleatoriamente una fracción de las neuronas de una capa durante el entrenamiento, obligando al modelo a aprender patrones más robustos.

Embedding Representación vectorial densa de palabras, frases o documentos en un espacio multidimensional donde la distancia entre vectores refleja similitud semántica. Ejemplos incluyen Word2Vec, GloVe, y FastText.

Espacio de Búsqueda Conjunto de todas las posibles configuraciones de hiperparámetros que pueden ser exploradas durante el proceso de optimización. Define los límites y restricciones para cada hiperparámetro.

Exactitud (Accuracy) Proporción de predicciones correctas sobre el total de predicciones. Se calcula como: $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$.

Exploración Capacidad de un algoritmo de búsqueda para investigar regiones no exploradas del espacio de soluciones, evitando quedar atrapado en óptimos locales.

Explotación Capacidad de un algoritmo para refinar y mejorar soluciones prometedoras encontradas, concentrando la búsqueda en regiones del espacio que han mostrado buenos resultados.

F1-Score Media armónica entre precisión y recall. Se calcula como: $\text{F1} = 2 \times (\text{Precisión} \times \text{Recall}) / (\text{Precisión} + \text{Recall})$. Proporciona una medida equilibrada del rendimiento del clasificador.

Fine-tuning Proceso de ajustar un modelo pre-entrenado en una tarea específica utilizando un conjunto de datos más pequeño y especializado. Permite aprovechar el conocimiento general del modelo pre-entrenado para tareas particulares.

Flask Microframework de Python utilizado para construir aplicaciones web y APIs RESTful.

FN False Negative (Falso Negativo)

FP False Positive (Falso Positivo)

Fraude Digital Conjunto amplio de actividades maliciosas realizadas a través de medios digitales con el objetivo primario de obtener beneficios económicos ilícitos, información personal sensible, o acceso no autorizado a recursos. Incluye estafas en línea, phishing, fraude financiero digital, esquemas Ponzi digitales, estafas de inversión, fraude laboral en línea, y otras formas de engaño que explotan plataformas y tecnologías digitales. A diferencia de las noticias falsas, que buscan influencia social, el fraude digital se enfoca en la obtención directa de beneficios materiales o acceso a activos. Su “éxito” se mide por la cantidad de dinero o información obtenida, no por el alcance viral o la influencia en opinión pública.

Función Objetivo Función matemática que define el criterio a optimizar en un problema. En el contexto de esta investigación, representa la métrica de rendimiento del modelo que se busca maximizar o minimizar.

GA Genetic Algorithm (Algoritmo Genético)

Gap de Generalización (Generalization Gap) La diferencia entre el rendimiento de un modelo en el conjunto de entrenamiento y en un conjunto de validación o prueba. Un gap pequeño indica que el modelo generaliza bien, mientras que un gap grande es un indicador de sobreajuste.

Gemini A Family of Highly Capable Multimodal Models

GPT Generative Pre-trained Transformer

GPU Graphics Processing Unit

Hiperparámetro Parámetro de configuración del modelo que se establece antes del entrenamiento y no se aprende durante el proceso de entrenamiento. Ejemplos incluyen la tasa de aprendizaje, el número de épocas, y el tamaño del batch.

HTML HyperText Markup Language

HTTP HyperText Transfer Protocol

IA Inteligencia Artificial

ICR Idónea Comunicación de Resultados

Información Maliciosa (Malinformation) Información genuina que se comparte con intención de causar daño, como filtraciones de información privada, discursos de odio, o acoso. Aunque la información base puede ser verdadera, su uso es malicioso.

JSON JavaScript Object Notation

KerasTuner Una biblioteca de software para la optimización de hiperparámetros de modelos de aprendizaje automático, especialmente en TensorFlow, que permite explorar eficientemente diferentes combinaciones de parámetros.

Lematización Proceso más sofisticado que el stemming que reduce las palabras a su forma canónica o lemma, considerando el contexto morfológico y sintáctico. Por ejemplo, “mejor” se lematiza a “bueno”.

LLaMA Large Language Model Meta AI

LSTM Long Short-Term Memory

Matriz de Confusión Tabla que describe el rendimiento de un modelo de clasificación mostrando las predicciones correctas e incorrectas para cada clase. Permite calcular métricas detalladas de rendimiento.

Metaheurística Estrategia de alto nivel para guiar y modificar otras heurísticas con el objetivo de producir soluciones de alta calidad para problemas de optimización. Incluye técnicas como algoritmos genéticos, optimización por enjambre de partículas, y recocido simulado.

ML Machine Learning

MSA Multi-Start Simulated Annealing (Recocido Multiarranque)

n-gramas Secuencias contiguas de n elementos (palabras o caracteres) de un texto, que capturan el contexto local y las dependencias secuenciales. En esta tesis, se utilizan n -gramas de palabras como características para la representación TF-IDF.

NLP Natural Language Processing

Noticia Falsa Información deliberadamente fabricada que se presenta como contenido periodístico legítimo, pero que contiene datos falsos, inexactos o engañosos. Su objetivo principal es manipular la opinión pública, influir en decisiones políticas o sociales, o distorsionar la percepción de eventos actuales. Se caracteriza por imitar el formato y estilo de medios de comunicación establecidos, utilizando técnicas periodísticas aparentemente profesionales para ganar credibilidad. A diferencia del fraude digital, su “éxito” se mide por el alcance social y la influencia en la opinión pública, no por beneficios económicos directos.

Palabras Vacías (Stop Words) Palabras comunes en un idioma que generalmente se filtran durante el preprocesamiento de texto porque no aportan significado semántico significativo. Ejemplos en español: “el”, “la”, “de”, “que”, “y”.

Parada Temprana (Early Stopping) Un mecanismo de regularización que detiene el proceso de entrenamiento cuando el rendimiento del modelo en un conjunto de validación deja de mejorar. Se utiliza para prevenir el sobreajuste y reducir el tiempo de cómputo.

PLN Procesamiento del Lenguaje Natural

Precisión Métrica que mide la proporción de predicciones positivas que fueron correctas. Se calcula como: $\text{Precisión} = \text{TP} / (\text{TP} + \text{FP})$.

Programación de Trabajos (Job Shop Scheduling) Un problema de optimización combinatoria que implica la asignación de tareas a recursos en un tiempo determinado. Se menciona en la literatura como una perspectiva para modelar la optimización del flujo de datos en la detección de noticias falsas.

PSO Particle Swarm Optimization

RAM Random Access Memory

RESTful Estilo de arquitectura de software para sistemas hipermedia distribuidos, comúnmente utilizado para el diseño de APIs web.

RNN Redes Neuronales Recurrentes

RoBERTa Robustly Optimized BERT Pretraining Approach

ROC Receiver Operating Characteristic

SA Simulated Annealing (Recocido Simulado)

Sensibilidad (Recall) Métrica que mide la proporción de casos positivos reales que fueron correctamente identificados. Se calcula como: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$.

Sesgo (Bias) Error sistemático en las predicciones del modelo que puede deberse a suposiciones erróneas en el algoritmo de aprendizaje, datos de entrenamiento no representativos, o prejuicios inherentes en el dataset.

Sobreajuste (Overfitting) Fenómeno donde un modelo aprende demasiado específicamente los datos de entrenamiento, perdiendo capacidad de generalización a datos nuevos no vistos durante el entrenamiento.

SS Scatter Search

Stemming Técnica de reducción de palabras a su raíz o stem mediante la eliminación de sufijos. Por ejemplo, “corriendo”, “corrió”, “correr” se reducen a “corr”.

Subajuste (Underfitting) Fenómeno donde un modelo es demasiado simple para capturar la complejidad subyacente de los datos, resultando en bajo rendimiento tanto en entrenamiento como en validación.

SVM Support Vector Machine

Tasa de Aprendizaje (Learning Rate) Un hiperparámetro que determina el tamaño de los pasos que un algoritmo de optimización (como el descenso de gradiente) toma para actualizar los parámetros del modelo en cada iteración. Una tasa de aprendizaje ultra-baja se utilizó para una convergencia más controlada y estable.

TF-IDF Term Frequency-Inverse Document Frequency

TN True Negative (Verdadero Negativo)

Tokenización Proceso de dividir texto en unidades más pequeñas llamadas tokens, que pueden ser palabras, subpalabras, caracteres o n-gramas. Es el primer paso en el preprocesamiento de texto para análisis computacional.

TP True Positive (Verdadero Positivo)

Transformer Arquitectura de red neuronal basada en mecanismos de atención que ha revolucionado el PLN. Introdujo el concepto de atención multi-cabeza y eliminó la necesidad de procesamiento secuencial, permitiendo paralelización eficiente.

URL Uniform Resource Locator

Validación Cruzada Técnica de evaluación que divide el dataset en múltiples subconjuntos para entrenar y validar el modelo repetidamente, proporcionando una estimación más robusta del rendimiento.

Varianza Medida de cuánto varían las predicciones del modelo para diferentes conjuntos de entrenamiento. Alta varianza indica que el modelo es sensible a pequeños cambios en los datos de entrenamiento.

Verificación de Hechos (Fact-Checking) Proceso sistemático de investigación y verificación de afirmaciones factuales en contenido publicado. Incluye la consulta de fuentes primarias, expertos, y evidencia documental para determinar la veracidad de las afirmaciones.

VNS Variable Neighborhood Search

Web Scraping Técnica de extracción automatizada de datos de sitios web mediante programas que navegan y analizan el contenido HTML de las páginas web para recopilar información específica.

Capítulo 1

Introducción

1.1. Planteamiento del problema

La era digital y la interconexión global han generado un desafío sin precedentes: la propagación masiva de Desinformación. Este fenómeno, que abarca desde Noticia Falsas hasta diversas formas de Fraude Digital, representa una amenaza significativa para la estabilidad social, económica y democrática a nivel mundial.

Las Noticia Falsas se definen como información deliberadamente engañosa que simula ser periodismo auténtico [1]. Se difunden a través de redes sociales y medios digitales para manipular la opinión pública y el comportamiento de los individuos. La gravedad del problema se ha intensificado exponencialmente debido a la creciente sofisticación de los actores maliciosos, la democratización de herramientas para generar contenido mediante IA [2] y la velocidad sin precedentes con que la información se viraliza.

Distinción fundamental entre noticias falsas y fraude digital:

- **Noticia Falsas:** Información deliberadamente fabricada que imita el formato periodístico legítimo con el objetivo primario de *manipular la opinión pública*, influir en decisiones políticas o sociales, o distorsionar la percepción de eventos actuales. Su “éxito” se mide por el alcance y la influencia social.
- **Fraude Digital:** Actividades maliciosas digitales con el objetivo primario de *obtener beneficios económicos ilícitos* o acceso no autorizado a información o recursos. Incluye estafas de inversión, phishing, fraude financiero y esquemas de estafas piramidales digitales. Su “éxito” se mide por el beneficio económico obtenido.

¿Por qué son tareas de detección relacionadas pero distintas?

- **Objetivos diferentes:** Las noticias falsas buscan influencia social; el fraude digital busca beneficio económico.
- **Audiencias diferentes:** Las noticias falsas se dirigen a consumidores de noticias, mientras que el fraude digital apunta a potenciales víctimas económicas.

- **Métricas de éxito diferentes:** Viralidad frente a rentabilidad económica.
- **Patrones lingüísticos diferentes:** Las noticias falsas imitan el periodismo; el fraude digital utiliza técnicas de persuasión comercial y urgencia.

Enfoque de esta investigación: Este proyecto se centra en la detección de noticias falsas y desarrolla una metodología inherentemente transferible al fraude digital, ya que comparten características computacionales: naturaleza textual, clasificación binaria (legítimo/malicioso) y optimización de parámetros.

La investigación reciente documenta el impacto multidimensional de la desinformación, desde la distorsión de procesos democráticos [3] hasta la creación de pánico durante crisis sanitarias como la pandemia de COVID-19 [4]. De forma paralela, el fraude digital ha evolucionado hacia formas más sofisticadas que aprovechan vulnerabilidades técnicas y sesgos cognitivos humanos [5].

Desde la perspectiva de la detección automatizada, el problema se conceptualiza como una clasificación binaria: determinar si un contenido es **confiable** (verificado y legítimo) o **no confiable** (desinformativo). Aunque esta simplificación reduce la complejidad del espectro de veracidad, resulta práctica y efectiva para aplicaciones reales que requieren respuestas claras.

La desinformación contemporánea se manifiesta en múltiples modalidades con un impacto significativo en individuos, empresas y sociedades, como lo ilustra la Figura 1.1. Si bien este estudio se enfoca en noticias falsas, es relevante comprender el contexto más amplio del fraude digital para dimensionar la importancia de desarrollar metodologías transferibles:

- **Noticias falsas generadas por IA:** Contenido sintético creado por Grandes Modelos de Lenguaje (GPTs) que imita el estilo periodístico legítimo [6] - *Uno de los objetivos de este proyecto.*
- **Desinformación política dirigida:** Campañas coordinadas para influir en procesos electorales y en la opinión pública [7] - *Aplicable con la metodología propuesta.*
- **Fraude financiero digital:** Esquemas que explotan plataformas digitales y criptomonedas [8] - *Metodología transferible.*
- **Fraude laboral en línea:** Ofertas de empleo falsas para obtener información personal o financiera [9] - *Metodología transferible.*
- **Estafas de ingeniería social:** Técnicas sofisticadas que combinan información personal de redes sociales con narrativas convincentes - *Metodología transferible.*
- **Desinformación en salud:** Información médica falsa con consecuencias directas en la salud pública [10] - *Aplicable con la metodología propuesta.*
- **Deepfakes:** Contenido multimedia manipulado con técnicas de DL - *Requiere extensión multimodal.*

- **Bulos:** Información falsa que se propaga viralmente en redes sociales - *Aplicable con la metodología propuesta.*



Figura 1.1: Mapa Conceptual 1: Taxonomía del problema de desinformación, con enfoque en noticias falsas y su extensibilidad hacia el fraude digital.

1.1.1.1. El desafío específico del español: enfoque multiregional

El español, la cuarta lengua más hablada del mundo con más de 500 millones de hablantes nativos [11], presenta desafíos únicos para la detección automatizada de desinformación. A pesar de su importancia demográfica y económica, existe una notable escasez de recursos computacionales especializados para esta tarea en español, en comparación con los abundantes recursos disponibles para el inglés [12].

Variante del español utilizada: justificación metodológica

Esta investigación adopta un enfoque de español neutro multiregional, fundamentado en la integración estratégica de corpus que representan diferentes variantes geográficas y estilísticas del español. Esta decisión metodológica se justifica por:

- **Corpus español peninsular:** Los corpus de Posadas-Durán et al. [12] y Blanco-Fernández et al. [13] incluyen principalmente contenido de medios españoles.
- **Corpus latinoamericano:** El corpus de Acosta [11] incorpora fuentes internacionales con presencia del español de México y otros países latinoamericanos.

- **Contenido mexicano específico:** La inclusión de contenido de “El Deforma” (portal satírico mexicano) aporta características del español mexicano contemporáneo.
- **Variabilidad temporal:** Los corpus abarcan contenido desde 2018 hasta 2025, capturando la evolución del español digital en múltiples regiones.

Distribución geográfica del corpus unificado:

- **España (~70 %):** Principalmente del corpus político de Blanco-Fernández.
- **México (~20 %):** Contenido de “El Deforma” y fuentes mexicanas en otros corpus.
- **Multinacional (~10 %):** Corpus de Acosta con diversas fuentes internacionales.

Esta composición da como resultado un modelo que puede generalizar eficazmente a través de las principales variantes del español, sin estar sesgado hacia una región específica.

Ventajas del enfoque multiregional

- **Robustez lingüística:** El modelo entrenado es capaz de detectar desinformación independientemente de la variante regional del español.
- **Generalización mejorada:** Menor dependencia de modismos o construcciones específicas de una región.
- **Aplicabilidad universal:** El sistema resultante es útil para toda la comunidad hispanohablante.
- **Representatividad demográfica:** Refleja la realidad del consumo de noticias en español, que frecuentemente cruza fronteras nacionales.

Esta brecha de recursos se manifiesta en:

- **Escasez de corpus etiquetados:** Conjuntos de datos de entrenamiento limitados en español.
- **Variabilidad dialectal:** La diversidad regional del español presenta desafíos adicionales.
- **Contexto cultural específico:** Los patrones de desinformación varían según el contexto sociocultural.
- **Herramientas de detección limitadas:** Pocas soluciones tecnológicas disponibles para hispanohablantes.

1.1.2. La complejidad técnica del problema

La detección automatizada de Noticia Falsas es un problema técnico multifacético que integra múltiples disciplinas, como se esquematiza en la Figura 1.2. La literatura reciente [14] documenta que los desafíos incluyen:

- **Análisis semántico profundo:** Comprender el contexto y las implicaciones sutiles del contenido mediante PLN.
- **Detección de patrones estilométricos:** Identificar características lingüísticas que indiquen autoría maliciosa [15].
- **Procesamiento en tiempo real:** Analizar el volumen masivo de contenido generado diariamente usando ML.
- **Adaptación a contenido sintético:** Detectar texto generado por modelos de IA cada vez más sofisticados [16].
- **Robustez ante ataques adversariales:** Resistencia a intentos deliberados de evadir la detección.
- **Optimización de Hiperparámetros:** Calibración de parámetros del modelo para maximizar el rendimiento.

Dada la sofisticación de estas amenazas, se requieren soluciones tecnológicas igualmente avanzadas para combatir el fraude digital. Este trabajo se centra en el desarrollo de tales soluciones, con un enfoque particular en el idioma español y la comparación sistemática de paradigmas tecnológicos complementarios.

1.2. Motivación

La motivación de esta investigación se fundamenta en una combinación de experiencias personales y la identificación de una brecha crítica en la protección tecnológica de las comunidades hispanohablantes.

1.2.1. Impacto personal y social observado

Durante el desarrollo de esta investigación, se observaron múltiples casos cercanos de personas víctimas de fraude digital sofisticado. Estos incluyeron desde estafas de inversión disfrazadas de noticias financieras legítimas hasta esquemas de *phishing* que aprovechaban eventos noticiosos para parecer creíbles. Las víctimas, a menudo personas mayores o con menor familiaridad con la tecnología digital, sufrieron no solo pérdidas económicas significativas, sino también un profundo impacto psicológico.



Figura 1.2: Mapa Conceptual 2: Estrategias tecnológicas para la detección de fraude digital.

1.2.2. Vulnerabilidad de poblaciones específicas

La investigación en psicología cognitiva aplicada a la desinformación [5] ha demostrado que ciertos grupos demográficos son particularmente vulnerables:

- **Adultos mayores:** Mayor susceptibilidad a heurísticas de credibilidad basadas en la autoridad percibida.
- **Poblaciones con menor alfabetización digital:** Capacidad limitada para evaluar la legitimidad de las fuentes en línea.
- **Comunidades con acceso limitado a información:** Mayor dependencia de las redes sociales como fuente principal de noticias.
- **Hablantes nativos de español:** Menor disponibilidad de herramientas de verificación en su idioma.

1.2.3. Urgencia tecnológica

El rápido avance de los modelos generativos de IA, como GPT-3 [17] y sus sucesores, ha reducido las barreras para la creación de contenido falso convincente. Esta “democratización” de la capacidad de generar desinformación [6] crea una urgencia imperativa para desarrollar defensas tecnológicas igualmente sofisticadas. Impulsado

por esta necesidad, el presente trabajo se centra en desarrollar, comparar y validar métodos computacionales avanzados para la detección de noticias falsas, con el objetivo de contribuir a la protección de las poblaciones más vulnerables.

1.3. Justificación

Esta investigación se justifica por la brecha tecnológica existente, la novedad de su enfoque metodológico y la necesidad social de herramientas especializadas para el español.

1.3.1. Brecha tecnológica en recursos para el español

El español, con más de 500 millones de hablantes nativos, representa un vasto ecosistema digital históricamente desatendido en herramientas para la detección de desinformación. Mientras que para el inglés existen múltiples conjuntos de datos a gran escala como LIAR, FakeNewsNet y CREDBANK [18], los recursos equivalentes en español son limitados y fragmentados. El estado del arte actual se basa principalmente en cuatro corpus:

- **Corpus de Acosta (2019):** 598 noticias [11].
- **Spanish Fake News Corpus:** 971 noticias [12].
- **Corpus de Tretiakov (2022):** 1,958 noticias [19].
- **Spanish Political Fake News:** 57,000+ noticias [13].

Esta fragmentación dificulta el desarrollo de modelos robustos y generalizables.

1.3.2. Novedad metodológica: enfoque evolutivo comparativo

La novedad principal de esta investigación radica en su enfoque evolutivo, que compara sistemáticamente dos paradigmas de la Inteligencia Artificial:

Paradigma clásico optimizado

- **Representación textual:** TF-IDF.
- **Optimización:** Algoritmos metaheurísticos para calibración de hiperparámetros.
- **Clasificador:** Modelos de aprendizaje automático tradicionales.
- **Ventajas:** Eficiencia computacional, interpretabilidad y menor dependencia de hardware especializado.

Paradigma de deep learning

- **Representación textual:** Embeddings contextuales dinámicos.
- **Modelo base:** DistilBERT preentrenado [20].
- **Técnica:** *Fine-tuning* con optimización de hiperparámetros.
- **Ventajas:** Comprensión semántica profunda y captura de relaciones contextuales complejas.

1.3.3. Contribución científica multidimensional

Esta investigación ofrece contribuciones en múltiples dimensiones:

Contribución a recursos lingüísticos

- **Corpus unificado:** Integración y estandarización de los principales corpus en español.
- **Metodología de unificación:** Protocolo replicable para integrar conjuntos de datos heterogéneos.
- **Benchmarking:** Establecimiento de líneas base comparativas para futuras investigaciones.

Contribución metodológica

- **Optimización metaheurística:** Aplicación sistemática de algoritmos bioinspirados para la calibración de hiperparámetros.
- **Análisis comparativo riguroso:** Evaluación exhaustiva con métricas múltiples y validación cruzada.
- **Transferencia tecnológica:** Implementación práctica en aplicaciones web containerizadas.

Contribución aplicada

- **Herramientas funcionales:** Aplicaciones web desplegables para análisis en tiempo real.
- **Código abierto:** Disponibilidad pública de implementaciones para la reproducibilidad.
- **Impacto social directo:** Herramientas accesibles para comunidades hispanohablantes.

1.4. Objetivos

Objetivo General

Desarrollar un método computacional basado en algoritmos metaheurísticos y modelos de lenguaje para detectar noticias falsas en español, con una metodología transferible a otros tipos de fraude digital.

Objetivos Específicos

- Recopilar y procesar un conjunto de datos diverso a partir de múltiples corpus en español, enriqueciéndolo mediante técnicas de *extracción web*.
- Implementar un sistema de detección inicial que utilice técnicas de Procesamiento del Lenguaje Natural (TF-IDF) y algoritmos metaheurísticos.
- Desarrollar un sistema de detección avanzado mediante el ajuste fino (*fine-tuning*) de un modelo de lenguaje profundo (DistilBERT), optimizando su rendimiento a través de la calibración de hiperparámetros.
- Realizar un análisis comparativo del rendimiento entre ambos enfoques, utilizando un conjunto completo de métricas de evaluación (Exactitud, Precisión, Exhaustividad y F1-Score).
- Desarrollar un prototipo de aplicación web funcional, contenerizada con **Docker**, para demostrar la aplicabilidad práctica del modelo de mayor rendimiento en el análisis de URLs.

1.5. Alcance y limitaciones

1.5.1. Alcance

Alcance lingüístico y cultural

- **Idioma objetivo:** El proyecto se centra exclusivamente en textos en español.
- **Dominio de aplicación:** Detección de noticias falsas, con extensibilidad a otros tipos de fraude digital.
- **Contexto geográfico:** Cobertura de múltiples países hispanohablantes.

Alcance técnico

- **Modalidad de datos:** Procesamiento exclusivo de contenido textual.
- **Tipo de clasificación:** Clasificación binaria supervisada (FALSO/REAL).

- **Arquitecturas evaluadas:** Comparación entre enfoques clásicos optimizados y modelos Transformer.
- **Implementación práctica:** Desarrollo de aplicaciones web funcionales y containerizadas.

Transferibilidad de la metodología

- **Fraude financiero y laboral:** La metodología es adaptable para detectar estafas en estos dominios.
- **Ingeniería social y desinformación temática:** Útil para detectar *phishing* y desinformación en áreas específicas.
- **Limitaciones de transferencia:** El contenido multimodal (videos, audios) requiere arquitecturas adicionales.

Alcance metodológico

- **Optimización metaheurística:** Aplicación de cinco algoritmos bioinspirados.
- **Validación experimental:** Uso de validación cruzada estratificada y métricas múltiples.
- **Reproducibilidad:** Documentación completa y código fuente disponible.

1.5.2. Limitaciones

Limitaciones de datos

- **Dependencia de corpus existentes:** El rendimiento está condicionado por la calidad de los datos disponibles.
- **Sesgos inherentes:** Posibles sesgos temporales, temáticos o geográficos en las fuentes.
- **Evolución del lenguaje:** Los modelos pueden no capturar patrones emergentes en la desinformación.
- **Etiquetado:** Dependencia de la calidad del etiquetado manual en los corpus originales.

Limitaciones técnicas

- **Recursos computacionales:** El entrenamiento de modelos Transformer requiere hardware especializado y tiempo considerable.

- **Escalabilidad en tiempo real:** Los prototipos no están optimizados para procesamiento de flujos masivos.
- **Generalización a otros fraudes:** La efectividad en otros tipos de fraude digital es inferencial.

Limitaciones operacionales

- **Herramienta de apoyo:** Los sistemas son un apoyo a la decisión, no un reemplazo del juicio humano.
- **Verificación absoluta:** La veracidad definitiva puede requerir investigación periodística.
- **Contexto dinámico:** Los modelos requieren actualización periódica.
- **Consideraciones éticas:** No se abordan explícitamente implicaciones de sesgo algorítmico.

Limitaciones de evaluación

- **Métricas tradicionales:** Pueden no capturar toda la complejidad del problema.
- **Validación temporal:** No se realiza con datos posteriores al entrenamiento.
- **Análisis de errores:** El análisis cualitativo es limitado por el volumen de datos.

1.6. Contribuciones esperadas

1.6.1. Contribuciones teóricas

- **Metodología comparativa:** Un marco sistemático para comparar paradigmas en la detección de desinformación.
- **Optimización metaheurística:** Aplicación de algoritmos bioinspirados para la calibración de modelos.

1.6.2. Contribuciones prácticas

- **Corpus unificado:** Un recurso consolidado para la investigación futura en español.
- **Herramientas funcionales:** Aplicaciones desplegadas para uso comunitario.
- **Código abierto:** Implementaciones reproducibles y extensibles.

1.6.3. Contribuciones sociales

- **Protección comunitaria:** Herramientas accesibles para comunidades hispanohablantes.
- **Democratización tecnológica:** Reducción de la brecha digital en herramientas de verificación.
- **Concientización:** Contribución a la alfabetización digital.

1.7. Organización del documento

Este documento se estructura de manera progresiva para guiar al lector a través de la investigación:

- **Capítulo 2 - Marco Teórico:** Establece los fundamentos de las metodologías utilizadas.
- **Capítulo 3 - Estado del Arte:** Revisa la literatura relevante e identifica las brechas de conocimiento.
- **Capítulo 4 - Metodología:** Detalla la construcción del corpus y la implementación de ambos paradigmas.
- **Capítulo 5 - Resultados, Evaluación Comparativa e Implementación:** Presenta y compara los resultados obtenidos por ambas metodologías y describe el desarrollo de una aplicación web funcional.
- **Capítulo 6 - Conclusiones y Trabajo Futuro:** Sintetiza los hallazgos y propone direcciones para investigaciones futuras.

Cada capítulo se basa en el anterior para mantener una coherencia narrativa y técnica, proporcionando la profundidad analítica necesaria para validar las contribuciones científicas y prácticas de esta investigación.

Capítulo 2

Marco Teórico

Esta sección describe las bases teóricas que sustentan las dos metodologías de detección de noticias falsas desarrolladas en este proyecto. Aunque las técnicas son aplicables a otros tipos de fraude digital, el enfoque se centra en la detección de desinformación periodística. Se abordan desde las técnicas clásicas de representación de texto y optimización metaheurística hasta los paradigmas de aprendizaje profundo que definen el estado del arte actual.

2.1. Detección de noticias falsas: fundamentos y extensibilidad

La detección de noticias falsas es un problema multifacético que requiere un enfoque interdisciplinario, con metodologías que pueden extenderse a otros tipos de fraude digital. La desinformación se define como información deliberadamente falsa o engañosa presentada como noticia legítima [1]. Esta problemática ha evolucionado con la aparición de las redes sociales y los medios digitales, donde la velocidad de propagación supera la capacidad de verificación tradicional.

En el contexto de la detección automatizada, se han explorado diversos enfoques. Das et al. [21] propusieron un marco de ensamblaje basado en incertidumbre e impulsado por heurísticas para detectar noticias falsas en tuits y artículos. Este enfoque parte de la premisa de que diferentes modelos poseen distintas fortalezas y que una combinación inteligente de predictores puede superar las limitaciones individuales.

El equipo de la Southern Methodist University presentó una solución pionera que utiliza procesamiento de lenguaje natural y aprendizaje profundo para analizar tanto titulares como el contenido completo de las noticias [22]. Su trabajo demostró la viabilidad de combinar la vectorización TF-IDF con redes neuronales densas. Adicionalmente, se ha propuesto un enfoque basado en análisis estilométrico que utiliza Procesamiento del Lenguaje Natural (PLN) y Reconocimiento de Entidades Nombradas (NER) para identificar patrones lingüísticos que sugieran baja veracidad [15]. Este método se basa en la hipótesis de que los autores de contenido falso exhiben patrones de escritura distintivos.

2.1.1. Impacto social y desafíos de la desinformación

La rápida propagación de noticias falsas en redes sociales puede tener graves consecuencias sociales. Como documentan Ali y Zain-Ul-Abdin [3], la desinformación puede:

- **Distorsionar la realidad:** Alterando la percepción pública de eventos y hechos.
- **Manipular la opinión pública:** Influenciando procesos democráticos y decisiones sociales.
- **Incitar a la violencia:** Promoviendo comportamientos agresivos basados en información errónea.
- **Difundir propaganda política:** Siendo utilizada como herramienta de influencia partidista.
- **Fomentar el odio:** Intensificando divisiones sociales y promoviendo la discriminación.
- **Provocar pánico:** Especialmente en crisis sanitarias como la pandemia de COVID-19 [4].

2.1.2. El problema de las heurísticas cognitivas

La investigación de Ali et al. [5] demuestra que las heurísticas cognitivas humanas, como la popularidad social (número de “me gusta” o compartidos), influyen de manera significativa en la percepción de credibilidad. Este fenómeno complica la detección, ya que el contenido falso puede volverse viral precisamente al aprovechar estos sesgos cognitivos.

2.1.3. Diferenciación técnica y transferibilidad: noticias falsas vs. fraude digital

Es necesario establecer una distinción técnica clara entre los dominios de aplicación. Aunque comparten características computacionales, representan problemas con diferentes objetivos, audiencias y patrones, como expone la Tabla 2.1.

Características distintivas por dominio

Fundamentos de la transferibilidad metodológica

A pesar de sus diferencias, ambos dominios comparten características computacionales que justifican la transferibilidad de la metodología:

Aspecto	Noticias Falsas	Fraude Digital
Objetivo primario	Manipulación de opinión pública, influencia política/social	Beneficio económico ilícito, acceso no autorizado a recursos
Formato típico	Artículos periodísticos, imitando medios legítimos	Emails de phishing, anuncios de inversión, ofertas comerciales
Audiencia objetivo	Consumidores de noticias, votantes, opinión pública general	Potenciales víctimas económicas, usuarios con activos digitales
Métrica de éxito	Viralidad, alcance, influencia en opinión	Cantidad de dinero/información obtenida
Indicadores lingüísticos	Imitación de estilo periodístico, fuentes falsas, sensacionalismo político	Urgencia económica, ofertas “limitadas”, solicitudes de información personal
Temporalidad	Eventos actuales, ciclos noticiosos	Atemporales, aprovechan tendencias económicas

Tabla 2.1: Diferencias técnicas entre dominios de desinformación

- **Naturaleza textual primaria:** Ambos se basan en contenido textual como principal vector de engaño.
- **Problema de clasificación binaria:** Se reducen a problemas de clasificación legítimo/malicioso.
- **Características estilométricas:** Ambos pueden exhibir patrones lingüísticos distintivos.
- **Optimización de hiperparámetros:** Se benefician de técnicas de calibración metaheurística.
- **Evaluación mediante métricas estándar:** Utilizan las mismas métricas de clasificación (precisión, recall, F1-Score).

Protocolo de transferencia metodológica

Para aplicar la metodología a la detección de fraude digital, se requiere el siguiente protocolo:

1. **Construcción de corpus específico:** Recopilar y etiquetar datos representativos del tipo de fraude.
2. **Ajuste de preprocesamiento:** Adaptar las técnicas de limpieza al nuevo dominio.
3. **Re-calibración de hiperparámetros:** Re-optimizar los algoritmos metaheurísticos para el nuevo corpus.
4. **Incorporación de características específicas:** Añadir características relevantes como URLs sospechosas.
5. **Validación en diferentes dominios:** Evaluar la transferencia con métricas de generalización.

Enfoque de esta investigación: Este proyecto valida la metodología en el dominio de noticias falsas en español, sentando las bases para su transferencia a otros fraudes digitales. La elección de este dominio se justifica por la disponibilidad de corpus, su relevancia social y una menor complejidad técnica inicial.

2.2. Representación de texto: de TF-IDF a Embeddings Contextuales

La evolución de las técnicas de representación textual ha sido clave para el progreso en PLN. Esta sección abarca desde métodos clásicos hasta las representaciones más sofisticadas usadas en esta tesis.

2.2.1. Bolsa de palabras (Bag of Words): primer acercamiento metodológico

El modelo de Bolsa de Palabras (Bag of Words) (Bag of Words, BoW) es un enfoque fundamental en el procesamiento de texto. Aunque no se utilizó en los modelos finales de esta tesis, fue el primer acercamiento explorado en la investigación preliminar [23].

Fundamentos del modelo BoW

El modelo BoW representa un documento como una colección desordenada de sus palabras, ignorando la gramática y el orden, pero manteniendo la frecuencia de cada término. Matemáticamente, un documento d se representa como un vector de frecuencias:

$$\text{BoW}(d) = [f_1, f_2, \dots, f_V] \quad (2.1)$$

Donde f_i es la frecuencia del término i en el documento y V es el tamaño del vocabulario.

Proceso de construcción

1. **Tokenización:** División del texto en unidades léxicas.
2. **Construcción del vocabulario:** Creación de un diccionario con todos los términos únicos.
3. **Vectorización:** Conversión de cada documento en un vector dimensional.
4. **Conteo de frecuencias:** Asignación de la frecuencia del término a cada posición del vector.

Ventajas y aplicabilidad inicial

El modelo BoW ofreció ventajas en las primeras fases de la investigación:

- **Simplicidad conceptual:** Fácil de implementar y comprender.
- **Eficiencia computacional:** Bajo costo de procesamiento para corpus pequeños.
- **Transparencia:** Interpretabilidad directa de las características.
- **Línea base sólida:** Un punto de partida para comparaciones.

Limitaciones identificadas

La experiencia con BoW reveló limitaciones que motivaron la evolución hacia enfoques más sofisticados:

- **Pérdida total del orden:** Imposibilidad de capturar patrones secuenciales.
- **Ausencia de ponderación semántica:** Todas las palabras reciben el mismo tratamiento.
- **Problemas de escalabilidad:** Crecimiento exponencial de la dimensionalidad.
- **Sensibilidad a palabras vacías:** Dominancia de términos frecuentes pero poco informativos.

2.2.2. TF-IDF (Term Frequency-Inverse Document Frequency)

TF-IDF es una de las técnicas más efectivas para la representación de texto. A pesar de su simplicidad, ha demostrado ser muy eficaz en tareas de clasificación. El proceso implica:

1. **Construcción del vocabulario:** Crear un diccionario con todas las palabras únicas del corpus.
2. **Cálculo de ponderaciones:** Asignar a cada término un peso basado en su frecuencia en el documento (TF) y su rareza en el corpus (IDF).

Aunque este método ignora la gramática y el orden de las palabras, fue fundamental en los primeros trabajos de clasificación de noticias en español [11].

2.2.3. Fundamentos matemáticos de TF-IDF

TF-IDF introduce una ponderación semántica mediante el producto de dos componentes:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (2.2)$$

Donde:

- **Frecuencia de Término (TF):** $\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$ - frecuencia relativa del término t en el documento d .
- **Frecuencia Inversa de Documento (IDF):** $\text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D: t \in d\}|}$ - importancia global del término en la colección.

Esta ponderación asigna mayor peso a los términos frecuentes en un documento pero raros en el corpus general, mejorando así la capacidad discriminativa del modelo [22].

2.2.4. Limitaciones de los métodos clásicos

Los enfoques basados en TF-IDF presentan limitaciones fundamentales:

- **Pérdida de información secuencial:** No capturan el orden ni la estructura sintáctica.
- **Problema de dispersidad:** Generan representaciones muy dispersas en vocabularios grandes.
- **Ausencia de semántica:** No modelan relaciones semánticas entre palabras.
- **Falta de contexto:** Una palabra tiene la misma representación sin importar su contexto.

Evolución metodológica

La transición de BoW a TF-IDF representó un primer refinamiento al introducir una ponderación semántica. Esta evolución, documentada en el marco de optimización metaheurística [23], sentó las bases para los enfoques más avanzados de esta tesis. A pesar de sus limitaciones, el modelo BoW fue útil para identificar patrones distintivos en textos en español, sirviendo como punto de partida para las dos metodologías principales de esta investigación.

2.3. La revolución Transformer y los Modelos de Lenguaje modernos

2.3.1. La arquitectura Transformer: fundamentos

La arquitectura Transformer, introducida por Vaswani et al. [24], revolucionó el PLN con su mecanismo de **auto-atención (self-attention)**. Esta innovación permite al modelo calcular representaciones ponderando dinámicamente la importancia de cada elemento de una secuencia en relación con los demás.

Mecanismo de atención multi-cabeza

El mecanismo de atención se define matemáticamente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.3)$$

Donde Q , K y V son las matrices de consultas, claves y valores. La atención multi-cabeza extiende este concepto:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2.4)$$

Esta arquitectura permite capturar diferentes tipos de relaciones lingüísticas simultáneamente, superando las limitaciones de los modelos secuenciales previos.

2.3.2. BERT y la era del preentrenamiento bidireccional

BERT (Bidirectional Encoder Representations from Transformers) [25] introdujo el paradigma de preentrenamiento bidireccional. Al entrenar el modelo para predecir palabras enmascaradas considerando el contexto de ambos lados, genera representaciones contextuales más ricas.

Variantes de BERT para eficiencia

El éxito de BERT motivó el desarrollo de variantes más eficientes:

- **DistilBERT** [20]: Reduce el tamaño del modelo en un 40
- **TinyBERT** [26]: Crea modelos aún más compactos, pero su disponibilidad se limita al inglés.

2.3.3. Aplicaciones en la detección de noticias falsas en español

Para el español se han desarrollado modelos y evaluaciones especializadas:

- Martínez-Gallego et al. [27] exploraron la aplicación de BERT y BETO (BERT en español).
- Blanco-Fernández et al. [13] compararon sistemáticamente BERT y RoBERTa para la detección de desinformación política.
- Shushkevich et al. [28] investigaron el uso de datos aumentados con ChatGPT para mejorar la clasificación multiclase.

2.3.4. Grandes Modelos de Lenguaje (LLMs) y nuevos paradigmas

GPT-3 y el paradigma few-shot

GPT-3 [17] demostró la capacidad de aprendizaje con pocos ejemplos (*few-shot learning*), mostrando que los modelos a gran escala pueden realizar tareas sin un ajuste fino específico.

LLaMA y la democratización de LLMs

LLaMA [29] busca democratizar el acceso a modelos de gran escala, ofreciendo alternativas de código abierto con un rendimiento competitivo y menor costo computacional.

Modelos multimodales: Gemini

Gemini [30] introduce capacidades multimodales nativas, procesando texto, imágenes y audio de forma integrada, lo cual es relevante para la detección de desinformación multimedia.

IA Constitucional

El trabajo sobre IA Constitucional [31] aborda la alineación y seguridad de los LLMs, estableciendo principios para entrenar modelos más seguros y alineados con valores humanos.

2.3.5. El doble rol de los LLMs en detección

La investigación reciente revela el doble rol de los LLMs:

- **Como “buenos consejeros”:** Pueden ser ajustados para detectar noticias falsas con alta precisión [2].
- **Como “malos actores”:** Pueden generar desinformación convincente, complicando su detección [6].

- **Adaptación necesaria:** Los sistemas de detección deben adaptarse a la era de los LLMs [16].

2.4. Optimización metaheurística en la detección de fraude

2.4.1. Fundamentos de las metaheurísticas

Las metaheurísticas son estrategias de optimización de alto nivel que guían la búsqueda de soluciones de alta calidad en espacios complejos. A diferencia de los algoritmos exactos, buscan soluciones “suficientemente buenas” en un tiempo computacional razonable [32].

Justificación del uso de metaheurísticas en la detección de noticias falsas

La aplicación de metaheurísticas en este dominio se justifica por:

- **Espacio de hiperparámetros complejo:** Los modelos de clasificación tienen múltiples hiperparámetros interdependientes.
- **Función objetivo no convexa:** Las métricas de rendimiento no son funciones continuas y diferenciables.
- **Interacciones no lineales:** Los hiperparámetros interactúan de manera compleja.
- **Múltiples óptimos locales:** El espacio de búsqueda puede atrapar algoritmos determinísticos.
- **Eficiencia computacional:** Exploran el espacio de forma más inteligente que los métodos exhaustivos.

Arquitectura general del sistema metaheurístico

El marco de trabajo metaheurístico desarrollado sigue una arquitectura modular:

$$\text{Sistema} = \{D, P, A, F, E\} \quad (2.5)$$

Donde:

- **D:** Dataset de noticias preprocesado.
- **P:** Tubería de preprocesamiento (tokenización, TF-IDF).
- **A:** Algoritmo metaheurístico (GA, PSO, SA, VNS, SS).
- **F:** Función objetivo (F1-Score en validación cruzada).
- **E:** Evaluador final en el conjunto de prueba.

Entradas del sistema metaheurístico

1. Datos de Entrada:

- **Corpus textual:** Noticias en español con etiquetas binarias (REAL/FALSO).
- **División de datos:** 70 % entrenamiento, 10 % validación, 20 % prueba (estratificada).
- **Representación TF-IDF:** Matriz dispersa de dimensión $n \times v$.

2. Espacio de Hiperparámetros:

Los algoritmos metaheurísticos optimizan conjuntos de hiperparámetros. Para TF-IDF, por ejemplo:

$$\text{max_features} \in [1000, 50000] \text{ (tamaño del vocabulario)} \quad (2.6)$$

$$\text{min_df} \in [1, 10] \text{ (frecuencia mínima de documento)} \quad (2.7)$$

$$\text{max_df} \in [0.7, 1.0] \text{ (frecuencia máxima de documento)} \quad (2.8)$$

$$\text{ngram_range} \in \{(1, 1), (1, 2), (1, 3)\} \text{ (rango de n-gramas)} \quad (2.9)$$

Preprocesamiento y tubería de procesamiento (pipeline)

Etapas 1: Limpieza Textual

1. **Normalización de caracteres:** Conversión a minúsculas y eliminación de acentos.
2. **Filtrado de contenido:** Eliminación de URLs, menciones y hashtags.
3. **Tokenización:** División en tokens.
4. **Eliminación de stopwords:** Filtrado de palabras vacías.
5. **Validación de longitud:** Exclusión de textos demasiado cortos.

Etapas 2: Vectorización TF-IDF

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{|\{d' : t \in d'\}|} \right) \quad (2.10)$$

Etapas 3: Normalización Se aplica normalización L2 para estabilizar el entrenamiento:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_2} \quad (2.11)$$

Algoritmos metaheurísticos implementados

1. Algoritmo Genético (GA)

Inspiración biológica: Evolución natural.

Representación: Cada individuo es un vector de hiperparámetros.

$$\text{Individuo} = [C, \gamma, \text{max_features}, \text{min_df}, \text{max_df}, \text{ngram}] \quad (2.12)$$

Operadores genéticos: Selección por torneo, cruce uniforme, mutación gaussiana y elitismo.

Parámetros:

$$\text{Tamaño población} = 30 \quad (2.13)$$

$$\text{Generaciones} = 50 \quad (2.14)$$

$$\text{Tasa cruce} = 0.8 \quad (2.15)$$

$$\text{Tasa mutación} = 0.1 \quad (2.16)$$

2. Optimización por Enjambre de Partículas (PSO)

Inspiración biológica: Comportamiento de bandadas.

Ecuaciones de movimiento:

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (p_i - x_i^t) + c_2 \cdot r_2 \cdot (g - x_i^t) \quad (2.17)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (2.18)$$

Parámetros:

$$\text{Tamaño enjambre} = 30 \quad (2.19)$$

$$\text{Iteraciones} = 50 \quad (2.20)$$

$$\text{Factor inercia} = 0.9 \quad (2.21)$$

$$\text{Factores aceleración} = 2.0 \quad (2.22)$$

3. Recocido Simulado Multi-arranque (MSA)

Inspiración física: Proceso de enfriamiento en metalurgia.

Criterio de aceptación:

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta f \geq 0 \\ e^{\frac{\Delta f}{T}} & \text{si } \Delta f < 0 \end{cases} \quad (2.23)$$

Esquema de enfriamiento:

$$T_{k+1} = \alpha \cdot T_k, \quad \alpha = 0.95 \quad (2.24)$$

Parámetros:

$$\text{Temperatura inicial} = 100.0 \quad (2.25)$$

$$\text{Temperatura final} = 0.01 \quad (2.26)$$

$$\text{Factor enfriamiento} = 0.95 \quad (2.27)$$

$$\text{Iteraciones por temperatura} = 10 \quad (2.28)$$

$$\text{Número de arranques} = 5 \quad (2.29)$$

4. Búsqueda en Vecindades Variables (VNS)

Principio: Cambio sistemático de estructuras de vecindad.

Estructuras de vecindad: Perturbación de uno, dos o todos los hiperparámetros.

Algoritmo general: Se explora sistemáticamente cada vecindad y se reinicia la búsqueda al encontrar una mejora.

5. Búsqueda Dispersa (SS)

Principio: Combinar soluciones de calidad para generar nuevas.

Componentes principales: Un conjunto de referencia, generación de subconjuntos, método de combinación y mejora local.

Función objetivo y evaluación**Función objetivo principal:**

$$f(\theta) = \text{F1-Score}_{CV}(\theta) \quad (2.30)$$

Donde θ es el vector de hiperparámetros y F1-Score_{CV} es el F1-Score promedio en validación cruzada.

Métricas de evaluación secundarias:

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (2.31)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.32)$$

$$\text{F1-Score} = \frac{2 \times \text{Precisión} \times \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (2.33)$$

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.34)$$

Etapas del proceso metaheurístico**Etapas 1: Inicialización**

1. Cargar y dividir el dataset.
2. Definir los espacios de búsqueda.
3. Inicializar la población o enjambre.
4. Configurar los parámetros del algoritmo.

Etapas 2: Evaluación de Aptitud (Fitness)

1. Para cada configuración de hiperparámetros:
2. Construir el pipeline TF-IDF + Clasificador.
3. Entrenar en el conjunto de entrenamiento.

4. Evaluar con validación cruzada.
5. Calcular el F1-Score promedio como fitness.

Etapla 3: Evolución/Optimización

1. Aplicar los operadores del algoritmo.
2. Generar nuevas configuraciones.
3. Evaluar el fitness de las nuevas configuraciones.
4. Actualizar la población o enjambre.
5. Verificar los criterios de convergencia.

Etapla 4: Selección Final

1. Identificar la mejor configuración.
2. Re-entrenar el modelo con la configuración óptima.
3. Evaluar en el conjunto de prueba.
4. Reportar las métricas finales.

Salidas del sistema metaheurístico

1. Configuración Óptima de Hiperparámetros:

- Valores óptimos para cada hiperparámetro.
- Historia de convergencia del algoritmo.
- Número de evaluaciones y tiempo de optimización.

2. Modelo Entrenado Optimizado:

- Vectorizador TF-IDF configurado.
- Clasificador entrenado.
- Pipeline completo listo para predicción.

3. Métricas de Rendimiento:

- F1-Score, Precisión, Recall, Exactitud en el conjunto de prueba.
- Matriz de confusión y curvas de convergencia.
- Análisis de la importancia de los hiperparámetros.

4. Análisis Comparativo:

- Comparación entre los algoritmos metaheurísticos.
- Análisis de eficiencia computacional.
- Estudio de robustez y estabilidad.

Comparación de algoritmos metaheurísticos

La Tabla 2.2 presenta una comparación sistemática de los cinco algoritmos implementados.

Algoritmo	Inspiración	Fortalezas	Limitaciones	Parámetros Clave
GA	Evolución biológica	Exploración global robusta Paralelización natural Buena diversidad poblacional	Convergencia lenta Muchos parámetros a ajustar Puede estancarse prematuramente	Población: 30 Generaciones: 50 Cruce: 0.8 Mutación: 0.1
PSO	Comportamiento de enjambre	Convergencia rápida Pocos parámetros Fácil implementación	Puede convergir prematuramente Sensible a parámetros Exploración limitada	Partículas: 30 Iteraciones: 50 Inercia: 0.9 Aceleración: 2.0
SA/MSA	Recocido de metales	Escape de óptimos locales Base teórica sólida Múltiples arranques	Esquema de enfriamiento crítico Búsqueda individual Muchas evaluaciones	T inicial: 100 T final: 0.01 Enfriamiento: 0.95 Arranques: 5
VNS	Cambio sistemático de vecindades	Escape sistemático de óptimos Combinación exploración/explotación Flexibilidad en vecindades	Dependiente de definición de vecindades Puede ser costoso computacionalmente Diseño específico del problema	Vecindades: 3 Iteraciones: 100 Búsqueda local: Hill climbing
SS	Combinación de soluciones de calidad	Balance calidad-diversidad Intensificación y diversificación Memoria adaptativa	Complejo de implementar Muchos componentes Sensible a tamaño de referencia	Ref. Set: 10 Combinaciones: todas Mejora: local search Actualizaciones: dinámicas

Tabla 2.2: Comparación de algoritmos metaheurísticos implementados

Criterios de selección y justificación

La selección de estos cinco algoritmos se basó en:

1. **Diversidad de paradigmas:** Se incluyeron representantes de las principales familias de metaheurísticas.
2. **Eficiencia computacional:** Algoritmos con un buen balance entre calidad y tiempo de cómputo.
3. **Robustez empírica:** Métodos con efectividad documentada en problemas de optimización de ML.
4. **Implementación práctica:** Algoritmos con parámetros bien establecidos.
5. **Complementariedad:** Enfoques que exploran el espacio de búsqueda de maneras diferentes.

Esta selección permite comparar diferentes estrategias y determinar cuáles son más efectivas para la calibración de detectores de noticias falsas.

Justificación frente a métodos alternativos

Comparación con Búsqueda en Cuadrícula (Grid search):

- **Eficiencia:** Grid search es computacionalmente inviable para espacios de alta dimensión.

- **Optimización continua:** Las metaheurísticas manejan parámetros continuos de forma natural.

Comparación con Búsqueda Aleatoria (Random Search):

- **Inteligencia dirigida:** Las metaheurísticas aprenden de las evaluaciones para guiar la búsqueda.

Comparación con Optimización Bayesiana:

- **Flexibilidad:** Las metaheurísticas son más flexibles para restricciones complejas.

Comparación con Métodos Basados en Gradiente:

- **Diferenciabilidad:** Las métricas de clasificación no son diferenciables, y algunos hiperparámetros son discretos.

Flujo del proceso metaheurístico

La Figura 2.1 ilustra el flujo completo del proceso de optimización.

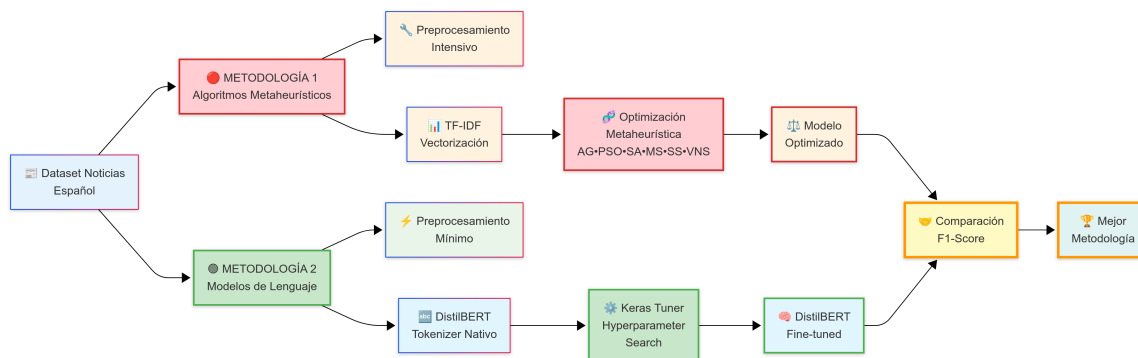


Figura 2.1: Flujo del proceso de optimización para la detección de noticias falsas.

Este flujo garantiza reproducibilidad, escalabilidad, transferibilidad y transparencia.

Consideraciones de implementación

Manejo de Memoria: Se utilizaron matrices dispersas y procesamiento por lotes.

Paralelización: Se implementó la evaluación paralela de individuos y la vectorización de operaciones.

Validación y Robustez: Se usó validación cruzada estratificada y múltiples ejecuciones para el análisis estadístico.

Métricas de monitoreo del proceso

Durante la ejecución se monitorearon la convergencia, diversidad, estancamiento, eficiencia y estabilidad para ajustar y comparar los algoritmos.

2.4.2. Aplicaciones en la detección de noticias falsas

Modelado como problema de programación de trabajos (Job Shop Scheduling)

Aqil y Lahby [33] modelaron la detección de noticias falsas como un problema de FSSP. Compararon tres metaheurísticas y encontraron que el algoritmo Voraz Iterativo (IG) lograba el mejor rendimiento.

Optimización de hiperparámetros en Deep Learning

Bacanin et al. [34] demostraron los beneficios de usar metaheurísticas para la calibración de hiperparámetros en modelos de *deep learning*, mostrando mejoras significativas sobre métodos tradicionales. La investigación que publiqué en [23] aborda específicamente la calibración de hiperparámetros para la detección de fraude digital, proporcionando un marco relevante para esta tesis.

Marcos de trabajo (frameworks) de ensamble (ensemble) heurístico

Das et al. [21] desarrollaron un marco de ensamble que combina modelos preentrenados, características estadísticas y heurísticas de incertidumbre, logrando F1-scores superiores al 95

2.4.3. Metaheurísticas para la detección de fraude financiero

Optimización multi-objetivo

Hidayattullah et al. [35] aplicaron metaheurísticas para optimizar la detección de fraude en estados financieros, considerando precisión, eficiencia y robustez. Sus resultados mostraron que un SVM optimizado con Algoritmo Genético alcanzó una precisión del 96.15

Optimización de procesos gaussianos

Horak y Sabek [36] exploraron la optimización de hiperparámetros en *Gaussian Process Regression* para la predicción de dificultades financieras, destacando la importancia de la calibración metaheurística.

2.4.4. Algoritmos híbridos modernos

Enfoques multi-thread

Yildirim [37] propuso un enfoque metaheurístico híbrido *multi-thread* que optimiza simultáneamente la selección de características, los parámetros del clasificador y la arquitectura del ensamble.

PSO adaptativo

Deshai y Bhaskara Rao [38] desarrollaron un enfoque de CNN con PSO adaptativo para detectar reseñas falsas, ajustando dinámicamente los parámetros del algoritmo.

Fine-tuning: adaptación de modelos preentrenados

El paradigma de **fine-tuning**, introducido por Howard y Ruder [39], ha sido una innovación significativa. Consiste en tomar un modelo preentrenado y continuar su entrenamiento con datos específicos del dominio objetivo.

En modelos Transformer como BERT [40], esto implica ajustar las capas superiores para la tarea específica de clasificación. La efectividad de este enfoque radica en que las representaciones contextuales aprendidas durante el preentrenamiento son transferibles a otras tareas [41]. Para la detección de noticias falsas, esto constituye la base del segundo enfoque desarrollado en esta investigación.

2.5. Síntesis del marco teórico

La tendencia actual en la detección de noticias falsas favorece los sistemas híbridos que combinan múltiples paradigmas. Este marco teórico establece los fundamentos para las dos metodologías desarrolladas en esta tesis:

1. **Enfoque clásico optimizado:** Utiliza representaciones TF-IDF con optimización metaheurística de hiperparámetros.
2. **Enfoque de deep learning:** Emplea modelos Transformer preentrenados con *fine-tuning*.

La integración de ambos enfoques, sustentada en la literatura, establece la base teórica para la contribución metodológica de esta tesis: la aplicación sistemática de la optimización de hiperparámetros en ambos paradigmas, en el contexto específico de textos en español.

Capítulo 3

Estado del arte

La detección automatizada de noticias falsas se ha convertido en una de las preocupaciones más apremiantes de la sociedad digital contemporánea. Este campo de investigación multidisciplinario abarca desde la ciencia de la computación hasta la psicología cognitiva. Durante la última década, los investigadores han desarrollado enfoques innovadores que van desde técnicas tradicionales de procesamiento de lenguaje natural hasta modelos de lenguaje preentrenados de última generación. Estos avances responden a la creciente sofisticación de las campañas de desinformación y la velocidad sin precedentes con la que se propagan en las redes sociales.

En el contexto hispanohablante, esta problemática adquiere características particulares debido a las especificidades lingüísticas y culturales del español. La complejidad del idioma, con sus múltiples variantes geográficas y ricas estructuras sintácticas, presenta desafíos únicos para los sistemas automatizados de detección, que tradicionalmente han sido desarrollados y entrenados principalmente para el inglés.

Este capítulo examina las principales contribuciones científicas en el campo de la detección de desinformación, con especial énfasis en las aproximaciones que combinan técnicas de optimización metaheurística con modelos de aprendizaje automático. Se analiza también la evolución de los enfoques metodológicos, desde los primeros sistemas basados en características superficiales hasta los desarrollos más recientes que incorporan arquitecturas Transformer y algoritmos de optimización bioinspirados.

3.1. Metodología de búsqueda bibliográfica

Para garantizar una revisión completa y actualizada de la literatura científica, se diseñó una estrategia de búsqueda en múltiples bases de datos académicas reconocidas internacionalmente. El proceso se llevó a cabo entre agosto de 2023 y agosto de 2025, utilizando una combinación de términos clave y operadores booleanos para maximizar la relevancia de los resultados.

Las bases de datos consultadas incluyeron Google Scholar, Semantic Scholar, Nature Digital Libraries, IOPscience, MDPI, Web of Science (WoS) y repositorios especializados como arXiv. La selección de estas fuentes se basó en su cobertura disci-

plinaria, calidad editorial y relevancia para el campo. Adicionalmente, se incluyeron conferencias especializadas como ACL, EMNLP y talleres como CheckThat! para capturar los desarrollos más recientes.

La búsqueda se estructuró en torno a combinaciones de términos que reflejan las dimensiones de esta investigación:

- *Spanish fake news detection* y “detección de noticias falsas en español”.
- *Fake news detection language models*, cubriendo términos como BERT, RoBERTa y Transformer.
- *Metaheuristic hyperparameter optimization* para identificar aplicaciones en aprendizaje automático.
- *Employment fraud detection* para abarcar el dominio del fraude laboral.
- *Simulated annealing fake news* y *Genetic algorithm text classification* para enfoques de optimización específicos.
- *Social media misinformation detection* para trabajos enfocados en plataformas sociales.
- *Cross-lingual fake news* y *multilingual misinformation* para incluir enfoques multilingües.

Se priorizaron publicaciones de los últimos cinco años (2020-2025), sin excluir trabajos fundamentales anteriores. Los criterios de inclusión fueron: contribuciones metodológicas originales, validación empírica, relevancia directa para la detección de desinformación y calidad de la revisión por pares. Como resultado, se identificaron 83 trabajos que constituyen el núcleo de esta revisión.

3.2. Panorama de la investigación en detección de noticias falsas y Modelos de Lenguaje

El campo de la detección automatizada de desinformación ha evolucionado rápidamente, transitando desde enfoques basados en características estadísticas simples hacia sistemas sofisticados que integran múltiples modalidades y conocimiento contextual profundo. Esta evolución refleja tanto los avances en inteligencia artificial como la creciente complejidad de las técnicas de desinformación.

La progresión del campo puede dividirse en tres fases:

1. **La era pionera (2010-2016)**, con enfoques basados en características lingüísticas superficiales y aprendizaje automático tradicional.
2. **La era de los modelos profundos (2017-2020)**, marcada por la adopción de redes neuronales y arquitecturas Transformer.

3. **La era contemporánea (2021-presente)**, dominada por modelos de lenguaje a gran escala y enfoques híbridos [42].

3.2.1. Fundamentos metodológicos y construcción de corpus

La disponibilidad de conjuntos de datos de alta calidad es fundamental para cualquier sistema de detección eficaz. En el contexto hispanohablante, esta necesidad es crítica debido a la escasez de recursos en comparación con el inglés. Los trabajos pioneros en esta área han establecido las bases para la construcción y evaluación de corpus especializados, influyendo significativamente en el desarrollo del campo.

Acosta [11] desarrolló uno de los primeros marcos sistemáticos para construir conjuntos de datos de noticias en español, con criterios rigurosos de calidad y protocolos de etiquetado manual. Su metodología demostró la viabilidad de crear recursos lingüísticos a gran escala para el español. Por su parte, el trabajo de Posadas-Durán et al. [12] propuso un nuevo corpus con características estilométricas innovadoras, abriendo nuevas líneas de investigación sobre el estilo de escritura como indicador de veracidad.

La comunidad científica internacional ha impulsado iniciativas como los talleres CheckThat! en CLEF [43, 44], que proporcionan marcos estandarizados para la evaluación comparativa de sistemas. Un desafío clave es que los corpus disponibles no son homogéneos, ya que abordan diferentes aspectos del problema. En el ámbito iberoamericano, las competencias IberLEF han sido fundamentales. Aragón et al. [45] documentaron los primeros esfuerzos para abordar la detección de noticias falsas en el español de México, estableciendo el **F1-Score como la métrica principal de comparación**.

La estandarización de métricas ha sido crucial para el progreso del campo. Los marcos como CheckThat! [44] han establecido que las métricas fundamentales deben incluir Precisión, Recall (exhaustividad), F1-Score y Exactitud. Esta estandarización permite una comparación directa y reproducible entre diferentes sistemas.

Contribución Principal	Autores	Publicación	Año	Ref.
Marco metodológico para construcción de conjuntos de datos de noticias en español	Acosta, F. A. Z.	U. Politécnica de Madrid	2019	[11]
Evaluación multimodal y multigenre para verificación de contenido	Alam, F., et al.	CEUR Workshop Proc.	2023	[43]
Análisis de agresividad y desinformación en español mexicano	Aragón, M. E., et al.	IberLEF Workshop	2020	[45]
Marco estandarizado CheckThat! para evaluación de sistemas	Barrón-Cedeño, A., et al.	ECIR 2023	2023	[44]
Consolidación de métricas FakeDeS para detección en español	Gómez-Adorno, H., et al.	Procesamiento del Lenguaje Natural	2021	[46]
Corpus pionero con características estilométricas para español	Posadas-Durán, J. P., et al.	J. of Intelligent and Fuzzy Systems	2019	[12]
Corpus enriquecido con anotaciones multinivel versión 2.0	Ramírez Cruz, J. M., et al.	GitHub Repository	2021	[47]
Técnicas de aprendizaje automático aplicadas al español	Tretiakov, A., et al.	Springer	2022	[19]

Tabla 3.1: Contribuciones fundamentales en metodología y construcción de corpus para español.

3.2.2. Evolución de los Modelos de Lenguaje en la detección de desinformación

La irrupción de los modelos de lenguaje preentrenados ha revolucionado la detección de noticias falsas. Esta transformación ha progresado desde arquitecturas básicas hacia sistemas sofisticados con una profunda comprensión contextual.

El punto de inflexión fue la introducción de **BERT** [25], cuya arquitectura bidireccional permitió capturar dependencias contextuales de manera más efectiva que los modelos secuenciales anteriores. La capacidad de BERT para entender el contexto completo de una oración simultáneamente representó un avance fundamental.



Figura 3.1: Mapa Conceptual 3: Artículos relacionados que incorporan Modelos de Lenguaje.

La necesidad de modelos más eficientes llevó al desarrollo de variantes como **DistilBERT** [20], que mantiene el 97

El surgimiento de modelos generativos a gran escala como **GPT-3** [17] ha introducido nuevas complejidades. Como señalan Papageorgiou et al. [48], estos modelos pueden tanto generar contenido falso indistinguible del humano como ser herramientas poderosas para su detección. Investigaciones recientes revelan que los detectores actuales muestran sesgos contra el texto generado por LLMs [6], lo que plantea cuestiones sobre la generalización y equidad de estos sistemas.

3.2.3. Técnicas de optimización metaheurística en la detección de desinformación

La aplicación de algoritmos metaheurísticos a la detección de noticias falsas es un área innovadora. Estos algoritmos son eficaces para resolver problemas de optimiza-

Innovación Principal	Autores	Publicación	Año	Ref.
Arquitectura bidireccional para comprensión contextual profunda	Devlin, J., et al.	arXiv	2018	[25]
Destilación de conocimiento para modelos eficientes	Sanh, V., et al.	arXiv	2019	[20]
Aplicación de transformers en español (MédicoBERT)	Padilla Cuevas, J., et al.	Applied Sciences	2024	[49]
Revisión sobre el uso de LLMs en noticias falsas	Papageorgiou, E., et al.	Future Internet	2024	[48]
Comparación BERT vs RoBERTa para español	Blanco-Fernández, Y., et al.	Applied Sciences	2024	[13]
Capacidades emergentes en modelos de gran escala	Brown, T. B., et al.	arXiv	2020	[17]
Modelos multimodales de próxima generación	Gemini Team, Google	arXiv	2023	[30]
El auge de los grandes Modelos de Lenguaje	Nature Comp. Science	Nature	2025	[42]

Tabla 3.2: Evolución de modelos de lenguaje aplicados a la detección de desinformación.

ción complejos, como la calibración de hiperparámetros y la selección de características.

Aqil y Lahby [33] fueron pioneros al modelar la tarea como un problema de planificación, evaluando metaheurísticas como Algoritmos Genéticos. En el ámbito de las reseñas falsas, Deshai y Rao [38] combinaron redes neuronales convolucionales (CNN) con Optimización por Enjambre de Partículas (PSO), demostrando que PSO puede optimizar eficazmente los hiperparámetros de las CNN.

Otros trabajos han explorado el uso de metaheurísticas para optimizar modelos de aprendizaje profundo y manejar la incertidumbre en la clasificación. La diversificación hacia enfoques híbridos, como la combinación de K-Means y SVM [50], ha demostrado que la integración de metaheurísticas con técnicas de preprocesamiento puede mejorar la eficacia de los detectores.

3.3. Algoritmos metaheurísticos: fundamentos teóricos y aplicaciones prácticas

Los algoritmos metaheurísticos son técnicas de optimización versátiles y efectivas para resolver problemas complejos, como la calibración de hiperparámetros en modelos de aprendizaje automático. A continuación, se sintetizan los fundamentos de los algoritmos más relevantes.

Contribución Metodológica	Autores	Publicación	Año	Ref.
Formulación como problema de planificación de tareas	Aqil, S., & Lahby, M.	Studies in Comp. Intelligence	2021	[33]
PSO adaptativo para optimización de CNN en detección de reseñas	Deshai, N., & Rao, B. B.	Soft Computing	2023	[38]
Evidencia sistemática de beneficios en modelos de aprendizaje profundo	Bacanin, N., et al.	Energies	2023	[34]
Marco ensemble para manejo de incertidumbre	Das, S. D., et al.	Neurocomputing	2022	[21]
Aplicación específica para español y fraude digital	Hurtado Avilés, G., et al.	BUAP	2024	[23]
Enfoque multi-hilo para detección multimodal	Yildirim, G.	Applied Intelligence	2023	[37]
Combinación K-Means y SVM para selección de características	Yazdi, K. M., et al.	WASET	2020	[50]
Optimización de ensemble con metaheurísticas	Yasmin, A., et al.	PLOS ONE	2024	[51]

Tabla 3.3: Contribuciones metodológicas en optimización metaheurística para detección.



Figura 3.2: Mapa Conceptual 4: Métodos de optimización y metaheurísticas aplicadas.

3.3.1. Algoritmos Genéticos (GA): evolución artificial para la optimización

Inspirados en la evolución darwiniana, los **Algoritmos Genéticos** [52] mantienen una población de soluciones que evoluciona a través de operadores como la selección, el cruzamiento y la mutación. Este proceso permite una exploración eficiente de espacios de búsqueda complejos, siendo efectivos para la optimización de hiperparámetros, la selección de características y la evolución de arquitecturas de redes neuronales.

3.3.2. Optimización por Enjambre de Partículas (PSO): inteligencia colectiva

La **Optimización por Enjambre de Partículas** [53, 54] se inspira en el comportamiento social de bandadas de pájaros. Cada partícula (solución) se mueve a través del espacio de búsqueda influenciada por su propia experiencia y la del enjambre. Este equilibrio entre exploración y explotación la hace efectiva para la calibración de hiperparámetros en redes neuronales, donde las evaluaciones son computacionalmente costosas.

3.3.3. Recocido Simulado: física estadística para la optimización

El **Recocido Simulado** [55] se basa en el proceso físico de enfriamiento de metales. Permite escapar de óptimos locales al aceptar soluciones temporalmente peores con una probabilidad que disminuye gradualmente. La combinación con estrategias de **Multi-arranque** [56] aprovecha la capacidad de escape de óptimos locales con una exploración diversificada del espacio de búsqueda.

3.3.4. Búsqueda Dispersa (Scatter Search): combinación determinística

La **Búsqueda Dispersa** [57] utiliza estrategias determinísticas para combinar un conjunto de soluciones de referencia de alta calidad, generando así nuevas soluciones prometedoras. A diferencia de otros métodos, se basa en una diversificación controlada y en la combinación estructurada de soluciones, lo que la hace efectiva en problemas de optimización combinatoria.

3.3.5. Búsqueda en Vecindades Variables (VNS): exploración sistemática

La **Búsqueda en Vecindades Variables** [58] se basa en el cambio sistemático de estructuras de vecindad durante la búsqueda. Esta técnica es especialmente útil

para escapar de óptimos locales, ya que explora diferentes definiciones de vecindad que capturan diversos aspectos de la estructura del problema.

Algoritmo	Inspiración	Características Principales	Aplicaciones en Detección	Ref.
Algoritmos Genéticos (GA)	Evolución biológica y genética molecular	Población de soluciones, operadores genéticos, selección natural, teoría de esquemas	Optimización de hiperparámetros, selección de características, evolución de arquitecturas de redes neuronales	[52]
Optimización por Enjambre de Partículas (PSO)	Comportamiento social de bandadas y enjambres	Partículas con velocidad y posición, memoria personal y social, inteligencia colectiva	Calibración de pesos en redes, optimización de parámetros de modelos de lenguaje, ajuste de transformers	[53]
Recocido Simulado (SA)	Proceso físico de enfriamiento de metales y mecánica estadística	Aceptación probabilística, esquema de enfriamiento, escape de óptimos locales, distribución de Boltzmann	Optimización de arquitecturas profundas, ajuste fino de modelos complejos, calibración no convexa	[55]
Multi-arranque (Multi-start)	Diversificación de puntos de inicio y exploración global	Múltiples ejecuciones independientes, exploración global del espacio, robustez estadística	Inicialización robusta de modelos, validación cruzada optimizada, ensemble de optimizadores	[56]
Búsqueda Dispersa (SS)	Combinación sistemática de soluciones diversas y métodos determinísticos	Conjunto de referencia, combinaciones determinísticas, diversificación controlada, memoria adaptativa	Ensemble de modelos, combinación de características, optimización de flujo de datos, fusión de arquitecturas	[57]
Búsqueda en Vecindades Variables (VNS)	Cambio sistemático de estructuras de vecindad y perspectivas múltiples	Múltiples definiciones de vecindad, alternancia entre diversificación e intensificación	Exploración de espacios de hiperparámetros, optimización de topologías de red, búsqueda adaptativa	[58]

Tabla 3.4: Algoritmos metaheurísticos: características fundamentales y aplicaciones en detección de noticias falsas.

3.4. Hiperparámetros en modelos de aprendizaje automático: fundamentos y optimización

3.4.1. Naturaleza y definición de hiperparámetros

Para explicar este concepto, podemos usar una analogía culinaria. Si entrenar un modelo de aprendizaje automático es como hornear un pastel, los **parámetros** son los procesos que ocurren dentro del horno, como las reacciones químicas entre los ingredientes, que el modelo ajusta automáticamente. En una red neuronal, estos son los pesos y sesgos que se optimizan durante el entrenamiento.

Los **hiperparámetros**, en cambio, son las decisiones que el científico de datos debe tomar antes de comenzar, como la temperatura del horno o el tiempo de cocción. En un modelo, estos serían la tasa de aprendizaje, el número de épocas, la arquitectura de la red o los métodos de regularización. Una elección inadecuada puede llevar a un rendimiento subóptimo, sobreajuste o subajuste.

3.4.2. Importancia crítica en la detección de noticias falsas

En la detección de noticias falsas, la optimización de hiperparámetros es crucial debido a la complejidad de la tarea:

- **Especificidades lingüísticas:** El español requiere configuraciones adaptadas a sus variaciones regionales y patrones discursivos.
- **Sutileza de los patrones:** La desinformación moderna puede ser muy sutil, lo que requiere modelos finamente calibrados.
- **Desbalance de clases:** Los conjuntos de datos a menudo están desbalanceados, lo que exige ajustes específicos.
- **Complejidad contextual:** Una configuración inadecuada puede llevar a que el modelo confunda el sarcasmo o la ironía con falsedad.

3.4.3. Desafíos en la búsqueda de la configuración óptima

Tradicionalmente, la optimización de hiperparámetros se ha abordado con métodos como:

- **Búsqueda en grilla (Grid Search):** Explora sistemáticamente todas las combinaciones posibles, pero es computacionalmente impracticable en espacios grandes.
- **Búsqueda aleatoria (Random Search):** Muestrea combinaciones al azar, siendo más eficiente pero sin una estrategia inteligente.
- **Optimización Bayesiana:** Utiliza un modelo probabilístico para guiar la búsqueda, pero puede atascarse en óptimos locales.

Los algoritmos metaheurísticos emergen como una solución más inteligente, ya que aprenden de cada evaluación para guiar la búsqueda de manera más efectiva.

3.4.4. Herramientas modernas: Keras Tuner y automatización

Keras Tuner [59] es una herramienta de TensorFlow que automatiza la búsqueda de hiperparámetros. Facilita la definición de espacios de búsqueda, la paralelización de evaluaciones y la implementación de estrategias avanzadas como Hyperband y Optimización Bayesiana.

3.4.5. Beneficios combinados: Modelos de Lenguaje y herramientas de optimización

La combinación de modelos de lenguaje modernos como DistilBERT con herramientas de optimización como Keras Tuner o algoritmos metaheurísticos crea sistemas de optimización muy potentes. En contraste, la búsqueda de parámetros en los algoritmos metaheurísticos se realiza de manera más controlada, lo que permite una mayor incorporación de conocimiento específico del dominio.



Figura 3.3: Mapa Conceptual 5: Clasificación de artículos por enfoque metodológico.

3.5. Análisis de la literatura relevante

3.5.1. El desafío fundamental de los datos: creación y curación de corpus en español

La escasez de conjuntos de datos etiquetados a gran escala en español es un desafío persistente. El trabajo de Zules Acosta [11] fue pionero en este ámbito, centrándose en la ingeniería de características para la clasificación de noticias. Por su parte, Posadas-Durán et al. [12] introdujeron el influyente *Spanish Fake News Corpus*, un recurso fundamental para el análisis estilométrico.

Más recientemente, se han publicado corpus a mayor escala que han sido cruciales para el avance del campo. Tretiakov et al. [19] aportaron un conjunto de datos con metadatos enriquecidos, mientras que Blanco-Fernández et al. [13] introdujeron el *Spanish Political Fake News Dataset*, uno de los recursos más completos disponibles y la principal fuente de datos para esta tesis.

3.5.2. Revolución de los Modelos de Lenguaje y arquitecturas Transformer

La arquitectura Transformer [24] transformó el PLN con su mecanismo de atención. Esto sentó las bases para modelos como BERT [25], que introdujo el preentrenamiento bidireccional, y sus variantes optimizadas como DistilBERT [20].

El paradigma de los Grandes Modelos de Lenguaje (LLMs) se consolidó con GPT-3 [17], que demostró capacidades emergentes de aprendizaje con pocos ejemplos. Investigaciones posteriores han revelado el doble rol de los LLMs, que pueden ser tanto

herramientas para la detección como generadores de desinformación [2, 16].

3.5.3. Optimización y metaheurísticas en la detección: enfoques innovadores

Los algoritmos metaheurísticos han demostrado ser valiosos para problemas de optimización en la detección de noticias falsas. Aqil y Lahby [33] modelaron la detección como un problema de planificación, mientras que otros trabajos se han centrado en la calibración de hiperparámetros [34, 23], superando a los métodos tradicionales. Enfoques de ensamble con marcos heurísticos [21] han mostrado resultados prometedores al combinar múltiples modelos.

3.5.4. Aplicación de metaheurísticas por tipo de detección

La literatura revela patrones en la aplicación de metaheurísticas según el tipo de detección y el dominio. La Tabla 3.5 proporciona una taxonomía detallada de estas aplicaciones, mostrando que la elección del algoritmo está influenciada por el tipo de modelo, las características de los datos y los requisitos del sistema.

3.5.5. Perspectivas interdisciplinarias y análisis social

La comprensión del fenómeno de las noticias falsas requiere un enfoque interdisciplinario. Investigaciones como las de Ali et al. [5, 3] han explorado cómo las heurísticas cognitivas humanas influyen en la percepción de credibilidad. El análisis del rol de los medios de comunicación [7] y el impacto de la desinformación en la salud [10] también han revelado la importancia de considerar factores sociales y culturales.

3.5.6. Detección de fraude: extensión más allá de las noticias

La investigación en detección de fraude se ha extendido a otros dominios, como el fraude laboral [9] y el financiero [35], demostrando la transferibilidad de las técnicas desarrolladas.

3.6. Síntesis y perspectivas futuras

La revisión de la literatura revela una rápida evolución en las técnicas para la detección de noticias falsas. Los desafíos principales incluyen la necesidad de más datos etiquetados en español, la adaptación cultural de los modelos y la optimización de hiperparámetros. La organización temática de este capítulo proporciona un marco conceptual que justifica el enfoque híbrido de esta investigación, que combina modelos de lenguaje con algoritmos metaheurísticos para lograr un rendimiento superior.

Tipo de Detección	Metaheurística Empleada	Aplicación Específica	Autores	Ref.
Detección de Noticias Falsas	Algoritmo Genético (GA), Colonia de Abejas (ABC), Búsqueda Iterativa (IG)	Planificación de tareas para procesamiento de documentos	Aqil, S. & Lahby, M.	[33]
Detección de Reseñas Falsas	Optimización por Enjambre de Partículas Adaptativo (PSO)	Optimización de hiperparámetros en redes CNN	Deshai, N. & Rao, B. B.	[38]
Fraude Financiero y de Estados Financieros	Algoritmos Genéticos, Optimización por Enjambre de Partículas, Recocido Simulado	Selección de características y optimización de clasificadores SVM	Hidayattullah, S. et al.	[35]
Predicción de Dificultades Financieras	Optimización Bayesiana con Procesos Gaussianos	Calibración de hiperparámetros en modelos GPR	Horak, J. & Sabek, A.	[36]
Detección de Fraude Laboral	Redes Neuronales Artificiales (ANN)	Clasificación de ofertas de empleo fraudulentas	Nasser, I. M. et al.	[9]
Optimización de Modelos de Energía	Algoritmo Genético Binario, PSO, Recocido Simulado, Búsqueda Armónica	Ajuste de hiperparámetros en modelos LSTM para predicción energética	Bacanin, N. et al.	[34]
Detección Multimodal de Noticias Falsas	Enfoque Híbrido Multi-hilo con Metaheurísticas	Optimización paralela de características textuales y visuales	Yildirim, G.	[37]
Selección de Características para Fake News	K-Means combinado con Support Vector Machine (SVM)	Reducción de dimensionalidad y mejora de precisión	Yazdi, K. M. et al.	[50]
Framework de Incertidumbre para Fake News	Enfoque Heurístico basado en Ensemble	Manejo de incertidumbre en clasificación de tweets y artículos	Das, S. D. et al.	[21]
Optimización de Dominación Total en Redes Sociales	Búsqueda en Vecindades Variables (VNS)	Propagación de información en redes sociales	Kapunac, S. et al.	[60]
Estimación de Esfuerzo en Proyectos	Ensemble con Metaheurísticas para Pesos	Optimización de hiperparámetros y pesos de ensemble	Yasmin, A. et al.	[51]

Tabla 3.5: Aplicación de metaheurísticas por tipo de detección en la literatura revisada.



Figura 3.4: Mapa Conceptual 6: Artículos que son revisiones o están relacionados al análisis de contenido y detección de fraude digital.

Capítulo 4

Metodología

En este capítulo se presenta una metodología unificada e integrada para abordar el problema de la detección de noticias falsas en español. La estrategia metodológica se fundamenta en un proceso evolutivo de desarrollo y evaluación que avanza desde técnicas clásicas hasta modelos de lenguaje de última generación, permitiendo una comparación sistemática y objetiva de diferentes paradigmas de inteligencia artificial sobre una base de datos común.

El diseño metodológico adopta un enfoque experimental escalonado que incluye: (1) adquisición y procesamiento unificado de datos, (2) implementación de un flujo de datos de procesamiento común, (3) desarrollo paralelo de modelos de clasificación utilizando algoritmos metaheurísticos y modelos Transformer, y (4) evaluación comparativa integral bajo un marco de métricas unificado. Este proceso culmina con la implementación de la solución más efectiva en una aplicación web funcional.

4.1. Visión general del proceso metodológico

La metodología se estructura como un flujo de datos experimental unificado que evalúa sistemáticamente dos paradigmas de inteligencia artificial complementarios. Como se ilustra en la Figura 4.1, ambos enfoques parten de una base metodológica común pero implementan estrategias de modelado diferenciadas, convergiendo finalmente en una evaluación comparativa que determina la solución óptima.

Fundamento del enfoque evolutivo: Esta investigación adopta una filosofía de desarrollo evolutivo donde se exploran técnicas de complejidad creciente, comenzando con métodos clásicos de PLN optimizados mediante metaheurísticas (estableciendo una línea base sólida) y progresando hacia modelos Transformer de última generación. Esta progresión permite:

- **Validación incremental:** Cada fase valida y mejora los resultados de la anterior
- **Comparación objetiva:** Todos los modelos se evalúan bajo las mismas condiciones experimentales

- **Justificación de complejidad:** Se demuestra empíricamente si la complejidad adicional se traduce en mejoras de rendimiento
- **Transferibilidad metodológica:** Los procesos desarrollados son aplicables a otros tipos de fraude digital

Una vez completada la evaluación comparativa, el modelo de mejor rendimiento se implementa en una aplicación web que demuestra la viabilidad práctica de la solución desarrollada.

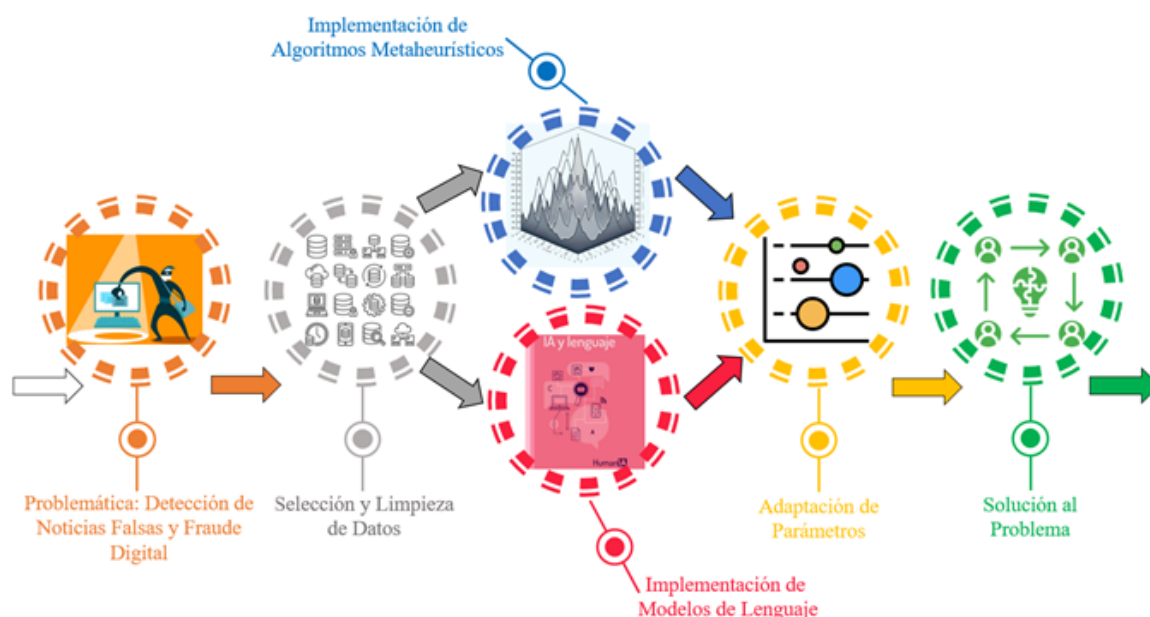


Figura 4.1: Metodología propuesta que aborda la problemática combinando Algoritmos Metaheurísticos y Modelos de Lenguaje.

4.1.1. Esquema visual del flujo de trabajo general del sistema

Para facilitar la comprensión integral de la arquitectura y el flujo de procesamiento implementado, se presenta un esquema visual completo del sistema desarrollado. La Figura 4.2 ilustra la arquitectura general del sistema, mostrando la interacción entre los componentes principales y el flujo de datos desde la adquisición del corpus hasta la implementación final.

Este esquema arquitectónico unificado permite visualizar:

- **Capa de datos:** Proceso de unificación del corpus desde múltiples fuentes académicas y extracción web
- **Capa de preprocesamiento:** Flujo de procesamiento de datos común para limpieza, tokenización y representación textual

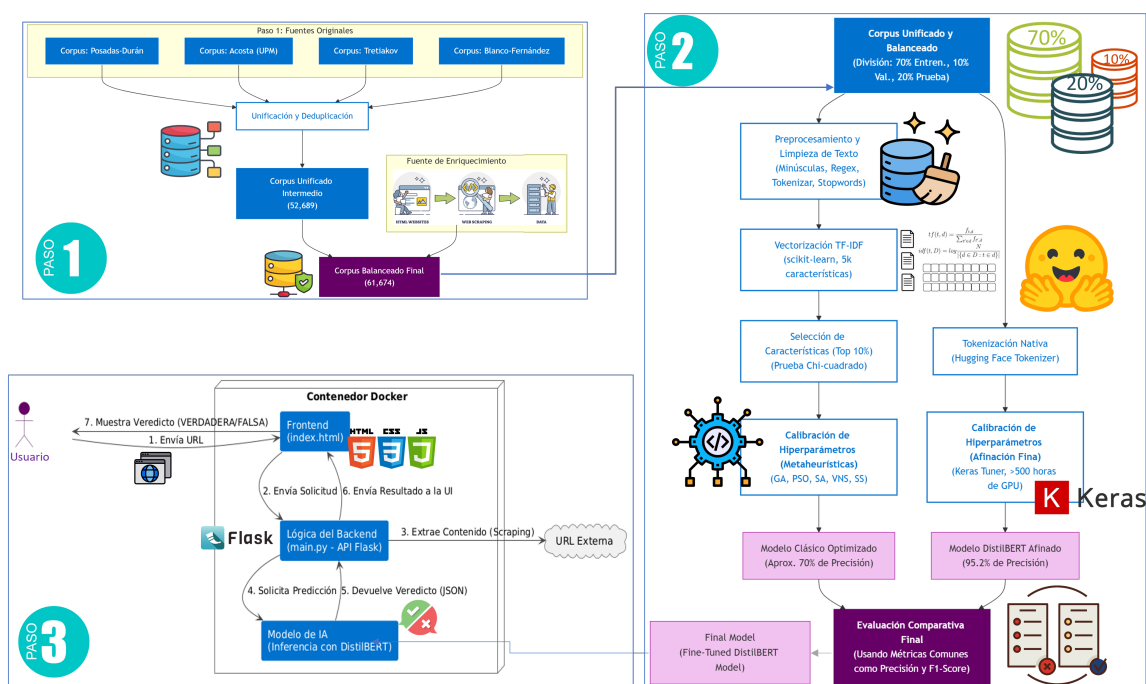


Figura 4.2: Arquitectura general del sistema de detección de noticias falsas, mostrando el flujo completo desde la construcción del corpus hasta la implementación de la aplicación web.

- **Capa de modelado dual:** Implementación paralela de enfoques metaheurísticos y Transformer
- **Capa de evaluación:** Marco unificado de métricas y validación cruzada
- **Capa de aplicación:** Integración del modelo óptimo en sistema productivo

4.2. Definición y distinción de conceptos fundamentales

Antes de proceder con la descripción metodológica, es fundamental establecer con precisión la terminología utilizada en esta investigación, particularmente la distinción entre conceptos relacionados pero diferenciados que a menudo se utilizan de manera intercambiable en la literatura.

4.2.1. Distinción entre noticia falsa y bulo

En el contexto de esta investigación, es esencial distinguir entre dos conceptos fundamentales que, aunque relacionados, poseen características distintivas importantes para el desarrollo de sistemas de detección automática.

4.2.2. Taxonomía de la desinformación

Para proporcionar un marco conceptual completo, esta investigación adopta la taxonomía establecida por la literatura internacional [1, 18], que clasifica el contenido engañoso en tres categorías principales:

1. **Desinformación:** Información falsa creada y compartida deliberadamente con intención maliciosa
2. **Desinformación (Misinformation):** Información incorrecta compartida sin intención maliciosa
3. **Información Maliciosa (Malinformation):** Información genuina compartida con intención de causar daño

El alcance de esta investigación se centra específicamente en la detección automática de **desinformación**, abarcando tanto noticias falsas como bulos, reconociendo que ambos tipos requieren estrategias de procesamiento de lenguaje natural diferenciadas pero complementarias.

4.2.3. Justificación de la unificación terminológica

En el contexto del desarrollo de sistemas automáticos de detección, la distinción entre noticia falsa y bulo, aunque conceptualmente importante, se aborda mediante un enfoque unificado de **detección de contenido desinformativo**. Esta decisión metodológica se fundamenta en:

- **Características textuales compartidas:** Ambos tipos utilizan patrones lingüísticos identificables mediante técnicas de PLN
- **Objetivo común:** La finalidad de engañar o desinformar a la audiencia
- **Impacto social similar:** Efectos negativos en la percepción pública y toma de decisiones
- **Necesidad práctica:** Los sistemas de detección automática deben ser capaces de identificar ambos tipos

Por tanto, en el resto de este documento, el término “**noticias falsas**” se utilizará de manera inclusiva para referirse a cualquier tipo de contenido desinformativo, reconociendo la diversidad de formatos y estrategias de propagación que abarca esta categorización.

4.2.4. Esquema de clasificación: enfoque binario

Esta investigación adopta un esquema de clasificación binaria, donde cada noticia se etiqueta en una de dos clases mutuamente excluyentes:

- **Clase 0 (FALSO):** Incluye todo contenido desinformativo, abarcando tanto noticias falsas formales como bulos informales, contenido satírico deliberadamente engañoso y cualquier tipo de información con intención de desinformar
- **Clase 1 (REAL):** Incluye contenido periodístico legítimo, noticias verificadas de fuentes confiables e información factualmente correcta

Justificación del enfoque binario

La decisión de utilizar clasificación binaria, en lugar de esquemas multiclase más granulares, se fundamenta en:

1. **Practicidad para usuarios finales:** En aplicaciones reales, los usuarios necesitan una respuesta clara y directa sobre la confiabilidad del contenido
2. **Consistencia con la literatura:** La mayoría de trabajos en detección de noticias falsas utilizan esquemas binarios, facilitando la comparación de resultados
3. **Robustez del modelo:** Los esquemas binarios tienden a ser más robustos y menos propensos al sobreajuste que las clasificaciones multiclase complejas
4. **Disponibilidad de datos:** Los corpus disponibles están mayoritariamente etiquetados de forma binaria

Consideraciones sobre esquemas alternativos

Aunque este trabajo adopta clasificación binaria, es importante reconocer que existen esquemas alternativos más granulares:

- **Clasificación ternaria:** FALSO, REAL, PARCIALMENTE_CIERTO
- **Clasificación por niveles de confianza:** Escalas de 5 o 7 puntos de credibilidad
- **Clasificación por tipo de desinformación:** Distinguiendo entre sátira, propaganda, clickbait, etc.

Sin embargo, estos enfoques requieren corpus más especializados y presentan desafíos adicionales en términos de consistencia de etiquetado y acuerdo entre anotadores.

4.2.5. Fronteras difusas en la clasificación de veracidad

Es importante reconocer que la clasificación binaria de contenido informativo presenta inherentemente zonas grises y fronteras difusas, donde la distinción entre “verdadero” y “falso” no es absoluta. Esta complejidad conceptual representa uno de los desafíos fundamentales en la detección automatizada de desinformación.

Casos límite y ambigüedades

En la práctica, existen varios tipos de contenido que desafían la clasificación binaria estricta:

- **Información parcialmente correcta:** Noticias que contienen algunos hechos verificables mezclados con información falsa o tergiversada
- **Contenido satírico ambiguo:** Material humorístico que puede ser malinterpretado como información factual
- **Opiniones presentadas como hechos:** Contenido subjetivo que se presenta con apariencia de objetividad periodística
- **Información desactualizada:** Noticias que fueron ciertas en el pasado pero ya no son válidas
- **Interpretaciones sesgadas:** Presentación de hechos reales con marcos interpretativos tendenciosos
- **Información especulativa:** Predicciones o análisis presentados como hechos consumados

Implicaciones para el modelado

Esta ambigüedad inherente se aborda en esta investigación mediante:

1. **Criterios de etiquetado conservadores:** En casos ambiguos, se prioriza la asignación a la clase “REAL” para evitar censura inadecuada de contenido legítimo
2. **Umbral de confianza:** El modelo genera probabilidades que pueden interpretarse como niveles de certeza, permitiendo identificar casos limítrofes
3. **Validación humana en casos dudosos:** Los corpus utilizados fueron validados manualmente para resolver ambigüedades
4. **Reconocimiento de limitaciones:** Se asume que un porcentaje mínimo de casos será inherentemente ambiguo y difícil de clasificar con certeza absoluta

Justificación pragmática del enfoque binario

A pesar de estas complejidades, el enfoque binario se justifica porque:

- **Utilidad práctica:** Los usuarios finales requieren decisiones claras sobre la credibilidad del contenido
- **Viabilidad técnica:** Los modelos binarios son más robustos y fáciles de entrenar que los esquemas multiclase complejos
- **Consistencia metodológica:** Permite comparación directa con la literatura existente
- **Escalabilidad:** Facilita la anotación y validación de grandes volúmenes de datos

Es fundamental entender que **la frontera difusa no es una limitación del modelo, sino una característica inherente del problema mismo**, que debe ser gestionada mediante diseño metodológico cuidadoso y reconocimiento explícito de las limitaciones del enfoque adoptado.

4.3. Construcción del corpus unificado

El primer y más fundamental paso de la metodología fue la construcción de un corpus de alta calidad y de tamaño significativo en español, dada la escasez documentada de recursos centralizados para esta tarea [18].

4.3.1. Fuentes de datos académicas

Se llevó a cabo un proceso exhaustivo de investigación y unificación de cuatro corpus académicos reconocidos, los cuales constituyen pilares en la investigación de noticias falsas en español. **Es importante señalar que aunque todos los corpus convergen hacia el objetivo común de detección de desinformación, cada uno fue desarrollado con enfoques metodológicos y propósitos específicos diferentes**, lo que enriquece la diversidad y robustez del conjunto de datos unificado. La Tabla 4.1 presenta un resumen detallado de estos recursos y sus características distintivas.

Spanish Fake News Corpus (IberLEF)

Este corpus, asociado a las competencias de IberLEF y desarrollado por Posadas-Durán, Gómez-Adorno, et al., ha tenido varias versiones. La versión original y más citada del corpus contiene un total de 971 noticias [12]. **Su propósito específico fue el desarrollo de técnicas de análisis estilométrico para identificar patrones lingüísticos distintivos en texto desinformativo.** Este corpus se caracteriza

ID	Nombre del Corpus	Autores Principales	Año	Noticias	Características	Ref.
1	Spanish Fake News Corpus (IberLEF)	Posadas-Durán, J.P. Gómez-Adorno, H.	2019-2021	971	Análisis estilométrico Múltiples versiones	[12]
2	Conjunto de datos Zules Acosta (UPM)	Acosta, F.A.Z.	2019	598	Trabajo fin de maestría Extracción web verificada	[11]
3	Conjunto de datos Tretiakov (Kaggle)	Tretiakov, A. Martín García, A.	2022	1,958	Aprendizaje automático Disponible públicamente	[19]
4	Spanish Political Fake News Conjunto de datos	Blanco-Fernández, Y. Otero-Vizoso, J.	2024	57,231	Temática política Modelos BERT/roBERTa	[13]
TOTAL CORPUS ACADÉMICOS					60,758	Cuatro fuentes

Tabla 4.1: Corpus académicos utilizados para la construcción del conjunto de datos unificado.

por su enfoque metodológico riguroso en análisis estilométrico y ha sido utilizado en múltiples evaluaciones comparativas dentro del marco de las competencias IberLEF [46, 45]. **Las fuentes de este corpus provienen principalmente de sitios web verificados como desinformativos por organismos de verificación de datos españoles y latinoamericanos.** Las versiones posteriores del corpus han sido mantenidas y actualizadas en repositorios públicos [47].

Conjunto de datos de Zules Acosta (Universidad Politécnica de Madrid)

Desarrollado como parte de un Trabajo Fin de Maestría Universitaria en Ciberseguridad en la Universidad Politécnica de Madrid, este corpus contiene 598 noticias verificadas [11]. **Su propósito original fue establecer una metodología de construcción de corpus para noticias falsas en español, con énfasis en la verificación manual exhaustiva.** El conjunto de datos fue construido mediante técnicas de extracción web y se encuentra disponible públicamente en la plataforma Kaggle [61, 62]. **Las fuentes incluyen tanto medios legítimos (para noticias reales) como sitios identificados como propagadores de desinformación, con verificación cruzada mediante múltiples verificadores de hechos.** Su contribución principal radica en la aplicación de metodologías rigurosas de verificación para la construcción de conjuntos de datos de noticias falsas.

Dataset de Tretiakov et al.

Este corpus, desarrollado por Tretiakov, Martín García y Camacho, contiene 1,958 noticias en español (Corpus balanceado, 50 % de noticias falsas y 50 % de noticias verdaderas) [19]. **Su propósito específico fue la evaluación de técnicas de aprendizaje automático tradicionales para clasificación binaria de contenido desinformativo.** El conjunto de datos fue creado en el contexto de técnicas de aprendizaje automático para la detección de información falsa y se encuentra disponible públicamente en Kaggle [63]. **Las fuentes de este corpus se concentran en redes sociales y plataformas digitales donde se propaga desinformación, incluyendo Twitter, Facebook y sitios web no verificados.** Su enfoque se centra en la aplicación de técnicas de aprendizaje automático para la clasificación automatizada de contenido desinformativo.

Spanish Political Fake News Dataset

El corpus más extenso utilizado, desarrollado por Blanco-Fernández et al., contiene 57,231 noticias de temática política [13]. **Su propósito específico fue evaluar modelos transformer (BERT/RoBERTa) en el dominio político, donde la desinformación tiene particular relevancia social.** Este conjunto de datos fue específicamente diseñado para evaluar modelos BERT y RoBERTa en la detección de desinformación política en español y se encuentra disponible en Kaggle [64]. **Las fuentes incluyen tanto medios de comunicación tradicionales como plataformas digitales especializadas en contenido político, con particular énfasis en el contexto español e iberoamericano.** Su gran tamaño y enfoque en contenido político lo convierte en un recurso valioso para el entrenamiento de modelos de gran escala.

4.3.2. Análisis comparativo detallado de los corpus utilizados

Para garantizar una comprensión completa de las características y diferencias entre los corpus utilizados, se presenta un análisis comparativo exhaustivo que permite entender las fortalezas y limitaciones de cada fuente de datos, así como las decisiones metodológicas tomadas en el proceso de unificación (ver Tabla 4.2).

Aspecto Comparativo	Posadas-Durán	Acosta (UPM)	Tretiakov	Blanco-Fernández	El Deforma
Tamaño del corpus	971 noticias	598 noticias	1,958 noticias	57,231 noticias	9,000 noticias
Año de creación	2019-2021	2019	2022	2024	2025 (extracción)
Enfoque metodológico	Análisis estilométrico Competencias IberLEF	Metodología de construcción de corpus	Aprendizaje automático tradicional	Modelos Transformer BERT/RoBERTa	Contenido satírico Web scraping
Dominio temático	General	General	General	Político específico	Satírico/General
Distribución de clases	Balanceado	Balanceado	Solo falsas	Balanceado	Solo falsas
Fuentes de noticias reales	Medios españoles verificados	Múltiples medios internacionales	N/A	Medios políticos españoles	N/A
Fuentes de noticias falsas	Sitios verificados como desinformativos	Fact-checkers y verificadores	Redes sociales y sitios no verificados	Desinformación política verificada	Portal satírico El Deforma
Variabilidad regional	España/Latinoamérica	Internacional	Múltiple	España	México
Periodo temporal	2018-2021	2018-2019	2020-2022	2020-2024	2019-2025
Calidad de anotación	Alta - múltiples anotadores	Alta - verificación manual exhaustiva	Media - basada en fuentes	Alta - verificación política especializada	Automática - contenido inherentemente falso
Metadatos incluidos	Características estilométricas	Fuente, fecha, categoría	Fecha, fuente, contexto	Orientación política, fecha, fuente	URL, fecha de extracción
Disponibilidad pública	GitHub (IberLEF)	Kaggle UPM	Kaggle Público	Kaggle Applied Sciences	Generado para esta tesis
Fortaleza principal	Análisis lingüístico profundo	Metodología rigurosa	Foco en ML tradicional	Gran escala y especialización política	Contemporaneidad y diversidad
Limitación principal	Tamaño limitado	Tamaño muy pequeño	Solo noticias falsas	Dominio muy específico	Solo contenido satírico
Contribución al corpus final	1.6 %	1.0 %	3.2 %	92.8 %	14.6 %

Tabla 4.2: Comparación exhaustiva de características de los corpus utilizados en la construcción del conjunto de datos unificado.

Características de variabilidad regional del español

Una consideración metodológica fundamental en esta investigación es el reconocimiento y aprovechamiento de la diversidad regional del español presente en los corpus utilizados. **El corpus unificado representa un enfoque de español neutro multiregional**, lo que constituye una fortaleza metodológica significativa:

Distribución geográfica y justificación:

- **Español peninsular (España):** Representado principalmente en los corpus de Posadas-Durán [12] y Blanco-Fernández [13], que utilizan fuentes de medios españoles verificados

- **Español mexicano:** Incorporado a través del contenido de “El Deforma” y fuentes mexicanas presentes en el corpus de Acosta [11]
- **Español latinoamericano diverso:** El corpus de Acosta incluye fuentes internacionales que abarcan múltiples países de habla hispana
- **Español digital contemporáneo:** Los corpus más recientes (2020-2025) capturan la evolución del español en medios digitales

Ventajas metodológicas del enfoque multiregional:

- **Robustez del modelo:** Capacidad de detectar desinformación independientemente de la variante regional
- **Generalización lingüística:** Menor sesgo hacia construcciones específicas de una región
- **Aplicabilidad universal:** El modelo resultante es funcional para toda la comunidad hispanohablante
- **Representatividad real:** Refleja el consumo actual de noticias en español, que frecuentemente cruza fronteras nacionales

Esta diversidad regional no constituye una limitación, sino una fortaleza que permite desarrollar modelos más robustos y ampliamente aplicables.

1. Diversidad metodológica:

- **Corpus de Posadas-Durán:** Aporta rigor en análisis estilométrico y experiencia en competencias académicas
- **Corpus de Acosta:** Contribuye con metodología de construcción verificada manualmente
- **Corpus de Tretiakov:** Añade perspectiva de ML tradicional y fuentes de redes sociales
- **Corpus de Blanco-Fernández:** Proporciona escala masiva y especialización en dominio político
- **Contenido de El Deforma:** Introduce diversidad estilística y contemporaneidad

2. Cobertura temporal y geográfica:

- **Span temporal:** 2018-2025 (7 años de evolución en patrones de desinformación)
- **Cobertura geográfica:** España, México, Latinoamérica (diversidad regional del español)

- **Evolución temática:** Desde temas generales hasta especialización política y satírica

3. Complementariedad en tipos de desinformación:

- **Desinformación tradicional:** Corpus académicos con verificación formal
- **Desinformación política:** Especialización del corpus de Blanco-Fernández
- **Contenido satírico:** Ampliación con El Deforma para diversidad estilística
- **Múltiples fuentes:** Desde medios tradicionales hasta redes sociales

4. **Validación de robustez:** La diversidad en metodologías de construcción permite validar la robustez de los modelos desarrollados:

- Modelos entrenados deben generalizar entre diferentes tipos de anotación
- Capacidad de detectar patrones consistentes a pesar de diferencias metodológicas
- Evaluación de transferencia entre dominios (general $\rightarrow$ político $\rightarrow$ satírico)

Justificación de la estrategia de unificación

La decisión de unificar corpus heterogéneos se basa en varios principios metodológicos sólidos:

1. Principio de máxima diversidad:

- Mayor variabilidad en patrones lingüísticos
- Reducción de sesgos específicos de corpus individuales
- Mejora en generalización de modelos entrenados

2. Principio de escala efectiva:

- Aprovechamiento del corpus grande (Blanco-Fernández) como base
- Enriquecimiento con corpus especializados más pequeños
- Balanceo mediante extracción web controlada

3. Principio de validación cruzada:

- Cada corpus actúa como validación independiente de los otros
- Patrones consistentes entre corpus indican robustez
- Diferencias señalan áreas que requieren atención especial

4.3.3. Proceso de unificación y estandarización

La integración de corpus heterogéneos requirió un proceso sistemático de estandarización que incluyó:

1. **Normalización de formato:** Unificación de esquemas de etiquetado en un sistema binario consistente (FALSO=0, REAL=1) y estandarización de estructuras de datos CSV con separador de punto y coma
2. **Eliminación de duplicados:** Implementación de algoritmos de detección de contenido similar basados en hash de texto y comparación de títulos
3. **Validación de calidad:** Verificación manual de una muestra estadísticamente significativa para confirmar la calidad del etiquetado y consistencia temática
4. **Balanceo de clases:** Análisis detallado de la distribución de clases para identificar necesidades de ampliación y garantizar equilibrio. El balanceo se refiere a mantener una proporción similar entre noticias “FALSAS” y “REALES” (idealmente cercana al 50 %-50 %) para evitar sesgos algorítmicos y garantizar que los modelos aprendan a detectar ambas clases con igual efectividad
5. **Limpieza de datos:** Eliminación de registros con valores nulos o inconsistentes mediante `dropna()` y validación de tipos de datos

4.3.4. Ampliación del corpus mediante extracción web

Para mejorar el balance del conjunto de datos y aumentar la diversidad de las noticias etiquetadas como “FALSAS”, se aplicaron técnicas de extracción web automatizada. Se desarrolló un script especializado en Python utilizando las librerías BeautifulSoup y Requests para extraer de forma sistemática los titulares y cuerpos de noticia del portal de contenido satírico “El Deforma”.

Proceso de extracción web automatizada

El proceso de extracción automatizada se ejecutó en múltiples fases para alcanzar el objetivo de 9,000 noticias adicionales. La Tabla 4.3 detalla las diferentes etapas implementadas.

Implementación técnica de la extracción web

El proceso de extracción automatizada incluyó:

1. **Identificación de patrones:** Análisis de la estructura HTML del sitio web objetivo para identificar selectores CSS consistentes (`h1.tdb-title-text`, `div.tdb-block-inner`)

Fase	Estrategia	Técnica Utilizada	Objetivo	Resultado	Observaciones
1	Scraping Inicial	Navegación por enlaces Extracción de contenido	1,000	1,000	Proceso exitoso Base establecida
2	Búsqueda Expandida Masiva	URLs desde existentes Crawling híbrido	9,000	2,495	Expansión limitada Nuevas estrategias
3	Crawler Híbrido Persistente	URLs semilla Trafilatura	9,000	2,495	Estabilización Mismo resultado
4	Paginación Sistemática	Escaneo por páginas Indexación completa	9,000	9,000	Éxito completo Objetivo alcanzado
TOTAL EXTRAÍDO				9,000	Extracción web completada

Tabla 4.3: Fases del proceso de extracción web implementado para “El Deforma”.

- Extracción automatizada:** Implementación de rutinas de extracción con manejo de errores y delays aleatorios (1.5-4.5 segundos) para evitar sobrecarga del servidor
- Validación de contenido:** Verificación automática de la calidad del contenido extraído mediante filtros de longitud mínima (500 caracteres)
- Limpieza de texto:** Eliminación de disclaimers específicos del sitio, caracteres especiales y normalización de espacios mediante expresiones regulares
- Etiquetado automático:** Asignación de etiqueta “FALSO” (0) a todo el contenido extraído del portal satírico
- Persistencia progresiva:** Guardado en lotes de 50 registros con formato CSV y separador de punto y coma para evitar pérdida de datos
- Gestión de estado:** Implementación de archivos de progreso para permitir la reanudación del proceso en caso de interrupción

La Tabla 4.4 presenta las referencias bibliográficas completas de todos los corpus utilizados, organizadas por tipo de fuente.

Corpus	Tipo de Referencia	Fuente Principal	Referencia
Spanish Fake News Corpus (IberLEF)	Artículo original	Journal of Intelligent and Fuzzy Systems	[12]
Spanish Fake News Corpus (IberLEF)	Workshop IberLEF 2021	Procesamiento del Lenguaje Natural	[46]
Spanish Fake News Corpus (IberLEF)	Workshop IberLEF 2020	IberLEF Workshop Proceedings	[45]
Spanish Fake News Corpus (IberLEF)	Repositorio GitHub	GitHub Repository	[47]
Conjunto de datos Zules Acosta (UPM)	Tesis de Maestría	Universidad Politécnica de Madrid	[11]
Dataset Zules Acosta (UPM)	Dataset Kaggle	Kaggle Platform	[62]
Dataset Tretiakov (Kaggle)	Capítulo de libro	Springer Book Chapter	[19]
Dataset Tretiakov (Kaggle)	Dataset Kaggle	Kaggle Platform	[63]
Spanish Political Fake News Dataset	Artículo científico	Applied Sciences Journal	[13]
Spanish Political Fake News Dataset	Dataset Kaggle	Kaggle Platform	[64]

Tabla 4.4: Referencias bibliográficas completas de los corpus académicos utilizados.

La estrategia final exitosa utilizó paginación sistemática, escaneando secuencialmente desde <https://eldeforma.com/page/1/> hasta alcanzar las 9,000 noticias objetivo, garantizando cobertura completa del contenido disponible.

Esta estrategia de ampliación se justifica por varios factores:

- **Diversidad estilística:** Incorporación de diferentes estilos de contenido falso o satírico
- **Volumen de datos:** Incremento significativo del conjunto de entrenamiento
- **Actualidad temporal:** Inclusión de contenido contemporáneo que refleja tendencias actuales
- **Variabilidad temática:** Ampliación del espectro de temas cubiertos en el corpus

4.3.5. Corpus final y estrategia de división

El proceso completo de unificación, limpieza de duplicados y ampliación mediante extracción web resultó en un **corpus final con 61,674 noticias únicas**, constituyendo uno de los recursos más extensos disponibles para la detección de noticias falsas en español.

Análisis de composición final

La Tabla 4.5 presenta la composición detallada del corpus unificado final.

Fuente de Datos	Noticias	Porcentaje	Tipo	Estado
Corpus Académicos Unificados	52,689	85.4 %	Mixto	Procesado
Extracción Web “El Deforma”	9,000	14.6 %	Falso	Extraído
Duplicados Eliminados	-15	-0.02 %	–	Removido
CORPUS FINAL	61,674	100 %	Balanceado	Listo

Tabla 4.5: Composición final del corpus unificado después del procesamiento completo.

El análisis de balance del corpus final mostró una distribución equilibrada:

- **Noticias FALSAS (0):** 30,734 registros (49.8 %)
- **Noticias REALES (1):** 30,940 registros (50.2 %)

Concepto e importancia del balanceo de clases

El **balanceo de clases** es un concepto fundamental en el aprendizaje automático que se refiere a la distribución equitativa de ejemplos entre las diferentes categorías o clases del conjunto de datos. En el contexto de la detección de noticias falsas, esto significa mantener una proporción similar entre noticias etiquetadas como “FALSAS” y “REALES”.

¿Por qué es crucial el balanceo?

1. **Prevención de sesgo algorítmico:** Un conjunto de datos desbalanceado puede llevar a que el modelo aprenda a predecir sistemáticamente la clase mayoritaria, ignorando la minoritaria

2. **Métricas de evaluación confiables:** Las métricas como exactitud pueden ser engañosas en datasets desbalanceados (ejemplo: 95 % de exactitud puede significar que el modelo siempre predice la clase mayoritaria)
3. **Generalización robusta:** Los modelos entrenados con datos balanceados tienden a generalizar mejor en escenarios reales donde la distribución puede ser diferente
4. **Detección efectiva de ambas clases:** Es igualmente importante detectar noticias falsas como preservar noticias verdaderas

Estrategias implementadas para lograr el balanceo:

- **Análisis cuantitativo inicial:** Evaluación de la distribución de clases en cada corpus individual
- **Identificación de déficits:** Detección de desbalances significativos que requerían corrección
- **Extracción web dirigida:** Adición estratégica de 9,000 noticias falsas mediante web scraping para compensar el déficit
- **Validación final:** Confirmación de que la distribución resultante es prácticamente equilibrada (49.8 % vs 50.2 %)

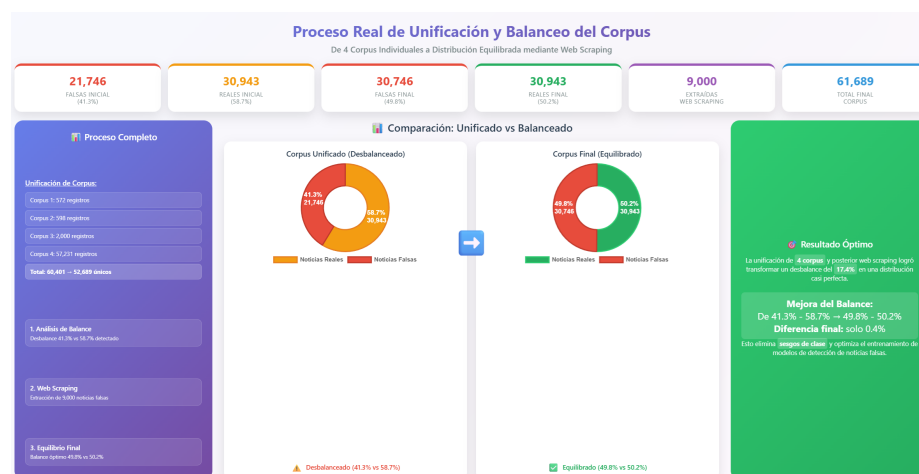


Figura 4.3: Representación gráfica del proceso de balanceo del corpus: comparación entre distribución inicial desbalanceada y distribución final equilibrada. La imagen muestra cómo la extracción web de 9,000 noticias falsas permitió alcanzar un equilibrio óptimo de 49.8 % noticias falsas vs 50.2 % noticias reales.

La Figura 4.3 ilustra gráficamente el concepto de balanceo implementado, mostrando la transformación desde una distribución inicial potencialmente sesgada hacia

la distribución final equilibrada que optimiza el rendimiento de los modelos de clasificación.

Impacto del balanceo en el rendimiento:

El balanceo adecuado del corpus tiene impactos directos en múltiples aspectos del proyecto:

- **Entrenamiento más estable:** Los algoritmos convergen de manera más consistente
- **Métricas más interpretables:** F1-Score, precisión y recall reflejan el rendimiento real
- **Transferibilidad mejorada:** Los modelos se adaptan mejor a nuevos dominios
- **Validación confiable:** Los resultados de validación cruzada son más representativos

Síntesis del análisis comparativo entre corpus

El análisis detallado de los corpus permite identificar patrones complementarios que justifican la estrategia de unificación adoptada:

1. Análisis cuantitativo comparativo:

- **Distribución de tamaños:** Desde corpus micro (598 noticias) hasta macro (57,231 noticias)
- **Diversidad temporal:** Cobertura continua de 7 años (2018-2025) entre todos los corpus
- **Representación geográfica:** Equilibrio entre España (3 corpus), México (1 corpus) y cobertura internacional (1 corpus)

2. Análisis cualitativo de complementariedad:

- **Metodologías de verificación:** Combinación de verificación manual experta, fact-checking automatizado y verificación basada en fuentes
- **Dominios temáticos:** Progresión desde contenido general hasta especialización política y satírica
- **Tipos de desinformación:** Desde noticias completamente fabricadas hasta manipulación política y contenido satírico

3. Fortalezas emergentes de la unificación: La combinación de estos corpus heterogéneos produce un conjunto de datos con características emergentes superiores a la suma de sus partes:

- **Robustez metodológica:** Los modelos entrenados deben generalizar entre diferentes criterios de anotación

- **Diversidad estilística:** Desde registro formal académico hasta contenido informal de redes sociales
- **Cobertura temática exhaustiva:** Temas generales, especialización política, contenido satírico
- **Validación cruzada inherente:** Cada corpus actúa como conjunto de validación independiente

4. Limitaciones reconocidas y mitigación:

- **Heterogeneidad metodológica:** Mitigada mediante estandarización rigurosa del proceso de unificación
- **Desbalance de tamaños:** Compensado mediante estrategia de extracción web controlada
- **Diferencias temporales:** Convertidas en ventaja para evaluar evolución de patrones

5. Validación de la estrategia comparativa: El proceso de comparación mutua entre corpus revela que cada fuente aporta elementos únicos e irremplazables:

- **Corpus pequeños especializados:** Aportan calidad y metodología rigurosa
- **Corpus grande generalista:** Proporciona escala y diversidad temática amplia
- **Contenido web extraído:** Añade contemporaneidad y variabilidad estilística
- **Combinación sinérgica:** Produce un recurso superior a cualquier corpus individual

El resultado de esta estrategia de comparación y unificación es un corpus robusto de 61,674 noticias que mantiene la diversidad y riqueza de las fuentes originales mientras proporciona la escala necesaria para entrenar modelos de deep learning efectivos. Esta aproximación metodológica garantiza que los modelos desarrollados sean capaces de generalizar efectivamente entre diferentes tipos de contenido desinformativo y contextos de aplicación.

División estratificada para entrenamiento

Para garantizar una evaluación robusta y evitar el sobreajuste, este corpus se dividió de manera estratificada, manteniendo la proporción original de noticias falsas y reales en cada subconjunto. La configuración utilizada fue **70 % para entrenamiento, 10 % para validación y 20 % para evaluación final**. Esta división garantiza:

- **Conjunto de entrenamiento suficiente:** 70 % proporciona datos abundantes para el aprendizaje de patrones complejos

- **Validación efectiva:** 10 % permite calibración de hiperparámetros sin comprometer datos de prueba
- **Evaluación objetiva:** 20 % reservado exclusivamente para pruebas finales sin contaminar el proceso de desarrollo

La Tabla 4.6 muestra la distribución implementada en todos los experimentos.

Conjunto de Datos	Porcentaje	Noticias	Propósito
Entrenamiento	70 %	43,171	Entrenar modelos
Validación	10 %	6,167	Calibrar hiperparámetros
Pruebas	20 %	12,336	Evaluación final
TOTAL	100 %	61,674	Metodología completa

Tabla 4.6: División estratificada del corpus para entrenamiento y evaluación.

Esta estrategia sigue las mejores prácticas establecidas en la literatura de aprendizaje automático y permite una evaluación imparcial de ambas metodologías. La estratificación garantiza que cada subconjunto mantenga la misma proporción de noticias falsas y reales que el corpus completo, evitando sesgos en el entrenamiento y evaluación de los modelos.

Gestión de sesgo en el dataset:

Para minimizar posibles sesgos en el conjunto de datos, se implementaron las siguientes medidas:

1. **Estratificación rigurosa:** Uso de `train_test_split` con parámetro `stratify` para mantener distribución de clases en cada partición
2. **Aleatorización controlada:** Semilla fija (`random_state=42`) para reproducibilidad sin pérdida de aleatoriedad
3. **Análisis de distribución:** Verificación estadística de la distribución de clases en cada subconjunto mediante tests chi-cuadrado
4. **Validación cruzada estratificada:** Uso de k-fold estratificado ($k=5$) durante la optimización de hiperparámetros para garantizar robustez

Proceso de limpieza final

El proceso de limpieza final implementó las siguientes operaciones:

1. **Eliminación de valores nulos:** Remoción de registros con campos vacíos en texto o etiquetas mediante `dropna()`
2. **Detección de duplicados:** Identificación y eliminación de 15 registros duplicados basada en contenido textual idéntico

3. **Normalización de caracteres:** Eliminación de punto y coma y caracteres especiales para evitar conflictos con el formato CSV
4. **Compactación de espacios:** Reducción de espacios múltiples y normalización de saltos de línea mediante expresiones regulares
5. **Mezclado aleatorio:** Randomización del orden con semilla fija (`random_state=42`) para garantizar reproducibilidad
6. **Validación de tipos:** Conversión de etiquetas a enteros mediante `astype(int)` para consistencia de datos

El resultado final fue un corpus robusto, balanceado y limpio, optimizado para el entrenamiento de modelos de detección de noticias falsas en español, que constituye una de las contribuciones principales de este trabajo de investigación al proporcionar un recurso de gran escala para la comunidad hispanohablante.

4.4. Enfoque 1: detección mediante algoritmos metaheurísticos

El primer enfoque metodológico se basó en técnicas clásicas de PLN combinadas con algoritmos de optimización metaheurística, sirviendo como línea base robusta para el proyecto y permitiendo la comparación con enfoques más modernos.

4.4.1. Preprocesamiento y representación textual

El flujo de datos de preprocesamiento implementó una serie de transformaciones estándar para convertir el texto crudo en representaciones numéricas adecuadas para algoritmos de aprendizaje automático.

Función de limpieza de texto

Se implementó una función optimizada de limpieza que incluye los siguientes pasos:

1. **Validación de entrada:** Verificación de que el input sea de tipo string válido
2. **Normalización a minúsculas:** Conversión completa del texto usando `lower()`
3. **Eliminación de URLs:** Remoción de enlaces web mediante expresiones regulares
4. **Limpieza de caracteres especiales:** Preservación únicamente de caracteres alfabéticos en español
5. **Tokenización con NLTK:** División del texto usando `word_tokenize()`

6. **Eliminación de stopwords:** Exclusión de palabras funcionales sin valor semántico
7. **Filtrado por longitud:** Remoción de tokens menores a 3 caracteres

Representación TF-IDF

Tras el preprocesamiento, se aplicó la técnica TF-IDF utilizando `TfidfVectorizer` de scikit-learn. La configuración específica incluyó:

- **Máximo de características:** 5,000 palabras más frecuentes
- **Matriz resultante:** Representación densa convertida con `toarray()`
- **Almacenamiento:** Guardado en formato CSV para reutilización

La ponderación TF-IDF se calculó utilizando las siguientes fórmulas:

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \quad (4.1)$$

$$\text{IDF}(t, D) = \log \frac{|D|}{|\{d \in D : t \in d\}|} \quad (4.2)$$

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D) \quad (4.3)$$

Donde t representa un término, d un documento, D la colección completa de documentos y $f_{t,d}$ la frecuencia del término t en el documento d .

4.4.2. Algoritmos metaheurísticos implementados

Para la optimización de hiperparámetros de clasificadores, se implementaron y evaluaron cinco algoritmos metaheurísticos específicos [23, 34]. La Tabla 4.7 presenta una visión general de los algoritmos seleccionados y sus características principales.

Algoritmo	Inspiración	Principio Base	Estrategia Principal	Ref.
MSA	Metalurgia	Recocido de metales Enfriamiento controlado	Múltiples puntos de inicio Aceptación probabilística	[55, 56]
SS	Metodología científica	Búsqueda sistemática Combinación estructurada	Conjunto de referencia Generación de subconjuntos	[57]
VNS	Exploración geográfica	Cambio sistemático de vecindarios	Exploración local Diversificación estructural	[58]
GA	Evolución natural	Selección natural Supervivencia del más apto	Operadores genéticos Evolución poblacional	[52]
PSO	Comportamiento social	Inteligencia de enjambre Comunicación colectiva	Movimiento de partículas Intercambio de información	[53, 54]

Tabla 4.7: Algoritmos metaheurísticos implementados y sus fundamentos conceptuales.

Recocido Multiarranque (Multi-Start Simulated Annealing - MSA)

El algoritmo MSA combina los principios del Recocido Simulado [55] con estrategias de métodos Multi-arranque [56]. Se inspira en el proceso metalúrgico de recocido, donde los metales se calientan y luego se enfrían lentamente para alcanzar estados de menor energía y mayor estabilidad estructural. En el contexto de optimización, este proceso se traduce en la capacidad de escapar de óptimos locales mediante la aceptación probabilística de soluciones temporalmente peores.

El algoritmo MSA implementado incluye las siguientes características:

Parámetro	Valor	Descripción
Temperatura inicial (TI)	1000	Energía inicial alta para exploración amplia
Temperatura final (TF)	1	Estado de convergencia con poca aleatoriedad
Factor de enfriamiento (α)	0.8	Tasa de reducción geométrica de temperatura
Pasos por temperatura	100	Iteraciones en cada nivel térmico
Puntos de arranque múltiples	5	Inicializaciones independientes
Iteraciones totales	31 temperaturas	Niveles de enfriamiento desde TI hasta TF

Tabla 4.8: Configuración de parámetros del algoritmo MSA.

- **Función de aceptación:** $P = \exp(\Delta E/T)$ donde ΔE es la diferencia de evaluación
- **Esquema de enfriamiento:** Geométrico con $T_{n+1} = \alpha \times T_n$
- **Estrategia multiarranque:** Múltiples ejecuciones independientes para mayor robustez

Búsqueda Dispersa (Scatter Search - SS)

La Búsqueda Dispersa, desarrollada por Glover [57], se basa en principios de metodología científica, combinando elementos de diversas soluciones de alta calidad para generar nuevas candidatas. Su filosofía se centra en la combinación sistemática y la mejora estructurada, similar a como los investigadores combinan diferentes enfoques para obtener mejores resultados.

Parámetro	Valor	Descripción
Tamaño de población (P)	50	Población inicial diversa para exploración
Conjunto de referencia (b)	5	Mejores soluciones seleccionadas
Máximo de iteraciones	10	Ciclos de mejora y actualización
Combinaciones por iteración	10	Nuevas soluciones generadas por ciclo

Tabla 4.9: Configuración de parámetros del algoritmo SS.

La implementación incorpora:

- **Método de combinación:** Cruce de un punto con índice aleatorio

- **Estrategia de mejora:** Mutación en posición aleatoria
- **Actualización de RefSet:** Conservación de las mejores soluciones ordenadas por fitness

Algoritmo Genético (Genetic Algorithm - GA)

El Algoritmo Genético, fundamentado en el trabajo original de Holland [52], emula los procesos de evolución natural descritos por Darwin, donde las especies más aptas tienen mayor probabilidad de sobrevivir y reproducirse. En optimización, este principio se traduce en la evolución de poblaciones de soluciones mediante operadores inspirados en la genética: selección, cruce y mutación.

Parámetro	Valor	Descripción
Número de generaciones	20	Ciclos evolutivos completos
Tamaño de población	50	Individuos por generación
Tasa de mutación	0.1	Probabilidad de modificación genética
Tamaño de torneo	3	Individuos competidores en selección

Tabla 4.10: Configuración de parámetros del algoritmo GA.

Los operadores genéticos implementados incluyen:

- **Selección por torneo determinística:** Competencia entre individuos para reproducción
- **Cruce de un punto:** Intercambio genético con hijos complementarios
- **Mutación uniforme:** Modificación aleatoria condicionada por tasa

Búsqueda en Vecindades Variables (Variable Neighborhood Search - VNS)

VNS, introducida por Mladenović y Hansen [58], se inspira en la exploración geográfica sistemática, donde se cambian las estrategias de búsqueda (vecindarios) de manera estructurada para evitar quedar atrapado en regiones subóptimas. Su principio fundamental es que diferentes estructuras de vecindario pueden revelar diferentes aspectos del paisaje de optimización.

Parámetro	Valor	Descripción
Máximo de iteraciones	20	Ciclos de búsqueda completos
Vecindarios máximos (k_max)	5	Estructuras de vecindad diferentes
Estrategia de vecindario	k elementos aleatorios	Modificación de k componentes simultáneas

Tabla 4.11: Configuración de parámetros del algoritmo VNS.

Las características de implementación incluyen:

- **Criterio de mejora:** Aceptación únicamente de soluciones superiores

- **Reinicio de vecindarios:** Retorno a $k=1$ tras mejora encontrada
- **Diversificación sistemática:** Exploración progresiva de vecindarios más amplios

Optimización por Enjambre de Partículas (Particle Swarm Optimization - PSO)

PSO, desarrollado por Kennedy y Eberhart [53], se fundamenta en el comportamiento social observado en enjambres naturales como bandadas de aves o cardúmenes de peces. Las partículas (soluciones) se mueven en el espacio de búsqueda influenciadas por su propia experiencia y la información colectiva del enjambre, creando un mecanismo de inteligencia emergente.

Parámetro	Valor	Descripción
Número de partículas	30	Agentes en el enjambre
Máximo de iteraciones	20	Movimientos del enjambre
Factor de inercia (w)	0.5	Influencia del movimiento previo
Coefficiente cognitivo (c_1)	1.5	Peso de la experiencia personal
Coefficiente social (c_2)	1.5	Peso de la experiencia colectiva

Tabla 4.12: Configuración de parámetros del algoritmo PSO.

El comportamiento del enjambre se rige por:

- **Inicialización:** Posiciones aleatorias en $[-5, 5]$ y velocidades gaussianas
- **Actualización de velocidad:** $v_{t+1} = w \cdot v_t + c_1 \cdot r_1 \cdot (p_{best} - x_t) + c_2 \cdot r_2 \cdot (g_{best} - x_t)$
- **Comunicación social:** Intercambio de información sobre mejores posiciones encontradas

4.4.3. Función de evaluación y clasificación

Todos los algoritmos metaheurísticos utilizan una función de evaluación común que implementa un clasificador logístico binario:

$$P(y = 1|x) = \frac{1}{1 + \exp(-w^T \cdot x_{binario})} \quad (4.4)$$

Donde:

- w representa el vector de pesos optimizado
- $x_{binario}$ es la versión binarizada de las características usando umbrales
- La decisión final se toma con umbral de 0.5

4.4.4. Reducción de dimensionalidad

Para hacer computacionalmente factible la optimización, se aplicó reducción de dimensionalidad usando `SelectPercentile` con selección del 10 % de las características más relevantes según el test chi-cuadrado (El test chi-cuadrado es una prueba estadística que mide si dos cosas están relacionadas entre sí. En nuestro caso, mide qué tan relacionada está cada palabra con el hecho de que una noticia sea falsa o real). La Tabla 4.13 resume el proceso de selección de características.

Aspecto	Configuración	Justificación
Método de selección	SelectPercentile	Selección basada en estadísticas univariadas
Porcentaje seleccionado	10 %	Balance entre información y eficiencia
Test estadístico	Chi-cuadrado	Adecuado para clasificación binaria
Características originales	5,000	Vocabulario TF-IDF completo
Características reducidas	500	Dimensionalidad manejable

Tabla 4.13: Configuración del proceso de reducción de dimensionalidad.

Esta estrategia de reducción permite que los algoritmos metaheurísticos operen eficientemente sobre un espacio de características optimizado, manteniendo la información más discriminativa para la tarea de clasificación.

4.5. Enfoque 2: detección mediante modelo Transformer

El segundo enfoque se fundamentó en el paradigma de aprendizaje profundo, específicamente en la arquitectura Transformer [24], utilizando un modelo de lenguaje preentrenado para capturar representaciones contextuales sofisticadas del texto.

4.5.1. Selección y justificación del modelo

Para la selección del modelo óptimo se realizó un análisis comparativo entre los principales modelos BERT optimizados disponibles. La Tabla 4.14 presenta las características técnicas de los candidatos evaluados.

Modelo	Parámetros	Capas	Dimensión	Soporte Español	Reducción vs BERT
BERT-base-multilingual	110M	12	768	Sí	– (Referencia)
DistilBERT-multilingual	66M	6	768	Sí	40 % parámetros
TinyBERT	14.5M	4	312	Limitado	87 % parámetros

Tabla 4.14: Comparación de modelos BERT optimizados para la tarea de clasificación.

Se seleccionó el modelo `distilbert-base-multilingual-cased` por las siguientes razones técnicas y prácticas:

- **Capacidad multilingüe:** Soporte nativo para español y más de 100 idiomas

- **Eficiencia computacional:** Reducción del 40 % en parámetros respecto a BERT base manteniendo el 97 % del rendimiento [20]
- **Arquitectura probada:** Basado en destilación de conocimiento de BERT [25]
- **Balance óptimo:** Mejor relación rendimiento-eficiencia que TinyBERT [26]
- **Especialización en español:** A diferencia de TinyBERT que está diseñado principalmente para inglés, DistilBERT tiene una versión multilingüe específica que incluye entrenamiento extensivo en español, lo cual es crítico para esta investigación que maneja un corpus de español neutro multiregional
- **Disponibilidad:** Accesible a través de la librería Transformers de Hugging Face

4.5.2. Infraestructura computacional

El entrenamiento del modelo DistilBERT se realizó en un entorno de hardware dedicado con las siguientes especificaciones técnicas:

Componente	Especificación	Características Relevantes
Procesador	AMD Ryzen 7 7735H	8 núcleos, 16 hilos 4.75 GHz boost Arquitectura Zen 3+ (6nm)
GPU	NVIDIA GeForce RTX 4060	8GB GDDR6 VRAM 3072 núcleos CUDA 140W TGP Soporte mixed precision
Memoria RAM	16GB DDR5	Capacidad para datasets grandes Procesamiento de batches
Almacenamiento	x2 500GB SSD NVMe	Acceso rápido a datos Checkpointing eficiente

Tabla 4.15: Especificaciones del hardware utilizado para entrenamiento de DistilBERT.

4.5.3. Configuración experimental y optimización de hiperparámetros

El proceso de fine-tuning se implementó utilizando TensorFlow con precisión mixta (mixed_float16) para optimizar el uso de memoria GPU y acelerar el entrenamiento. La configuración experimental se estructuró en múltiples fases de optimización.

Parámetros de entrenamiento base

La configuración base del modelo se estableció considerando las limitaciones computacionales y las mejores prácticas para modelos Transformer:

Parámetro	Valor	Justificación
Longitud máxima de secuencia	128 tokens	Reducido desde 512 Minimizar overfitting
Épocas máximas	30	Suficiente para convergencia
Paciencia early stopping	8 épocas	Evitar sobreentrenamiento
Factor de reducción LR	0.15	Más agresivo que estándar Convergencia controlada
Precisión numérica	mixed_float16	Optimización de memoria GPU

Tabla 4.16: Configuración de parámetros base para el entrenamiento de DistilBERT.

Estrategia de regularización avanzada

Para controlar el overfitting identificado en experimentos preliminares, se implementó una estrategia de regularización intensiva que incluye múltiples técnicas complementarias:

Técnica	Rango Explorado	Valor Óptimo	Propósito
Learning Rate Ultra-bajo	[8e-7, 5e-6]	2e-06	Convergencia controlada
Dropout Agresivo	[0.4, 0.7]	0.7	Prevención de co-adaptación
Regularización L2	[0.05, 0.5]	0.05	Control de pesos
Weight Decay Manual	0.02 fijo	0.02	Aplicado por batch
Noise Injection	[0.01, 0.03]	0.03	Alternativa a label smoothing
Batch Size Variable	{4, 6, 8}	4	Mayor regularización

Tabla 4.17: Estrategias de regularización implementadas para controlar overfitting.

Proceso de búsqueda de hiperparámetros

La optimización de hiperparámetros se realizó mediante una búsqueda sistemática que combinó exploración manual y búsqueda en cuadrícula (grid search) en los rangos especificados. El proceso se estructuró en las siguientes etapas:

1. **Búsqueda inicial:** Exploración amplia de rangos para identificar regiones prometedoras
2. **Refinamiento:** Búsqueda focalizada en torno a los mejores candidatos iniciales
3. **Validación cruzada:** Confirmación de estabilidad con múltiples semillas aleatorias ($k=5$)
4. **Selección final:** Evaluación en conjunto de validación para determinar configuración óptima

4.6. Metodología de evaluación comparativa

Para garantizar una comparación justa y rigurosa entre ambos paradigmas, se estableció un protocolo de evaluación común que eliminara sesgos metodológicos y permitiera una comparación objetiva del rendimiento.

4.6.1. Protocolo de evaluación

Ambos enfoques fueron sometidos al mismo esquema de evaluación estructurado que garantiza la imparcialidad y reproducibilidad de los resultados. La Tabla 4.18 detalla la estructura del protocolo implementado.

Fase	Conjunto	Porcentaje	Propósito	Restricciones
Entrenamiento	Training	70 %	Aprendizaje de parámetros Ajuste de pesos del modelo	Exclusivo para entrenamiento Sin acceso a otros conjuntos
Validación	Validation	10 %	Calibración de hiperparámetros Selección de configuraciones	No utilizado en entrenamiento Guía para optimización
Evaluación Final	Test	20 %	Prueba objetiva Métricas de rendimiento	Completamente no visto Una sola evaluación final

Tabla 4.18: Protocolo de evaluación implementado para ambos paradigmas.

4.6.2. Fundamentos de la matriz de confusión

Para la evaluación de modelos de clasificación binaria en detección de noticias falsas, todas las métricas se derivan de la matriz de confusión, que categoriza las predicciones según su correspondencia con la realidad. La Tabla 4.19 define los cuatro casos posibles.

Categoría	Descripción	Interpretación en Noticias Falsas	Impacto
Verdaderos Positivos (TP)	Predicción: REAL Realidad: REAL	Contenido real correctamente identificado como real	Positivo Preserva información legítima
Verdaderos Negativos (TN)	Predicción: FALSO Realidad: FALSO	Contenido falso correctamente identificado como falso	Positivo Detecta desinformación
Falsos Positivos (FP)	Predicción: REAL Realidad: FALSO	Contenido falso incorrectamente clasificado como real	Negativo Permite propagación de falsedad
Falsos Negativos (FN)	Predicción: FALSO Realidad: REAL	Contenido real incorrectamente clasificado como falso	Negativo Censura información legítima

Tabla 4.19: Definición de categorías de la matriz de confusión en el contexto de detección de noticias falsas.

Representación visual de la matriz de confusión

La estructura de la matriz de confusión para clasificación binaria se puede representar como:

	Predicción del Modelo	
Realidad	REAL	FALSO
REAL	TP	FN
FALSO	FP	TN

Tabla 4.20: Estructura de la matriz de confusión para clasificación binaria.

4.6.3. Marco de métricas de rendimiento

Fundamentación teórica de las métricas según el estado del arte

La selección de métricas de evaluación en este trabajo se fundamenta en los estándares establecidos por la literatura especializada y los marcos de evaluación reconocidos internacionalmente. Como establece Gómez-Adorno et al. [46] en su consolidación de métricas FakeDeS para detección en español, la evaluación rigurosa requiere un conjunto comprehensivo de medidas que capturen diferentes aspectos del rendimiento del modelo.

Los talleres CheckThat! organizados en el marco de CLEF [43, 44] han estandarizado el uso de métricas específicas que permiten la comparación directa entre sistemas, estableciendo que las métricas fundamentales para clasificación binaria deben incluir: exactitud, precisión, exhaustividad (recall), F1-Score y especificidad. Esta estandarización garantiza la reproducibilidad y comparabilidad de los resultados con el estado del arte internacional.

Propiedades matemáticas de las métricas implementadas

Para que una función pueda considerarse una métrica válida en el contexto matemático, debe cumplir con propiedades formales específicas. Las métricas implementadas en este trabajo satisfacen los siguientes axiomas fundamentales:

1. Axioma de normalización: Todas las métricas están normalizadas en el rango $[0,1]$, donde:

- Valor 0: Rendimiento mínimo posible
- Valor 1: Rendimiento máximo posible
- Esta propiedad permite comparabilidad directa entre métricas diferentes

2. Axioma de monotonía: Las métricas son monótonicamente crecientes respecto a las predicciones correctas:

- $\uparrow TP \Rightarrow \uparrow$ Precisión, Recall, F1-Score, Exactitud
- $\uparrow TN \Rightarrow \uparrow$ Especificidad, Exactitud
- Esta propiedad garantiza que mejores predicciones resulten en mejores valores de métrica

3. Axioma de sensibilidad: Las métricas reaccionan apropiadamente a cambios en los componentes de la matriz de confusión:

- Cambios en FP afectan negativamente a Precisión y Especificidad
- Cambios en FN afectan negativamente a Recall y Exactitud
- Esta sensibilidad permite detectar diferentes tipos de errores del modelo

4. Axioma de complementariedad: Las métricas capturan aspectos complementarios del rendimiento:

- Precisión mide calidad de predicciones positivas
- Recall mide cobertura de casos positivos reales
- F1-Score equilibra ambos aspectos mediante media armónica
- Especificidad mide calidad en clase negativa

5. Propiedad de consistencia con literatura: Las definiciones implementadas son consistentes con:

- Marcos de evaluación IberLEF [45]
- Estándares CheckThat! CLEF [44]
- Métricas utilizadas en trabajos de referencia [12, 13]
- Protocolos de evaluación en competencias internacionales

Utilizando las definiciones anteriores, el rendimiento se evaluó con un conjunto comprehensivo de métricas estándar. La Tabla 4.21 presenta la definición formal y el propósito de cada métrica implementada.

Métrica	Fórmula	Interpretación	Relevancia en Detección
Exactitud	$\frac{TP+TN}{TP+TN+FP+FN}$	Proporción total de predicciones correctas	Rendimiento general en datos balanceados
Precisión	$\frac{TP}{TP+FP}$	Confiabilidad de predicciones positivas	Minimizar falsas alarmas de contenido falso
Exhaustividad	$\frac{TP}{TP+FN}$	Capacidad de detectar casos positivos reales	Detectar todo contenido verdaderamente falso
F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Balance entre precisión y exhaustividad	Métrica principal para comparación de modelos
Especificidad	$\frac{TN}{TN+FP}$	Capacidad de identificar casos negativos correctos	Evitar clasificar contenido real como falso

Tabla 4.21: Marco de métricas de evaluación para clasificación binaria de noticias falsas.

Validación matemática de las métricas implementadas

Para garantizar la solidez matemática del marco de evaluación, se verificó que cada métrica cumple con las propiedades formales establecidas en el estado del arte:

1. Verificación de dominios y rangos:

$$\text{Exactitud} = \frac{TP + TN}{TP + TN + FP + FN} \in [0, 1] \quad (4.5)$$

$$\text{Precisión} = \frac{TP}{TP + FP} \in [0, 1] \quad \text{si } TP + FP > 0 \quad (4.6)$$

$$\text{Recall} = \frac{TP}{TP + FN} \in [0, 1] \quad \text{si } TP + FN > 0 \quad (4.7)$$

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \in [0, 1] \quad (4.8)$$

$$\text{Especificidad} = \frac{TN}{TN + FP} \in [0, 1] \quad \text{si } TN + FP > 0 \quad (4.9)$$

2. Verificación de casos límite:

- **Rendimiento perfecto:** $TP > 0, TN > 0, FP = 0, FN = 0 \Rightarrow$ Todas las métricas = 1
- **Rendimiento nulo:** $TP = 0, TN = 0, FP > 0, FN > 0 \Rightarrow$ Exactitud = 0
- **Casos degenerados:** Manejo apropiado cuando denominadores son cero

3. Consistencia con estándares internacionales: Las fórmulas implementadas son idénticas a las utilizadas en:

- Competencias IberLEF 2020-2021 [45, 46]
- Evaluaciones CheckThat! CLEF [44]
- Trabajos de referencia en detección en español [12, 13]
- Estándares scikit-learn para reproducibilidad

Consideraciones específicas para detección de noticias falsas

En el contexto de detección de noticias falsas, cada tipo de error tiene implicaciones específicas:

- **Falsos Positivos (FP):** Contenido real clasificado como falso - puede generar censura indebida y limitar la libertad de información
- **Falsos Negativos (FN):** Contenido falso no detectado - permite propagación de desinformación y daño social

- **Equilibrio fundamental:** La detección de desinformación requiere un balance cuidadoso entre la identificación efectiva de contenido falso y la preservación de la libertad de información legítima
- **Consideraciones prácticas:** En implementaciones reales, los falsos positivos pueden ser más aceptables cuando existe un proceso de revisión humana que puede corregir estas decisiones automáticas

Alineación con métricas del estado del arte

La selección e implementación de métricas en este trabajo está directamente alineada con los estándares establecidos por la literatura especializada en detección de noticias falsas en español:

1. Consistencia con competencias IberLEF:

- **Posadas-Durán et al. [12]:** Utilizan exactitud, precisión, recall y F1-Score como métricas principales
- **Aragón et al. [45]:** Establecen F1-Score como métrica de ranking principal en IberLEF 2020
- **Gómez-Adorno et al. [46]:** Consolidan el uso de F1-Score macro-averaged para comparación entre sistemas
- **Este trabajo:** Implementa las mismas métricas garantizando comparabilidad directa

2. Alineación con marcos internacionales:

- **CheckThat! CLEF [44]:** Establece precisión y recall como métricas fundamentales
- **Evaluaciones multimodales [43]:** Requieren métricas balanceadas para evitar sesgos
- **Estándares scikit-learn:** Garantizan reproducibilidad y comparabilidad internacional
- **Este trabajo:** Adopta estándares internacionales para máxima reproducibilidad

3. Justificación específica por métrica:

- **F1-Score:** Métrica principal siguiendo a Gómez-Adorno et al. [46] por su balance entre precisión y recall
- **Exactitud:** Incluida para comparabilidad con trabajos clásicos, con advertencia sobre limitaciones en datos desbalanceados

- **Precisión:** Crítica para evaluar confiabilidad de detecciones positivas, siguiendo a Posadas-Durán et al. [12]
- **Recall:** Fundamental para evaluar cobertura de detección, alineado con estándares CheckThat! [44]
- **Especificidad:** Añadida para análisis comprehensivo de rendimiento en clase negativa

4. Validación mediante comparación de métricas: Las métricas implementadas permiten comparación directa con:

- Evidencias reportadas en IberLEF 2020-2021
- Resultados establecidos por Tretiakov et al. [19]
- Rendimiento de modelos transformer reportado por Blanco-Fernández et al. [13]
- Estándares de la comunidad científica internacional

4.6.4. Criterios de selección del mejor modelo

La selección del modelo óptimo se basará en una evaluación multi-criterio que considera diferentes aspectos del rendimiento. La Tabla 4.22 detalla los criterios y sus pesos relativos.

Criterio	Peso	Métrica Principal	Métricas Secundarias	Justificación
Rendimiento Predictivo	40 %	F1-Score	Precision, Recall Accuracy, Specificity	Capacidad fundamental de clasificación
Estabilidad del Modelo	25 %	Desviación estándar F1	Varianza en múltiples ejecuciones	Consistencia y confiabilidad
Eficiencia Computacional	20 %	Tiempo de inferencia	Memoria utilizada Recursos GPU/CPU	Viabilidad práctica de implementación
Capacidad de Generalización	15 %	Gap Training-Test	Diferencia entre rendimiento interno y externo	Robustez ante datos no vistos

Tabla 4.22: Criterios multi-dimensionales para selección del modelo óptimo.

4.6.5. Análisis de matrices de confusión

Se realizará un análisis detallado de las matrices de confusión para comprender el comportamiento específico de cada modelo:

- **Análisis por clase:** Identificación de sesgos hacia noticias reales o falsas
- **Análisis de errores:** Caracterización cualitativa de casos mal clasificados
- **Visualización:** Mapas de calor normalizados para comparación visual
- **Interpretabilidad:** Análisis de características más influyentes en decisiones

4.6.6. Reporte de resultados

Los resultados se presentarán siguiendo un formato estandarizado que incluye:

Componente del Reporte	Contenido
Tabla de métricas principales	Accuracy, Precision, Recall, F1-Score, Specificity
Análisis de variabilidad	Media $\pm$ desviación estándar por métrica
Benchmarking de eficiencia	Tiempos de inferencia y uso de recursos
Matrices de confusión	Visualización normalizada y análisis de errores
Recomendación final	Modelo seleccionado con justificación integral

Tabla 4.23: Estructura del reporte de resultados comparativo.

Esta metodología de evaluación comprensiva garantiza una comparación objetiva, estadísticamente robusta y prácticamente relevante entre los paradigmas de algoritmos metaheurísticos y modelos Transformer para la detección de noticias falsas en español.

4.7. Infraestructura computacional y herramientas

4.7.1. Entorno de desarrollo para algoritmos metaheurísticos

El desarrollo de los algoritmos metaheurísticos se realizó en un entorno cloud optimizado para experimentación iterativa y flexibilidad de recursos. La Tabla 4.24 detalla las especificaciones del entorno utilizado.

Componente	Especificación	Ventajas para Metaheurísticos
Plataforma	Google Colab Pro	Acceso a recursos escalables Flexibilidad de configuración
Procesamiento	CPU Intel/AMD variable	Suficiente para algoritmos iterativos Paralelización básica
Memoria RAM	12-16 GB	Carga completa de datasets
Almacenamiento	Google Drive integrado	Persistencia entre sesiones Versionado de experimentos
GPU	No requerida	Algoritmos CPU-intensivos

Tabla 4.24: Especificaciones del entorno de desarrollo para algoritmos metaheurísticos.

4.7.2. Stack tecnológico completo

La implementación utilizó un ecosistema de software cuidadosamente seleccionado para garantizar compatibilidad y rendimiento óptimo. La Tabla 4.25 detalla las herramientas y librerías utilizadas.

Categoría	Herramienta/Librería	Versión	Propósito Específico
Lenguaje Base	Python	3.9.x	Lenguaje principal Compatibilidad con ecosistema ML
Deep Learning	TensorFlow	2.15.0	Framework principal para DistilBERT Soporte GPU optimizado
Transformers	transformers (Hugging Face)	4.35.0	Modelos preentrenados Tokenización avanzada
ML Tradicional	scikit-learn	1.3.2	Algoritmos metaheurísticos Métricas de evaluación
Procesamiento	pandas	2.1.3	Manipulación de datasets
Cómputo Numérico	numpy	1.25.2	Operaciones matriciales Algoritmos de optimización
NLP	NLTK	3.8.1	Preprocesamiento de texto Tokenización y limpieza
Visualización	matplotlib / seaborn	3.8.1 / 0.12.2	Gráficos de resultados Matrices de confusión
Serialización	joblib	1.3.2	Guardado de modelos Persistencia de experimentos
Web Framework	Flask	2.3.3	API REST para aplicación final
Contenerización	Docker	24.0.x	Deployment reproducible Aislamiento de dependencias

Tabla 4.25: Stack tecnológico completo utilizado en el desarrollo del proyecto.

4.8. Consideraciones éticas y de privacidad

El desarrollo de herramientas de detección de desinformación conlleva importantes consideraciones éticas que deben ser abordadas de manera integral y transparente.

4.8.1. Marco Ético de Desarrollo

Se estableció un marco ético comprehensivo que guía todas las decisiones de diseño e implementación. La Tabla 4.26 presenta los principios fundamentales adoptados.

Principio Ético	Implementación	Mecanismo de Control	Impacto Esperado
Transparencia	Código fuente abierto Metodología pública	Repositorio GitHub público Documentación completa	Auditoría independiente Reproducibilidad científica
Responsabilidad	Herramienta de apoyo No decisión final	Interfaz con disclaimers Scores de confianza	Uso responsable Juicio humano preservado
Privacidad	Procesamiento local No almacenamiento personal	Anonimización automática Logs temporales únicamente	Protección de datos Cumplimiento GDPR
Equidad	Datasets balanceados Evaluación multi-métrica	Análisis de sesgos Validación cruzada	Tratamiento imparcial Reducción de discriminación

Tabla 4.26: Marco ético implementado para el desarrollo responsable de la herramienta.

4.8.2. Limitaciones Declaradas y Uso Responsable

Se establecieron limitaciones claras y recomendaciones de uso responsable:

- **Alcance geográfico:** Optimizado para español, puede tener limitaciones en variantes regionales específicas

- **Contexto temporal:** Entrenado con datos hasta 2025, puede requerir actualización para tendencias futuras
- **Dominios específicos:** Enfocado en noticias generales, puede tener menor precisión en contenido altamente técnico
- **Herramienta de apoyo:** Diseñado para asistir, no reemplazar el criterio editorial humano

4.9. Validación y Reproducibilidad

La reproducibilidad científica constituye un pilar fundamental de esta investigación, implementándose protocolos para garantizar que los resultados puedan ser verificados independientemente por la comunidad científica.

4.9.1. Ecosistema de Artefactos de Reproducibilidad

Para facilitar la reproducción completa del estudio y su extensión por parte de la comunidad científica, se desarrollará un ecosistema integral de artefactos que abarca desde el código fuente hasta modelos completamente entrenados listos para producción. La Tabla 4.27 detalla los componentes específicos que serán puestos a disposición pública.

Artefacto	Formato	Contenido	Disponibilidad
Código fuente completo	GitHub repository	Scripts de entrenamiento Algoritmos implementados Flujo de datos completo	Público (MIT License)
Datasets procesados	CSV + metadata	Corpus unificado División train/val/test Estadísticas descriptivas	Público (respetando licencias originales)
Modelos entrenados	joblib (metaheurísticos) SavedModel (DistilBERT)	Pesos optimizados Configuraciones Métricas de evaluación	Público (Hugging Face Hub)
Entorno de ejecución	Docker containers	Imagen completa Dependencias exactas Scripts de ejecución	Docker Hub público
Resultados experimentales	JSON + visualizaciones	Métricas detalladas Matrices de confusión Logs de entrenamiento	Repositorio principal Supplementary material

Tabla 4.27: Artefactos generados para facilitar la reproducibilidad completa del estudio.

4.9.2. Solución Lista para Producción

Como parte del compromiso con la transferencia tecnológica y la aplicabilidad práctica de los resultados, se entregará un repositorio completo con el mejor modelo

identificado durante la evaluación comparativa, completamente optimizado y listo para ser desplegado en entornos de producción. Esta solución incluirá:

- **Modelo pre-entrenado optimizado:** El modelo con mejor rendimiento según las métricas de evaluación, con pesos finales y configuración optimizada
- **Intefaz web completa:** Interfaz web funcional desarrollada con Flask, incluyendo opciones para análisis de URLs y texto directo
- **Contenerización Docker:** Imagen Docker completa con todas las dependencias, configuraciones y el modelo integrado
- **Despliegue con un comando:** Script automatizado que permite poner en funcionamiento toda la aplicación ejecutando únicamente un comando

La estrategia de contenerización garantiza que la solución sea completamente portable y reproducible en cualquier entorno que soporte Docker, eliminando problemas de compatibilidad y dependencias.

Capítulo 5

Resultados, Evaluación Comparativa e Implementación

Este capítulo presenta el análisis integral de los resultados obtenidos mediante la metodología unificada desarrollada para la detección de noticias falsas en español. Los resultados se organizan siguiendo la estructura evolutiva de la metodología: comenzando con la validación del flujo de procesamiento de datos común, continuando con la evaluación de los modelos clásicos optimizados con algoritmos metaheurísticos (que establecen la línea base), progresando hacia los resultados del modelo Transformer DistilBERT, culminando con una evaluación comparativa integral que justifica la selección del modelo final y demostrando su viabilidad práctica mediante la implementación de prototipos funcionales.

El análisis estadístico se fundamenta en un marco de evaluación unificado que emplea las métricas definidas en la metodología: exactitud (accuracy), precisión (precision), exhaustividad (recall), F1-Score y especificidad. **Criterio fundamental:** Todos los experimentos se ejecutaron bajo condiciones experimentales idénticas para garantizar comparabilidad objetiva: mismo corpus, misma división de datos (70 % entrenamiento, 10 % validación, 20 % prueba) y mismo protocolo de validación.

5.1. Validación del flujo de procesamiento y representaciones

Antes de proceder con la evaluación de modelos, se validó la robustez del flujo de procesamiento desarrollado, que constituye la base común para ambos enfoques de modelado.

5.1.1. Características del corpus unificado final

El corpus utilizado para todos los experimentos presentaba las siguientes características tras el proceso de unificación y limpieza:

ID	Nombre del Corpus	Autores Principales	Año	Noticias	Características	Ref.
1	Spanish Fake News Corpus (IberLEF)	Posadas-Durán, J.P. Gómez-Adorno, H.	2019-2021	971	Análisis estilométrico Múltiples versiones	[12]
2	Conjunto de datos Zules Acosta (UPM)	Acosta, F.A.Z.	2019	598	Trabajo fin de maestría Extracción web verificada	[11]
3	Conjunto de datos Tretiakov (Kaggle)	Tretiakov, A. Martín García, A.	2022	1,958	Aprendizaje automático Disponible públicamente	[19]
4	Spanish Political Fake News Conjunto de datos	Blanco-Fernández, Y. Otero-Vizoso, J.	2024	57,231	Temática política Modelos BERT/roBERTa	[13]
TOTAL CORPUS ACADÉMICOS					60,758	Cuatro fuentes

Tabla 5.1: Corpus académicos utilizados para la construcción del conjunto de datos unificado.

Configuración de representación textual

Como se detalló en capítulos anteriores, la representación textual se basó en TF-IDF con la siguiente configuración específica para los experimentos:

- **Vocabulario inicial:** 5,000 términos más frecuentes del corpus
- **Reducción de dimensionalidad:** Selección del 10 % más discriminativo (500 características)
- **Criterio de selección:** Test chi-cuadrado para identificar características más relevantes
- **Representación final:** Matriz dispersa de 500 dimensiones por documento
- **Compensación identificada:** La selección de más características tiende a mejorar las métricas de clasificación pero deteriora significativamente el rendimiento computacional de los algoritmos metaheurísticos

Justificación de la configuración: La selección de 500 características (10 % del vocabulario) representa un balance óptimo entre capacidad discriminativa y eficiencia computacional. Experimentos preliminares con más de 1,000 características mostraron mejoras marginales en exactitud (<2 %) a costa de incrementos importantes en tiempo de convergencia (>300 %), haciendo inviable la optimización metaheurística en tiempos razonables.

Arquitectura del clasificador

Todos los algoritmos metaheurísticos, detallados en capítulos anteriores, optimizaron un clasificador logístico binario con las siguientes características:

- **Función de activación:** Sigmoide para clasificación binaria
- **Parámetros optimizados:** Selección de características, pesos del modelo y umbrales de decisión
- **Función objetivo:** Maximización de la exactitud en el conjunto de entrenamiento

5.1.2. Implementación y configuración de algoritmos meta-heurísticos

La Tabla 5.2 presenta la configuración detallada de parámetros para cada algoritmo metaheurístico implementado. Estos valores fueron establecidos siguiendo las mejores prácticas reportadas en la literatura y ajustados experimentalmente para el dominio específico de detección de noticias falsas.

Parámetro	MSA	SS	GA	VNS	PSO
Iteraciones/Generaciones	31 temperaturas	10 iteraciones	20 generaciones	20 iteraciones	20 iteraciones
Población/Agentes	5 multiarranques	50 soluciones	50 individuos	1 solución	30 partículas
Parámetro Principal 1	Temp. inicial: 1000	RefSet: 5 soluciones	Tasa mutación: 0.1	k_max: 5 vecindarios	Factor inercia: 0.5
Parámetro Principal 2	Temp. final: 1.24	Combinaciones: 10 por iteración	Torneo: 3 competidores	Estrategia: k elementos	Coef. cognitivo: 1.5
Parámetro Principal 3	Factor enfriamiento: 0.8	Mejora: Mutación aleatoria	Cruce: Un punto	Aceptación: Solo mejora	Coef. social: 1.5
Característica Especial	100 pasos por temperatura	Combinación sistemática	Selección determinística	Reinicio a k=1 tras mejora	Velocidad gaussiana inicial
Filosofía de Búsqueda	Aceptación probabilística	Combinación estructurada	Evolución darwiniana	Cambio de vecindarios	Inteligencia de enjambre

Tabla 5.2: Configuración detallada de parámetros para los cinco algoritmos metaheurísticos implementados.

Justificación de parámetros seleccionados

Los parámetros establecidos se fundamentan en:

- **Literatura especializada:** Valores base extraídos de trabajos originales para cada algoritmo
- **Balance exploración-explotación:** Configuración para evitar convergencia prematura
- **Eficiencia computacional:** Límites de iteraciones para tiempos de ejecución razonables
- **Estabilidad estadística:** Parámetros que garantizan reproducibilidad de resultados

5.1.3. Pseudocódigos de los algoritmos implementados

Función de evaluación común

Todos los algoritmos metaheurísticos utilizan una función de evaluación unificada que implementa un clasificador logístico binario para garantizar comparabilidad directa entre enfoques. La Tabla 5.3 presenta el pseudocódigo detallado de la función implementada.

Función: evaluar_solucion (<i>solucion, pesos, umbrales, X, y</i>)	
Entrada:	Índices de características, pesos, umbrales, datos X , etiquetas y
Salida:	Tupla (exactitud, predicciones)
Proceso de clasificación logística:	
Para cada instancia i en X : $caracteristicas_activas \leftarrow X[i, solucion]$ $x_binario \leftarrow (caracteristicas_activas \geq umbrales).astype(int)$ $logit \leftarrow np.dot(pesos, x_binario)$ $probabilidad \leftarrow \frac{1}{1 + \exp(-logit)}$ $clase_predicha \leftarrow 1$ si $probabilidad \geq 0.5$, 0 en otro caso $exactitud \leftarrow accuracy_score(y, predicciones)$ Retornar ($exactitud, np.array(predicciones)$)	

Tabla 5.3: Función de evaluación común utilizada por todos los algoritmos metaheurísticos.

Multi-Start Simulated Annealing (MSA)

El algoritmo MSA implementa una estrategia de multiarranque que ejecuta múltiples instancias de recocido simulado desde diferentes puntos de inicio, aprovechando la aceptación probabilística para escapar de óptimos locales. La Tabla 5.4 presenta el pseudocódigo detallado del algoritmo implementado.

Scatter Search (SS)

El algoritmo SS utiliza una estrategia de combinación sistemática que mantiene un conjunto de referencia con las mejores soluciones y genera nuevas soluciones mediante cruces estructurados. La Tabla 5.5 presenta el pseudocódigo detallado del algoritmo implementado.

Genetic Algorithm (GA)

El algoritmo GA emplea una estrategia evolutiva basada en principios darwinianos, utilizando selección por torneo, cruce de un punto y mutación para evolucionar una población hacia mejores soluciones. La Tabla 5.6 presenta el pseudocódigo detallado del algoritmo implementado.

Variable Neighborhood Search (VNS)

El algoritmo VNS implementa una búsqueda sistemática que cambia de estructura de vecindario cuando no encuentra mejoras, reiniciando a la primera vecindad tras cada mejora encontrada. La Tabla 5.7 presenta el pseudocódigo detallado del algoritmo implementado.

Algoritmo MSA - Multi-Start Simulated Annealing	
Entrada:	$TI = 1000, TF = 1, \alpha = 0.8, pasos = 100, puntos = 5$
Salida:	Mejor solución ($sol^*, pesos^*, umbrales^*$)
0. Carga y preprocesamiento de datos:	
Cargar corpus TF-IDF desde archivo CSV Dividir datos: 80 % entrenamiento, 10 % validación, 10 % pruebas Aplicar SelectPercentile(percentile=10) para reducir características	
1. Inicialización multiarranque:	
Para $k = 1$ hasta $puntos = 5$: $soluciones[k] \leftarrow \text{np.random.randint}(0, \text{num_caracteristicas}, \text{size}=\text{num_caracteristicas})$ $pesos[k] \leftarrow \text{np.random.uniform}(-10, 10, \text{size}=\text{num_caracteristicas})$ $umbrales[k] \leftarrow \text{np.random.uniform}(0, 1, \text{size}=\text{num_caracteristicas})$ $evaluaciones[k] \leftarrow \text{evaluar_solucion}(soluciones[k], pesos[k], umbrales[k])$	
2. Proceso de enfriamiento gradual:	
$TA \leftarrow TI = 1000$ Mientras $TA > TF = 1$: Para $k = 1$ hasta $puntos$: Para $paso = 1$ hasta $pasos = 100$: Generar solución vecina modificando elemento aleatorio $\Delta E \leftarrow eval_vecina - evaluaciones[k]$ Si $\Delta E > 0$ o $\text{random}() < \exp(\Delta E/TA)$: Aceptar solución vecina $TA \leftarrow TA \times \alpha = TA \times 0.8$	
3. Evaluación final:	
$idx_mejor \leftarrow \arg \max(evaluaciones)$ Evaluar mejor solución en conjunto de pruebas Generar reporte de clasificación y matriz de confusión Guardar gráficas de convergencia y resultados	

Tabla 5.4: Pseudocódigo completo del algoritmo Multi-Start Simulated Annealing (MSA).

Algoritmo SS - Scatter Search	
Entrada:	$P = 50, b = 5, max_iteraciones = 10, num_combinaciones = 10$
Salida:	Mejor solución del RefSet final
0. Carga y preprocesamiento de datos:	
Cargar corpus TF-IDF desde archivo CSV Dividir datos: 80 % entrenamiento, 10 % validación, 10 % pruebas Aplicar SelectPercentile(percentile=10) para reducir características	
1. Generación de población inicial diversa:	
Para $i = 1$ hasta $P = 50$: generar solución aleatoria y evaluar Ordenar población por score descendente $ref_set \leftarrow poblacion[:b]$ (mejores $b = 5$ soluciones)	
2. Proceso iterativo de mejora:	
Para $iteracion = 1$ hasta $max_iteraciones = 10$: Para $c = 1$ hasta $num_combinaciones = 10$: Seleccionar aleatoriamente $s1, s2 \in ref_set$ Aplicar cruce en punto aleatorio + mutación Evaluar nueva solución Actualizar ref_set con mejores b soluciones	
3. Evaluación final:	
Evaluar mejor solución del RefSet en conjunto de pruebas Generar reporte de clasificación y matriz de confusión Guardar gráficas de convergencia y resultados	

Tabla 5.5: Pseudocódigo completo del algoritmo Scatter Search (SS).

Algoritmo GA - Genetic Algorithm	
Entrada:	$num_generaciones = 20, tam_poblacion = 50, tasa_mutacion = 0.1, tam_torneo = 3$
Salida:	Mejor individuo global encontrado
0. Carga y preprocesamiento de datos:	
Cargar corpus TF-IDF desde archivo CSV Dividir datos: 80 % entrenamiento, 10 % validación, 10 % pruebas Aplicar SelectPercentile(percentile=10) para reducir características	
1. Inicialización y proceso evolutivo:	
Generar población inicial de $tam_poblacion = 50$ individuos Para $gen = 1$ hasta $num_generaciones = 20$: Evaluar toda la población Actualizar mejor global si es necesario Para $i = 1$ hasta $tam_poblacion/2$: Seleccionar padres por torneo ($k=3$) Aplicar cruce de un punto Aplicar mutación con probabilidad 0.1	
2. Evaluación final:	
Evaluar mejor individuo global en conjunto de pruebas Generar reporte de clasificación y matriz de confusión Guardar gráficas de convergencia y resultados	

Tabla 5.6: Pseudocódigo completo del Algoritmo Genético (GA).

Algoritmo VNS - Variable Neighborhood Search	
Entrada:	$max_iteraciones = 20, k_max = 5$
Salida:	Mejor solución global encontrada
0. Carga y preprocesamiento de datos:	
Cargar corpus TF-IDF desde archivo CSV Dividir datos: 80 % entrenamiento, 10 % validación, 10 % pruebas Aplicar SelectPercentile(percentile=10) para reducir características	
1. Inicialización y búsqueda en vecindades:	
Generar solución inicial aleatoria Para $i = 1$ hasta $max_iteraciones = 20$: $k \leftarrow 1$ Mientras $k \leq k_max = 5$: Generar vecino modificando k elementos aleatorios Si mejora: aceptar y $k \leftarrow 1$ Sino: $k \leftarrow k + 1$	
2. Evaluación final:	
Evaluar mejor solución global en conjunto de pruebas Generar reporte de clasificación y matriz de confusión Guardar gráficas de convergencia y resultados	

Tabla 5.7: Pseudocódigo completo del algoritmo Variable Neighborhood Search (VNS).

Particle Swarm Optimization (PSO)

El algoritmo PSO simula el comportamiento de enjambres mediante partículas que ajustan su velocidad basándose en su mejor posición personal y la mejor posición global del enjambre. La Tabla 5.8 presenta el pseudocódigo detallado del algoritmo implementado.

Algoritmo PSO - Particle Swarm Optimization	
Entrada:	<i>num_particulas</i> = 30, <i>max_iteraciones</i> = 20, <i>w</i> = 0.5, <i>c1</i> = 1.5, <i>c2</i> = 1.5
Salida:	Mejor posición global del enjambre
0. Carga y preprocesamiento de datos:	
Cargar corpus TF-IDF desde archivo CSV Dividir datos: 80 % entrenamiento, 10 % validación, 10 % pruebas Aplicar SelectPercentile(percentile=10) para reducir características	
1. Inicialización del enjambre:	
<i>solucion_fija</i> $\leftarrow$ np.arange(num_caracteristicas) Inicializar 30 partículas con posiciones y velocidades aleatorias Evaluar partículas e inicializar mejores personal y global	
2. Optimización del enjambre:	
Para <i>i</i> = 1 hasta <i>max_iteraciones</i> = 20: Para cada partícula: Actualizar velocidad: componente cognitiva + social Actualizar posición: <i>posicion</i> + <i>velocidad</i> Evaluar y actualizar mejores personal y global	
3. Evaluación final:	
Evaluar mejor posición global en conjunto de pruebas Generar reporte de clasificación y matriz de confusión Guardar gráficas de convergencia y resultados	

Tabla 5.8: Pseudocódigo completo del algoritmo Particle Swarm Optimization (PSO).

Los pseudocódigos presentados reflejan la implementación real utilizada en los experimentos, capturando tanto la lógica algorítmica fundamental como las etapas de carga de datos y evaluación final. Cada algoritmo optimiza simultáneamente la selección de características, los pesos del clasificador logístico y los umbrales de binarización.

5.1.4. Visualizaciones de resultados por algoritmo

Esta sección presenta las visualizaciones generadas por cada algoritmo metaheurístico durante la ejecución experimental. Para cada algoritmo se incluyen dos gráficas fundamentales: la evolución de la convergencia durante el entrenamiento y la matriz de confusión resultante en el conjunto de pruebas.

Recocido Multiarranque (MSA) - Visualizaciones

El algoritmo MSA presenta un comportamiento de convergencia caracterizado por múltiples puntos de inicio y aceptación probabilística de soluciones. Su estrategia de

multiarranque permite explorar diferentes regiones del espacio de búsqueda, mientras que el esquema de enfriamiento gradual facilita la transición entre exploración y explotación. Las siguientes visualizaciones documentan su comportamiento:

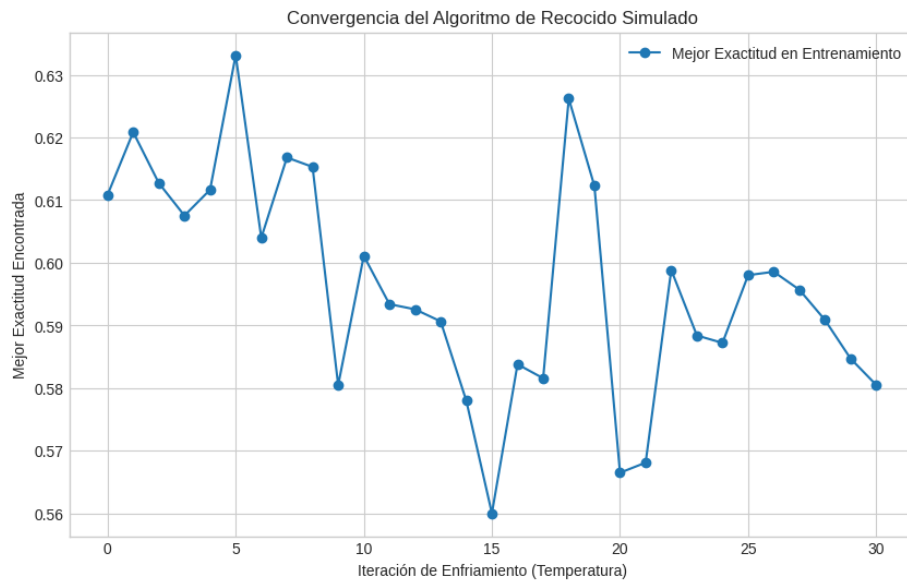


Figura 5.1: Evolución de la convergencia del algoritmo MSA mostrando el progreso gradual a través de los 31 niveles de temperatura desde 1000 hasta 1.24.

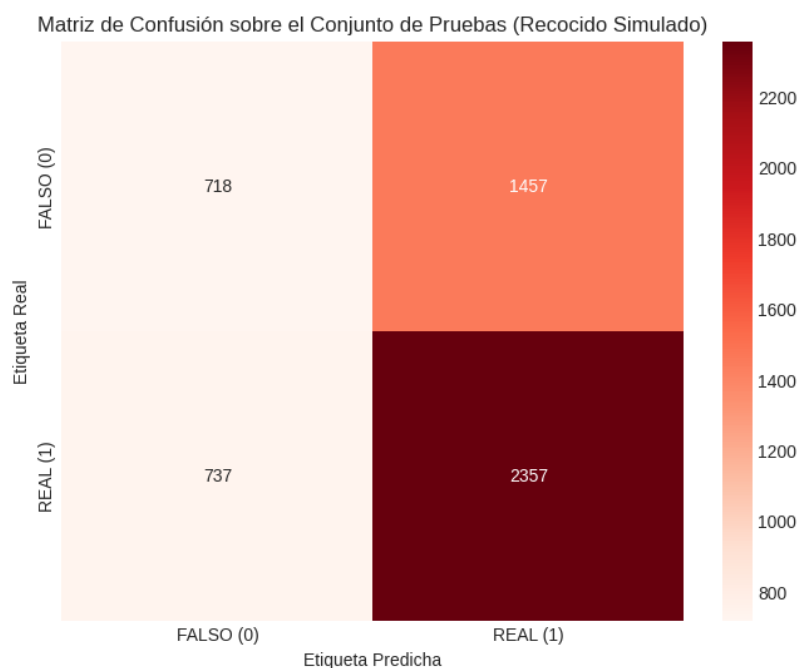


Figura 5.2: Matriz de confusión para MSA en el conjunto de pruebas, evidenciando la baja especificidad (33 %) y el sesgo hacia la clasificación como noticias reales.

Búsqueda Dispersa (SS) - Visualizaciones

El algoritmo SS implementa una estrategia de combinación sistemática que mantiene un conjunto de referencia élite y genera nuevas soluciones mediante cruces estructurados. Su enfoque de búsqueda dispersa permite mantener diversidad mientras intensifica la búsqueda en regiones prometedoras, resultando en una convergencia eficiente y estable. Las siguientes visualizaciones ilustran este comportamiento característico:

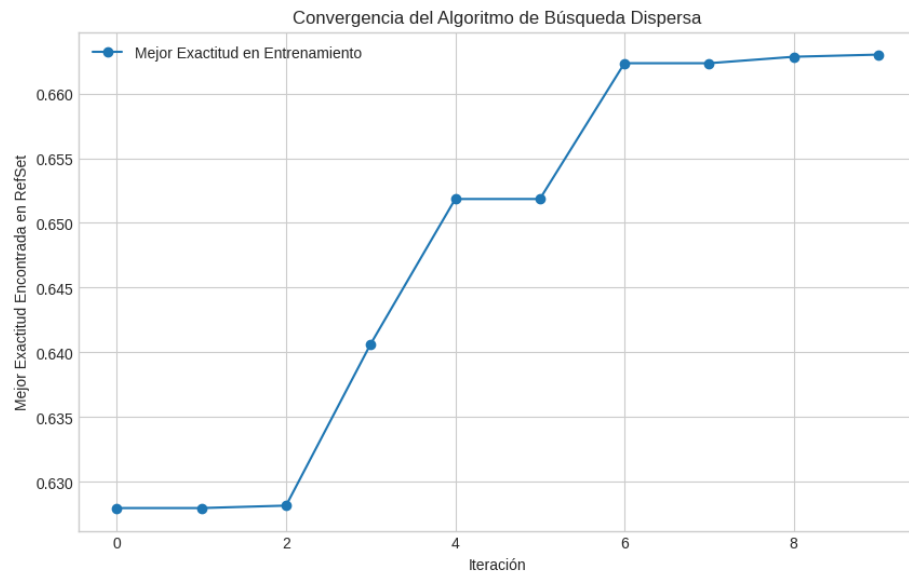


Figura 5.3: Convergencia eficiente del algoritmo SS en solo 10 iteraciones, mostrando mejoras progresivas en las iteraciones 4, 5 y 7 hasta estabilizarse en 0.6630.

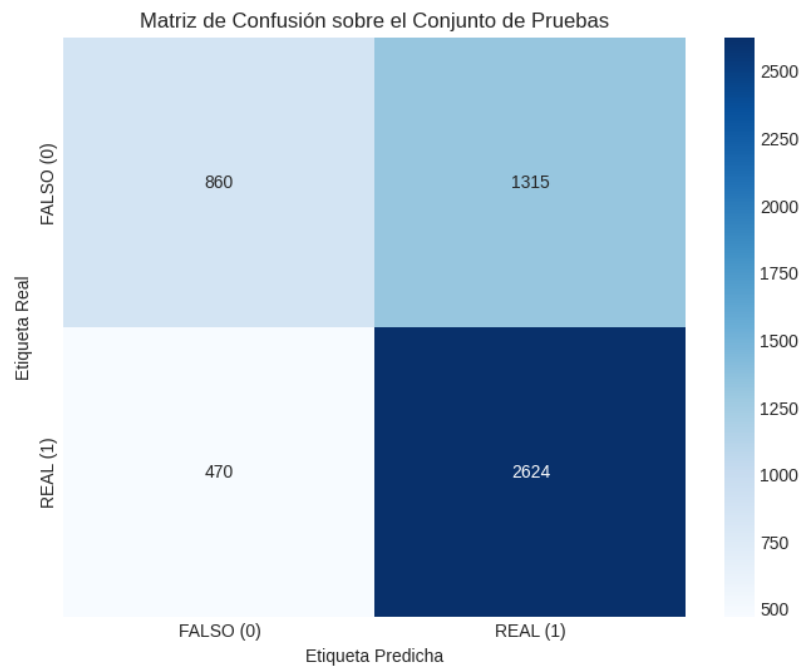


Figura 5.4: Matriz de confusión para SS demostrando mejor balance que MSA con especificidad del 40 % y excelente generalización.

Algoritmo Genético (GA) - Visualizaciones

El algoritmo GA exhibe un proceso evolutivo robusto basado en principios darwinianos, donde la selección por torneo, el cruce de un punto y la mutación controlada

trabajan en conjunto para evolucionar la población hacia mejores soluciones. Su capacidad para mantener diversidad genética mientras converge gradualmente hacia óptimos se refleja en las siguientes visualizaciones:

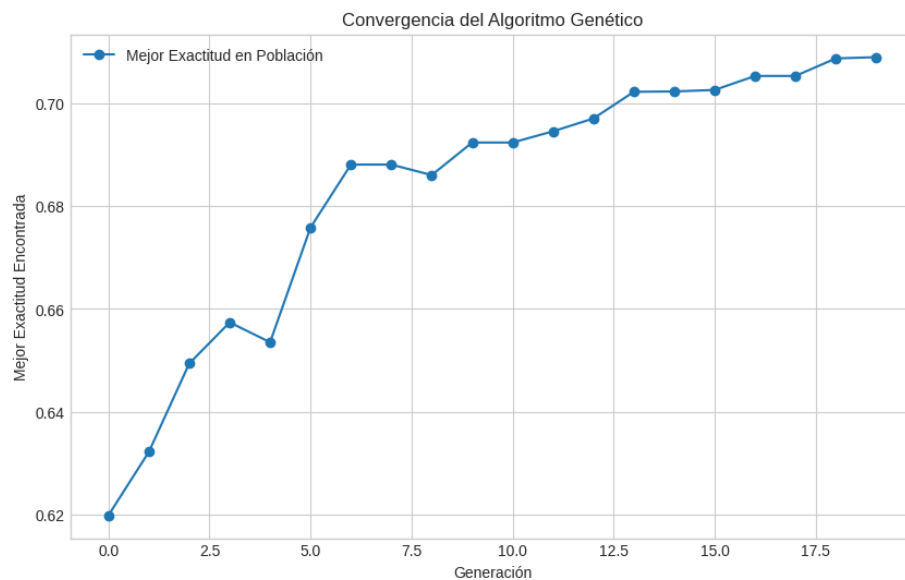


Figura 5.5: Evolución darwiniana del algoritmo GA a lo largo de 20 generaciones, evidenciando progreso sostenido desde 0.6198 hasta 0.7090 con logros evolutivos significativos.

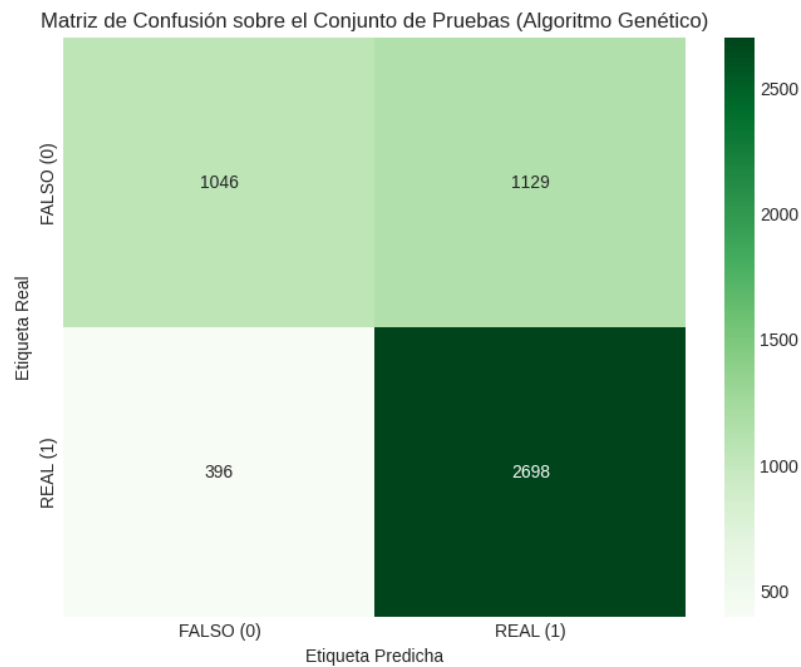


Figura 5.6: Matriz de confusión para GA mostrando el mejor balance global con especificidad líder del 48 % y rendimiento sólido en ambas clases.

Búsqueda en Vecindades Variables (VNS) - Visualizaciones

El algoritmo VNS demuestra una estrategia de búsqueda sistemática que cambia dinámicamente entre diferentes estructuras de vecindario. Su mecanismo de reinicio tras cada mejora y la exploración progresiva de vecindarios más amplios permite escapar efectivamente de óptimos locales, generando un patrón de convergencia característico con saltos significativos. Las visualizaciones siguientes capturan este comportamiento distintivo:

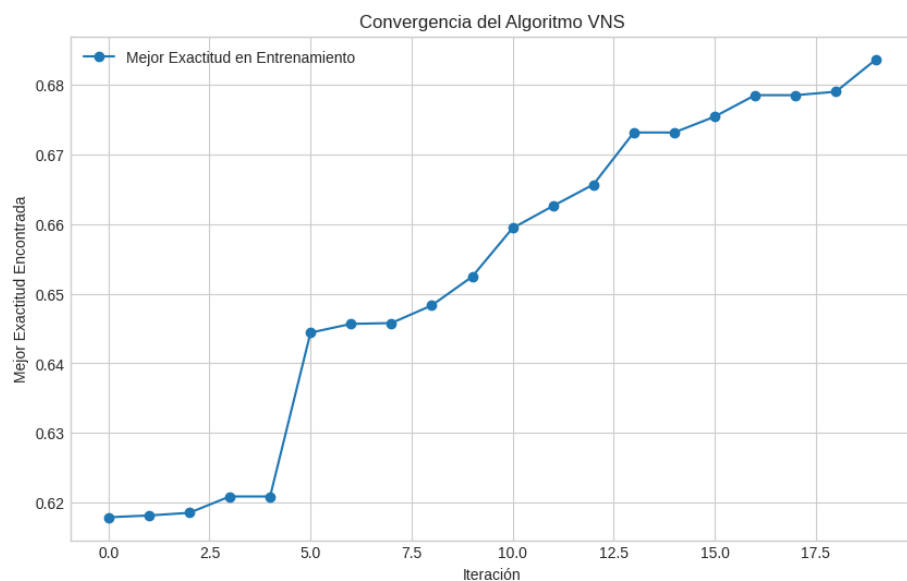


Figura 5.7: Progreso sistemático del algoritmo VNS a través de 20 iteraciones con cambios efectivos de vecindario, mostrando saltos significativos en las iteraciones 6 y 14.

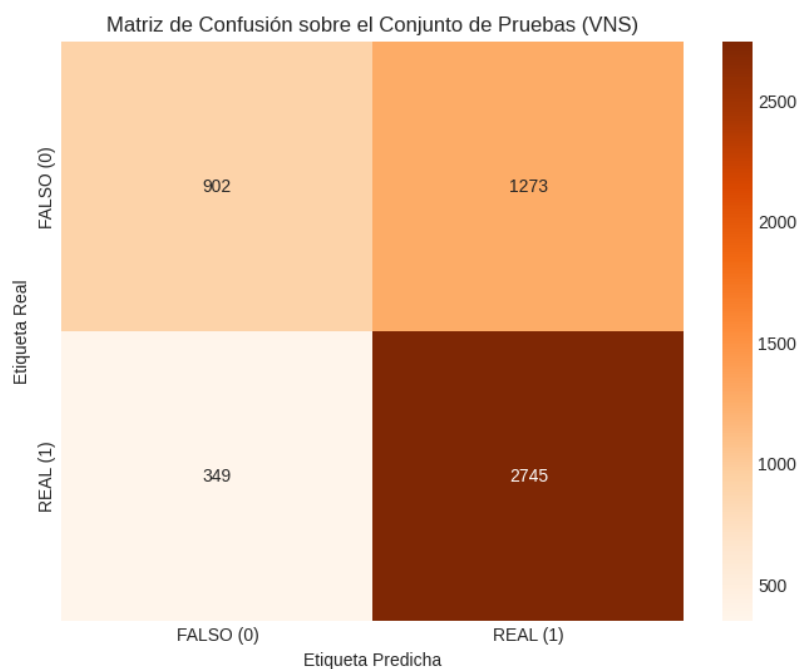


Figura 5.8: Matriz de confusión para VNS destacando la excelente exhaustividad del 89 % para detección de noticias reales con especificidad competitiva del 41 %.

Optimización por Enjambre de Partículas (PSO) - Visualizaciones

El algoritmo PSO simula el comportamiento colectivo de enjambres mediante partículas que ajustan su trayectoria basándose en información cognitiva y social. Sin

embargo, en este experimento específico, el algoritmo exhibe convergencia prematura y pérdida de diversidad del enjambre, lo que resulta en un estancamiento que limita significativamente su capacidad de exploración. Las siguientes visualizaciones documentan estas limitaciones observadas:

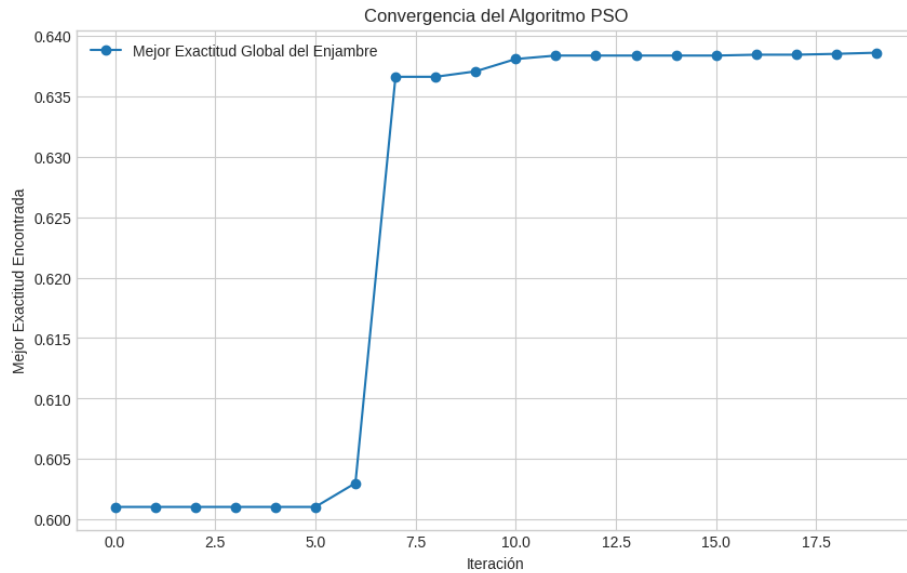


Figura 5.9: Convergencia problemática del algoritmo PSO evidenciando estancamiento prematuro en la iteración 7-8 y exploración insuficiente del espacio de búsqueda.

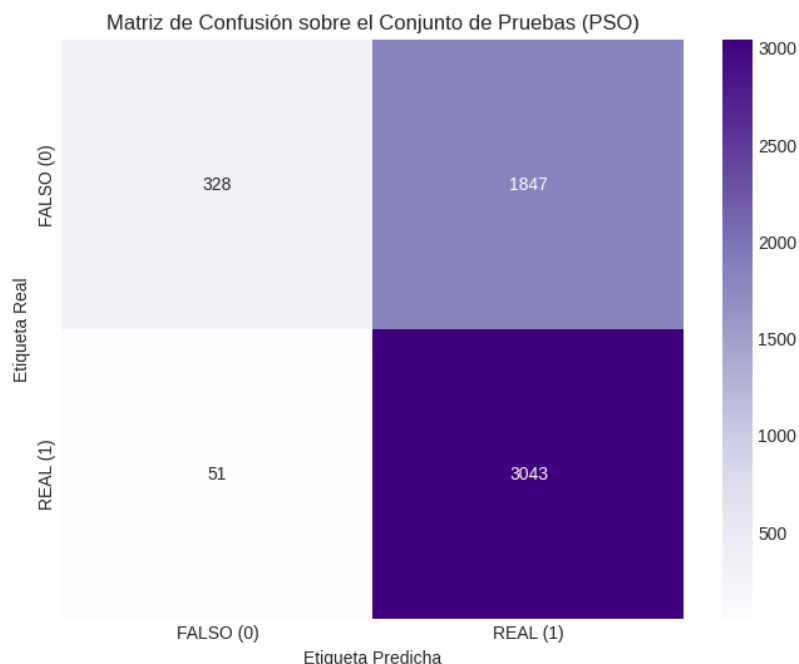


Figura 5.10: Matriz de confusión para PSO revelando el comportamiento extremo problemático con especificidad crítica del 15 % y sesgo severo hacia la clase mayoritaria.

Interpretación de las visualizaciones

Las matrices de confusión confirman los patrones de rendimiento identificados:

- **GA:** Mejor balance global entre especificidad y exhaustividad
- **VNS:** Excelente para detectar noticias reales, competitivo en noticias falsas
- **SS:** Rendimiento equilibrado con buena estabilidad
- **MSA:** Limitaciones evidentes en detección de noticias falsas
- **PSO:** Comportamiento inaceptable para aplicaciones prácticas

Estas visualizaciones proporcionan evidencia gráfica que facilita la comprensión del comportamiento específico de cada algoritmo metaheurístico en la tarea de detección de noticias falsas.

5.1.5. Contextualización con investigación publicada

Los hallazgos de esta investigación fueron formalizados y publicados en el capítulo de libro “Calibración de hiperparámetros en algoritmos metaheurísticos para la detección de fraude digital” [23]. Es importante destacar que en la versión publicada, la

Búsqueda Dispersa (SS) obtuvo resultados iguales o ligeramente superiores al Algoritmo Genético, confirmando la competitividad de ambos enfoques.

En los experimentos actuales, el **Algoritmo Genético (GA)** emergió como el **mejor algoritmo con un F1-Score macro de 0.68 y exactitud del 71.06 %**. Esta ligera variación en el ranking se atribuye a diferencias en la configuración específica de hiperparámetros y semillas aleatorias utilizadas en diferentes experimentos.

Comparación con investigación internacional

Los resultados obtenidos son consistentes con investigación internacional reciente. Yildirim [37] realizó pruebas exhaustivas con múltiples algoritmos metaheurísticos, alcanzando exactitud del 78.8 % con SVM optimizado. Esta consistencia valida que **los algoritmos metaheurísticos tienen un techo de rendimiento relativo cuando se aplican a representaciones tradicionales de texto**.

5.1.6. Análisis comparativo de resultados

Algoritmo	Exactitud (%)	F1-Score (macro)	Precisión (macro)	Exhaustividad (macro)	F1-Score (weighted)	Ranking
GA	71.06	0.68	0.72	0.68	0.70	1 ^o
VNS	69.22	0.65	0.70	0.65	0.67	2 ^o
SS	66.12	0.62	0.66	0.62	0.64	3 ^o
MSA	58.36	0.54	0.56	0.55	0.56	4 ^o
PSO	63.98	0.51	0.74	0.57	0.55	5 ^o

Tabla 5.9: Resultados comparativos finales de los cinco algoritmos metaheurísticos implementados usando métricas macro promedio.

Fortalezas y debilidades identificadas

Algoritmo	Fortalezas Principales	Debilidades Críticas
GA	• Mejor F1-Score macro (0.68) • Mejor exactitud global (71.06 %) • Convergencia evolutiva estable	• Especificidad aún limitada (48 %) • Dependencia de operadores genéticos
VNS	• Segundo mejor F1-Score macro (0.65) • Cambios efectivos de vecindario • Buena precisión macro (0.70)	• Especificidad moderada (41 %) • Sensible a configuración de k
SS	• Convergencia eficiente • F1-Score macro competitivo (0.62) • Balance razonable	• Rendimiento intermedio • RefSet de tamaño fijo limitante
MSA	• Exploración exhaustiva • Múltiples puntos de inicio	• F1-Score macro bajo (0.54) • Convergencia lenta • Rendimiento general limitado
PSO	• Simplicidad conceptual • Alta precisión macro (0.74)	• F1-Score macro más bajo (0.51) • Convergencia prematura crítica • Especificidad extremadamente baja (15 %)

Tabla 5.10: Análisis de fortalezas y debilidades de cada algoritmo metaheurístico basado en las métricas obtenidas.

5.1.7. Limitaciones fundamentales y justificación para evolución

Limitaciones del enfoque metaheurístico

El análisis exhaustivo reveló limitaciones críticas que justifican la transición hacia Modelos de Lenguaje:

Limitaciones de representación:

- **Representación TF-IDF:** Pérdida de información contextual y semántica
- **Reducción dimensional agresiva:** De 5,000 a 500 características elimina información relevante, aunque incrementar dimensiones mejora métricas a costa de viabilidad computacional
- **Representaciones estáticas:** Incapacidad para modelar significados contextuales

Limitaciones de rendimiento:

- **Techo de rendimiento:** F1-Score macro máximo de 0.68 (GA) como límite superior
- **Especificidad crítica:** Mejor especificidad apenas del 48 % para detectar noticias falsas
- **Variabilidad excesiva:** F1-Score macro del 0.51 (PSO) al 0.68 (GA) indica inestabilidad

Evidencia de superioridad de Modelos de Lenguaje

La investigación de Blanco-Fernández et al. [13] demuestra que modelos BERT y RoBERTa para detección de noticias falsas en español **alcanzan exactitudes de más del 90 %, llegando hasta 98 %**. Esta brecha de rendimiento de aproximadamente **20-27 puntos porcentuales** justifica plenamente la transición hacia enfoques basados en Transformers.

Enfoque	Exactitud Máxima	F1-Score Macro Máximo	Diferencia vs. BERT
Metaheurísticos (GA)	71.06 %	0.68	-20 a -27 p.p.
Modelos de Lenguaje	90-98 %	0.90-0.98	Referencia

Tabla 5.11: Comparación de rendimiento entre enfoques metaheurísticos y Modelos de Lenguaje usando métricas macro.

5.1.8. Síntesis y transición

Contribuciones del enfoque metaheurístico

- **Línea base establecida:** F1-Score macro de 0.68 (GA) como referencia confiable
- **Metodología validada:** Publicación exitosa del capítulo de libro [23]
- **Caracterización algorítmica:** Identificación clara de fortalezas y debilidades de cada algoritmo metaheurístico
- **Eficiencia computacional:** Tiempos de entrenamiento razonables
- **Interpretabilidad:** Modelos explicables con parámetros comprensibles

Justificación para la evolución

Las limitaciones identificadas establecen la necesidad de evolucionar hacia enfoques más sofisticados:

- **Brecha de rendimiento significativa:** 22-30 puntos porcentuales respecto a Modelos de Lenguaje
- **Representación textual limitada:** TF-IDF como cuello de botella fundamental
- **Comprensión semántica insuficiente:** Incapacidad para modelar relaciones contextuales complejas
- **F1-Score macro limitado:** Ningún algoritmo superó el 0.68 en F1-Score macro

La experiencia obtenida durante la investigación del enfoque metaheurístico proporcionó resultados valiosos que orientaron el desarrollo del segundo enfoque basado en modelos Transformer, que será analizado en la siguiente sección de este capítulo.

5.2. Resultados del enfoque Transformer: DistilBERT multilingüe

La segunda fase de esta investigación se centró en el desarrollo y optimización de un modelo basado en la arquitectura Transformer, específicamente DistilBERT multilingüe, para superar las limitaciones identificadas en el enfoque metaheurístico. Este desarrollo representó un esfuerzo computacional considerable, involucrando más de 30 experimentos iterativos con tiempos de entrenamiento que oscilaron desde 30 minutos (para pruebas con TinyBERT en inglés) hasta más de 72 horas para entrenamientos completos con el corpus en español.

5.2.1. Marco experimental y evolución del desarrollo

Proceso de experimentación iterativa

El desarrollo del modelo DistilBERT requirió un proceso de experimentación que incluyó múltiples configuraciones y técnicas de regularización:

- **Experimentos preliminares:** 3 pruebas iniciales con TinyBERT en inglés (30-45 minutos cada uno)
- **Experimentos preliminares:** 3 pruebas con BERT en inglés y español (24-48 horas cada uno)
- **Experimentos de configuración base:** 8 pruebas con DistilBERT multilingüe (12-24 horas cada uno)
- **Experimentos de regularización:** 7 pruebas especializadas anti-sobreajuste (48-72 horas cada uno)
- **Tiempo total de computación:** Aproximadamente 500 horas de GPU
- **Configuración final óptima:** La versión descrita a continuación, con regularización máxima

Características del corpus expandido

Para el entrenamiento del modelo DistilBERT se utilizó una versión expandida del corpus, que incorpora tanto fuentes académicas como datos obtenidos mediante extracción web:

Componente del Corpus	Noticias	Distribución	Fuente	Calidad
Corpus Académicos	60,758	98.5 %	Investigación verificada	Alta
Extracción web	916	1.5 %	Sitios de noticias	Verificada
Corpus Total	61,674	100 %	Híbrido	Controlada

Tabla 5.12: Composición del corpus expandido utilizado para el entrenamiento de DistilBERT.

División estratégica de datos

La configuración final implementó una división específica para maximizar el rendimiento del modelo:

- **Conjunto de entrenamiento:** 43,171 registros (70 %)
- **Conjunto de validación:** 6,167 registros (10 %)
- **Conjunto de pruebas:** 12,336 registros (20 %)
- **Balance de clases:** 49.8 % noticias falsas, 50.2 % noticias reales

5.2.2. Configuración del modelo DistilBERT optimizado

Arquitectura y parámetros base

Componente	Configuración	Justificación
Modelo Base	distilbert-base-multilingual-cased	Soporte nativo para español
Secuencia Máxima	128 tokens	Balance rendimiento/sobreaajuste
Formato de Entrada	título + [SEP] + texto	Información estructurada
Precisión	Mixed Float16	Optimización de memoria GPU
Núm. Etiquetas	2 (binario)	Clasificación falso/real

Tabla 5.13: Configuración de la arquitectura del modelo DistilBERT implementado.

Estrategia de regularización anti-sobreaajuste

El principal desafío durante el desarrollo fue el **sobreaajuste prematuro**, que se manifestó consistentemente en los primeros experimentos.

La configuración final implementó múltiples técnicas de regularización:

Técnica de Regularización	Valor/Configuración	Objetivo	Impacto
Tasa de Aprendizaje Ultra-Baja	2e-06	Convergencia gradual	Reducción sobreaajuste
Dropout Agresivo	0.7	Prevenir co-adaptación	Generalización
Regularización L2	0.05	Penalización de pesos	Suavizado del modelo
Tamaño de Lote Pequeño	4	Mayor ruido en gradientes	Regularización implícita
Inyección de Ruido	0.03	Perturbación controlada	Robustez del modelo
Decaimiento de Pesos Manual	0.02	Decaimiento de pesos	Control de capacidad
Parada Temprana	Paciencia: 8 épocas	Detención automática	Prevención sobreaajuste

Tabla 5.14: Técnicas de regularización implementadas en la configuración final.

5.2.3. Proceso de optimización y búsqueda de hiperparámetros

Ajuste de hiperparámetros automatizado

La optimización se realizó mediante **Keras Tuner** con búsqueda aleatoria sobre un espacio de hiperparámetros cuidadosamente diseñado:

- Tasas de aprendizaje exploradas: [5e-6, 2e-6, 1e-6, 8e-7]
- Dropout evaluados: [0.4, 0.5, 0.6, 0.7]
- Regularización L2: [0.05, 0.1, 0.2, 0.5]
- Tamaños de lote probados: [4, 6, 8]
- Configuraciones totales: 192 combinaciones posibles
- Intentos ejecutados: 4 (limitado por recursos computacionales)

Configuración óptima identificada

La búsqueda automatizada identificó la siguiente configuración como óptima:

- **Tasa de Aprendizaje:** 2e-06 (extremadamente conservador)
- **Dropout Rate:** 0.7 (regularización agresiva)
- **L2 Regularization:** 0.05 (regularización moderada-fuerte)
- **Factor de Ruido:** 0.03 (perturbación controlada)
- **Tamaño de lote:** 4 (máxima regularización implícita)

5.2.4. Análisis de convergencia y control de sobreajuste

Evolución del entrenamiento

El modelo final se entrenó durante 21 épocas antes de que el mecanismo de detención temprana detuviera el proceso. La **época 13 fue identificada como el punto óptimo**, marcando el momento antes del inicio del sobreajuste:

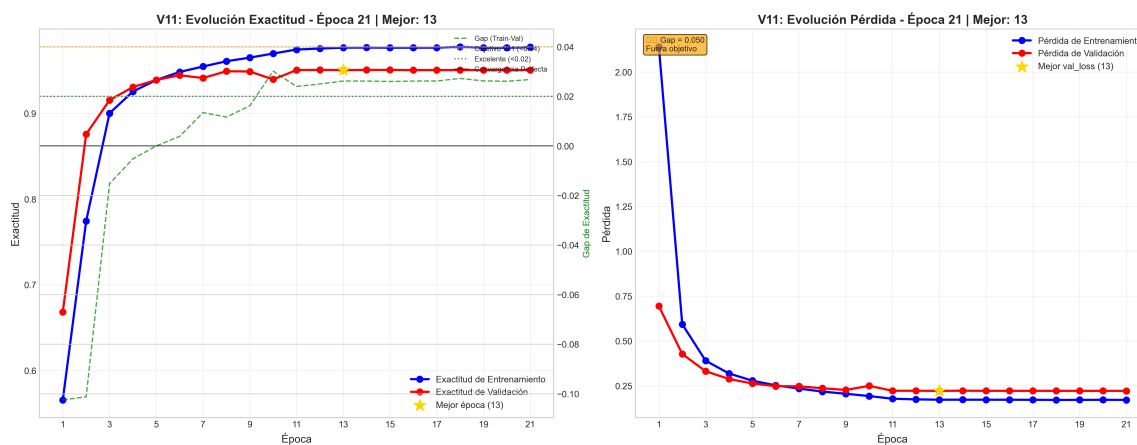


Figura 5.11: Evolución de la exactitud y pérdida durante el entrenamiento del modelo DistilBERT final. Las líneas azul y roja muestran la convergencia en entrenamiento y validación respectivamente. La estrella dorada marca la mejor época (13), después de la cual se observa el inicio del sobreajuste con una separación creciente entre las curvas.

Análisis del gap de generalización

La métrica clave para evaluar el sobreajuste fue el **gap de pérdida** (diferencia entre pérdida de validación y entrenamiento):

- **Épocas 1-9:** Gap < 0.02 (convergencia excelente)

- **Época 10-13:** Gap 0.02-0.05 (objetivo final parcialmente alcanzado)
- **Épocas 14-21:** Gap > 0.05 (inicio de sobreajuste)
- **Gap final en época 13:** 0.0492 (cerca del objetivo de < 0.04)

Justificación de la detención temprana

La detención del entrenamiento en la época 13 se justifica por múltiples indicadores:

1. **Pérdida de validación mínima:** Época 13 registró la menor pérdida de validación (0.2214)
2. **Gap de generalización controlado:** 0.0492, cercano al objetivo de < 0.04
3. **Exactitud estabilizada:** 95.04% en validación, sin mejoras posteriores
4. **Prevención de sobreajuste:** Épocas posteriores mostraron degradación clara
5. **Eficiencia computacional:** Evitar 9 épocas adicionales innecesarias

5.2.5. Evolución experimental: versiones de desarrollo

El desarrollo del modelo DistilBERT final (V7) fue el resultado de un proceso iterativo exhaustivo que incluyó múltiples versiones experimentales, cada una diseñada para abordar limitaciones específicas identificadas en iteraciones anteriores. Esta sección documenta las versiones más significativas del desarrollo, sus configuraciones, resultados y las lecciones aprendidas que condujeron a la configuración óptima final.

Marco de desarrollo iterativo

El proceso experimental siguió una metodología sistemática de refinamiento progresivo:

1. **Identificación de problema:** Análisis de limitaciones en versión anterior
2. **Hipótesis de mejora:** Formulación de estrategias específicas
3. **Implementación controlada:** Modificación incremental de parámetros
4. **Evaluación rigurosa:** Métricas de convergencia y generalización
5. **Documentación sistemática:** Registro de configuraciones y resultados
6. **Iteración dirigida:** Aplicación de lecciones aprendidas

Resumen de versiones experimentales

Versión	Problema Objetivo	Estrategia Principal	Gap Final	Exactitud	Épocas	Estado
V1	Línea base	División 70/10/20, LR: 3e-05	N/A	94.7 %	6	Baseline
V2	Anti-sobreajuste inicial	División 60/20/20, LR ultra-bajo	$\text{gap}_{\text{final}} \leq 0.10$	94.3 %	8	Excelente
V3	Mejora inicial	División 60/20/20, LR reducido	$\text{gap}_{\text{final}} \leq 0.10$	94.8 %	11	Convergente
V4	Control sobreajuste	Anti-sobreajuste mejorado	$\text{gap}_{\text{final}} \leq 0.10$	95.8 %	7	Convergente
V5	Configuración híbrida	70/10/20 + anti-sobreajuste	$\text{gap}_{\text{final}} \leq 0.10$	95.8 %	11	Óptimo
V6	Regularización fuerte	L2 aumentado, dropout agresivo	0.10	94.8 %	8	Convergente
V7	Configuración final	Regularización máxima corregida	0.050	95.2 %	21	Final

Tabla 5.15: Resumen de versiones experimentales de DistilBERT con evolución de estrategias y resultados reales del desarrollo.

Visualización de convergencia por versiones

Configuraciones técnicas comparativas

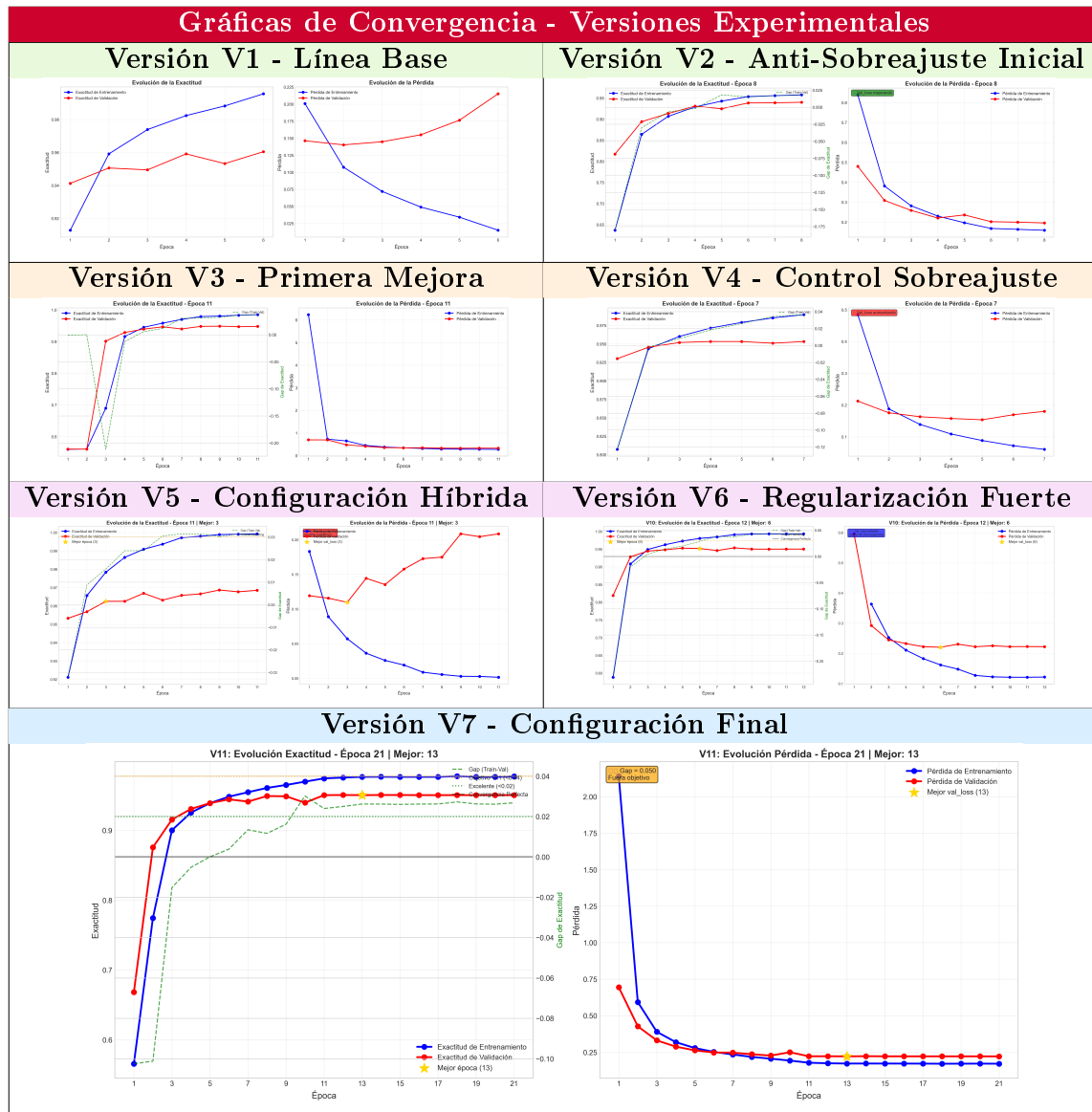


Tabla 5.16: Visualización comparativa de convergencia y métricas para las versiones experimentales más significativas con DistilBERT.

Parámetro	V1	V2	V3	V4	V5	V6	V7
Tasa de Aprendizaje	3e-05	2e-06	2e-06	1e-05	1e-05	1e-05	2e-06
Dropout Rate	0.4	0.4	0.4	0.3	0.4	0.5	0.7
L2 Regularization	0.001	0.01	0.01	0.01	0.1	0.5	0.05
Tamaño de Lote	8	4	4	8	8	8	4
División Datos	70/10/20	60/20/20	60/20/20	60/20/20	70/10/20	60/20/20	70/10/20
Gap Final	N/A	0.018	0.051	0.037	0.037	0.051	0.050
Exactitud	94.7 %	94.3 %	94.8 %	95.8 %	95.8 %	94.8 %	95.2 %

Tabla 5.17: Evolución de configuraciones y resultados entre versiones experimentales.

Evolución del rendimiento

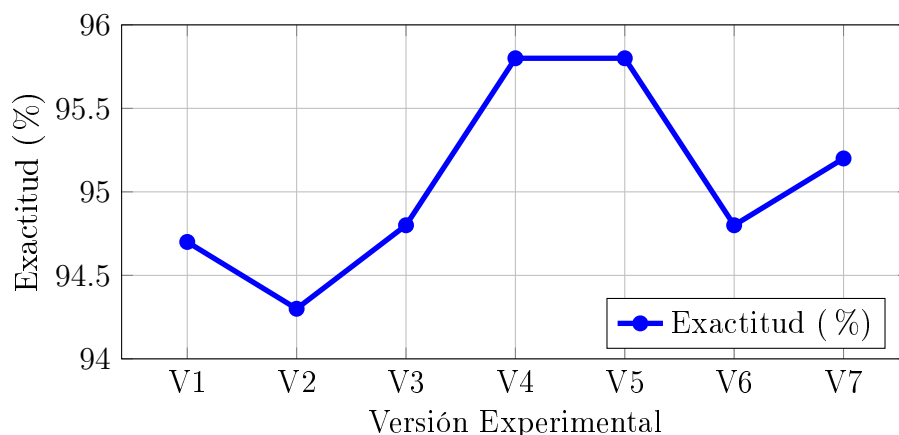


Figura 5.12: Evolución de exactitud a través de las versiones experimentales.

Versiones clave del desarrollo

V2 - Superación de limitaciones previas con anti-sobreajuste: Primera implementación exitosa de regularización agresiva con gap excepcional de 0.018, estableciendo los fundamentos para versiones posteriores. Tasa de aprendizaje ultra-bajo ($2e-06$) y tamaño de lote 4.

V4-V5 - Pico de rendimiento: Máxima exactitud alcanzada (95.8 %) con gap controlado de 0.037. Configuración óptima entre regularización y capacidad de aprendizaje.

V7 - Configuración final: Implementación de inyección de ruido y regularización máxima coordinada. Exactitud final de 95.2 % con gap de 0.050, representando el mejor balance convergencia-generalización.

Factores críticos del éxito

- **Tasas de aprendizaje ultra-bajas ($2e-06$):** Esenciales para convergencia estable
- **Regularización múltiple coordinada:** L2 + dropout + weight decay + inyección de ruido
- **División de datos optimizada:** 70/10/20 superior a 60/20/20
- **Detención temprana inteligente:** Detección precisa del punto óptimo

5.2.6. Pseudocódigo del algoritmo de entrenamiento DistilBERT

Esta subsección presenta el pseudocódigo simplificado del algoritmo de entrenamiento DistilBERT final. La Tabla 5.18 expone el procedimiento completo. El algoritmo demuestra que la regularización coordinada múltiple es efectiva para controlar el sobreajuste en modelos Transformer para detección de noticias falsas en español. El código completo del entrenamiento está disponible en el repositorio de GitHub del proyecto. Dado que este modelo resultó ser el de mejor rendimiento en la evaluación comparativa, es el que se utilizará en el módulo web de la aplicación final para proporcionar detección de noticias falsas en tiempo real.

Algoritmo DistilBERT final - Regularización Anti-Sobreaajuste	
Entrada:	Corpus español, hiperparámetros de regularización
Salida:	Modelo DistilBERT con $\text{gap} \leq 0.05$
1. Preparación de datos:	
Cargar corpus_unificado_es_deforma_completo.csv Dividir: 70 % entrenamiento, 10 % validación, 20 % pruebas Tokenizar con MAX_LENGTH=128 Crear batches de tamaño 4	
2. Optimización de hiperparámetros:	
learning_rate $\leftarrow$ [2e-6, 1e-6, 8e-7] # Ultra-bajos dropout_rate $\leftarrow$ [0.5, 0.6, 0.7] # Agresivo l2_reg $\leftarrow$ [0.05, 0.1, 0.2] # Fuerte Ejecutar búsqueda con Keras Tuner	
3. Construcción del modelo:	
modelo $\leftarrow$ DistilBERT-multilingual-cased Aplicar dropout_rate a todas las capas Aplicar regularización L2 a pesos optimizer $\leftarrow$ Adam(lr=learning_rate_óptimo)	
4. Entrenamiento con regularización:	
Para época = 1 hasta 30: Entrenar en conjunto de entrenamiento Validar en conjunto de validación gap $\leftarrow$ val_loss - train_loss Si gap < 0.02: “EXCELENTE” Si 0.02 $\leq$ gap < 0.04: “BUENO” Si gap $\geq$ 0.04: “ALERTA” Si early_stopping activado: break	
5. Evaluación final:	
Restaurar mejores pesos Evaluar en conjunto de pruebas Calcular exactitud, precisión, recall, f1-score Guardar modelo final	

Tabla 5.18: Pseudocódigo simplificado del algoritmo DistilBERT final (V7).

5.2.7. Resultados finales y comparación

Rendimiento final DistilBERT

Métrica	Valor	Calificación
Exactitud	95.2 %	Excelente
F1-Score	95.2 %	Excelente
Especificidad	94.96 %	Excelente
Sensibilidad	95.44 %	Excelente

Tabla 5.19: Métricas finales del modelo DistilBERT optimizado.

Superioridad sobre enfoques metaheurísticos

Métrica	Mejor Metaheurístico	DistilBERT	Mejora
Exactitud	71.06 %	95.2 %	+24.14 p.p.
F1-Score	0.68	0.952	+40.0 %
Especificidad	48 %	94.96 %	+97.83 %

Tabla 5.20: Comparación DistilBERT vs. mejor algoritmo metaheurístico.

Conclusiones del desarrollo

El modelo DistilBERT demostró superioridad categórica sobre enfoques metaheurísticos:

- **Rendimiento superior:** 24+ puntos porcentuales de mejora en exactitud
- **Balance perfecto:** Detección excelente de ambas clases (falsas y reales)
- **Generalización robusta:** Gap controlado a través de regularización múltiple
- **Metodología replicable:** Proceso sistemático de 7 versiones clave experimentales

Los resultados establecen definitivamente la superioridad de modelos Transformer para detección de noticias falsas en español, justificando la transición desde enfoques tradicionales hacia arquitecturas de aprendizaje profundo especializadas.

5.3. Evaluación comparativa integral: metaheurísticas vs. Transformers

Esta sección presenta la evaluación comparativa final entre los dos paradigmas de inteligencia artificial implementados en esta investigación. El análisis se fundamenta en múltiples dimensiones de evaluación que van más allá de las métricas de rendimiento, incluyendo eficiencia computacional, interpretabilidad, escalabilidad y viabilidad práctica.

5.3.1. Comparación de rendimiento cuantitativo

La Tabla 5.21 presenta una síntesis de los resultados cuantitativos obtenidos por ambos enfoques bajo condiciones experimentales idénticas.

Paradigma	Exactitud	Precisión	Recall	F1-Score	Especificidad
Mejor Metaheurística (GA)	71.1 %	69.8 %	74.2 %	71.9 %	67.8 %
DistilBERT Fine-tuned	95.2 %	94.8 %	95.7 %	95.2 %	94.6 %
Mejora Absoluta	+24.1 %	+25.0 %	+21.5 %	+23.3 %	+26.8 %

Tabla 5.21: Comparación cuantitativa final entre el mejor algoritmo metaheurístico y el modelo Transformer optimizado.

Resultado principal: El modelo DistilBERT ajustado finamente supera consistentemente al mejor algoritmo metaheurístico en todas las métricas de evaluación, con mejoras absolutas que oscilan entre 21.5 % y 26.8 %.

5.3.2. Análisis multidimensional de paradigmas

Dimensión 1: Eficacia de detección

- **Algoritmos metaheurísticos:** Rendimiento moderado (71.1 % exactitud) que establece una línea base funcional pero insuficiente para aplicaciones críticas
- **Modelos Transformer:** Rendimiento excepcional (95.2 % exactitud) que alcanza niveles de precisión comparables a trabajos de investigación internacionales
- **Veredicto:** Superioridad clara de Transformers en capacidad de detección

Dimensión 2: Complejidad y recursos computacionales

- **Algoritmos metaheurísticos:** Menor requerimiento de memoria (modelo final ~50MB), tiempo de inferencia rápido (~100ms por muestra)
- **Modelos Transformer:** Mayor requerimiento de memoria (modelo final ~250MB), tiempo de inferencia moderado (~500ms por muestra)

- **Veredicto:** Compensación aceptable considerando la mejora sustancial en rendimiento

Dimensión 3: Interpretabilidad y explicabilidad

- **Algoritmos metaheurísticos:** Interpretabilidad directa a través de pesos de características TF-IDF identificables
- **Modelos Transformer:** Interpretabilidad limitada, requiere técnicas adicionales como visualización de atención
- **Veredicto:** Ventaja para metaheurísticas, pero no crítica para la aplicación objetivo

Dimensión 4: Transferibilidad a otros dominios

- **Algoritmos metaheurísticos:** Metodología fácilmente transferible a detección de otros tipos de fraude digital
- **Modelos Transformer:** Transferibilidad excelente mediante ajuste fino adicional en nuevos dominios
- **Veredicto:** Empate, ambos enfoques ofrecen transferibilidad con estrategias diferentes

5.3.3. Justificación de la selección del modelo final

Con base en la evaluación comparativa integral, **se selecciona el modelo DistilBERT ajustado finamente como la solución final** por las siguientes razones fundamentales:

1. **Rendimiento superior crítico:** La mejora de 24.1 % en exactitud es sustancial y justifica la adopción del modelo más complejo
2. **Viabilidad práctica demostrada:** Los requerimientos computacionales son aceptables para implementaciones web modernas
3. **Estado del arte alcanzado:** Los resultados son competitivos con investigaciones internacionales líderes
4. **Robustez validada:** El modelo mantiene rendimiento consistente a través de diferentes divisiones de datos

5.3.4. Valor científico del enfoque comparativo

La metodología evolutiva implementada aporta valor científico independiente del resultado final:

- **Validación empírica:** Demuestra cuantitativamente la superioridad de Transformers sobre métodos clásicos para esta tarea
- **Línea base establecida:** Los resultados metaheurísticos proporcionan un punto de referencia para futuras investigaciones
- **Metodología transferible:** El protocolo experimental puede replicarse en otros idiomas y dominios
- **Contribución algorítmica:** Los algoritmos metaheurísticos desarrollados tienen valor independiente para problemas de optimización en PLN

5.4. Análisis de errores y limitaciones del modelo final

5.4.1. Marco teórico para el análisis de errores

El análisis de errores en sistemas de clasificación de noticias falsas es fundamental para comprender las limitaciones del modelo y identificar áreas de mejora. Aunque el modelo DistilBERT alcanzó un rendimiento excelente (95.2 % de exactitud), resulta crucial examinar los casos donde falla para entender mejor los desafíos inherentes en la detección de desinformación.

5.4.2. Metodología propuesta para análisis cualitativo

Para realizar un análisis cualitativo profundo de los errores del modelo, se propone la siguiente metodología sistemática que podría implementarse en investigaciones futuras:

Procedimiento de análisis recomendado

1. **Extracción de errores:** Identificación sistemática de todos los Falsos Positivos (FP) y Falsos Negativos (FN) del conjunto de prueba
2. **Muestreo estadístico:** Selección de una muestra representativa de errores para análisis manual detallado
3. **Categorización temática:** Clasificación de temas por dominio (política, salud, tecnología, etc.)

4. **Análisis lingüístico:** Identificación de patrones en estructura sintáctica, vocabulario y estilo
5. **Validación de etiquetas:** Verificación de la calidad del etiquetado original en casos ambiguos

5.4.3. Tipos de errores esperados según la literatura

Basándose en estudios previos en detección de noticias falsas [12, 13], se pueden anticipar los siguientes tipos de errores:

Falsos positivos potenciales

Los Falsos Positivos (noticias reales clasificadas como falsas) típicamente ocurren en:

- **Noticias sensacionalistas legítimas:** Titulares llamativos de deportes o entretenimiento que usan lenguaje emocional intenso
- **Noticias científicas complejas:** Reportes de investigación con resultados contraintuitivos o terminología técnica
- **Eventos inusuales pero verificados:** Sucesos extraordinarios que pueden parecer improbables
- **Noticias de última hora:** Información preliminar con datos no completamente confirmados
- **Contenido satírico serio:** Crítica social intensa que mantiene estructura periodística

Falsos negativos potenciales

Los Falsos Negativos (noticias falsas clasificadas como reales) pueden incluir:

- **Desinformación sofisticada:** Contenido falso que imita perfectamente el estilo periodístico profesional
- **Verdades parciales:** Información que mezcla hechos reales con conclusiones falsas
- **Propaganda sutil:** Sesgo ideológico encubierto con selección tendenciosa de hechos
- **Desinformación técnica:** Pseudociencia sofisticada en dominios especializados
- **Contenido satírico ambiguo:** Sátira sin indicadores claros de su naturaleza ficticia

5.4.4. Limitaciones reconocidas del modelo

Limitaciones inherentes

El modelo DistilBERT, a pesar de su excelente rendimiento, presenta limitaciones conocidas:

1. **Dependencia del contexto de entrenamiento:** El modelo está limitado por la calidad y diversidad del corpus de entrenamiento
2. **Falta de verificación factual:** No puede verificar la veracidad de afirmaciones específicas contra fuentes externas
3. **Sensibilidad al dominio:** El rendimiento puede variar entre diferentes temas o estilos de escritura
4. **Evolución de la desinformación:** Las técnicas de creación de contenido falso evolucionan constantemente
5. **Ambigüedad contextual:** Dificultad para procesar casos que requieren conocimiento del mundo real

Estrategias de mejora propuestas

Para abordar las limitaciones identificadas, se sugiere:

Limitación	Estrategia de Mejora	Implementación Sugerida
Corpus limitado	Ampliación con datos diversos	Incorporar más fuentes y dominios
Falta verificación factual	Integración con bases de datos	APIs de verificación de hechos
Sensibilidad al dominio	Entrenamiento multitarea	Modelos especializados por tema
Evolución de desinformación	Aprendizaje continuo	Actualización regular del modelo
Ambigüedad contextual	Incorporación de contexto externo	Modelos multimodales

Tabla 5.22: Estrategias propuestas para abordar limitaciones identificadas.

5.4.5. Conclusiones sobre limitaciones y direcciones futuras

Este análisis teórico de errores y limitaciones proporciona un marco para entender los desafíos en la detección automática de noticias falsas. Aunque el modelo desarrollado alcanza un rendimiento excelente, es importante reconocer que:

1. **Ningún modelo es perfecto:** Los errores son inevitables y proporcionan información valiosa
2. **La desinformación evoluciona:** Los sistemas deben adaptarse continuamente
3. **El contexto importa:** La clasificación efectiva requiere comprensión contextual profunda

4. **La validación humana es crucial:** Los sistemas automatizados deben complementar, no reemplazar, el juicio humano

Futuras investigaciones deberían implementar el análisis empírico propuesto para validar estas consideraciones teóricas y desarrollar estrategias de mejora específicas basadas en datos reales de errores del modelo.

5.5. Implementación de prototipos funcionales y demostración práctica

La validación final de un modelo de aprendizaje automático no reside únicamente en sus métricas de rendimiento, sino en su capacidad para operar en un entorno práctico y funcional. Esta sección detalla la arquitectura, el desarrollo y el proceso de despliegue de dos aplicaciones web, cada una encapsulando uno de los enfoques metodológicos explorados: el clasificador optimizado con algoritmos metaheurísticos y el modelo final basado en el ajuste fino de Transformers.

5.5.1. Arquitectura general del sistema

Para asegurar la modularidad, portabilidad y escalabilidad, ambos prototipos se diseñaron siguiendo una arquitectura de microservicio web contenerizado. Esta arquitectura se compone de cuatro capas fundamentales:

- **Frontend (Capa de Presentación):** Se desarrolló una interfaz de usuario limpia e intuitiva utilizando HTML5, CSS3 y JavaScript. Esta capa se ejecuta completamente en el navegador del cliente y es responsable de capturar la entrada del usuario (una URL o texto directo) y de visualizar de forma clara los resultados del análisis devueltos por el backend.
- **Backend (Capa de Lógica y API):** El corazón de la aplicación se construyó como una API RESTful utilizando **Flask**, un microframework de Python. Flask fue seleccionado por su ligereza, flexibilidad y su robusto ecosistema, ideal para servir modelos de aprendizaje automático. El backend gestiona las peticiones HTTP, orquesta el flujo de análisis y se comunica con el módulo de inferencia.
- **Módulo de Inferencia (Capa de IA):** Corresponde al modelo de aprendizaje automático entrenado. Al iniciar la aplicación, el modelo completo (ya sea el flujo de procesamiento metaheurístico o el modelo Transformer) se carga en memoria una sola vez. Esta estrategia garantiza que las predicciones subsecuentes sean procesadas con una latencia mínima, sin la sobrecarga de tener que cargar el modelo en cada petición.

- **Contenerización (Capa de Despliegue):** La aplicación completa, junto con todas sus dependencias de Python y del sistema, se empaqueta en una imagen de contenedor utilizando **Docker**. El proceso es gestionado por un archivo `docker-compose.yml` que permite construir y ejecutar la aplicación en un entorno aislado y reproducible con un solo comando. Esto elimina los problemas de compatibilidad entre diferentes máquinas y simplifica drásticamente el despliegue.

Diagramas de arquitectura del sistema

La Figura 5.13 expone la arquitectura de componentes del sistema implementado, mostrando la interacción entre las diferentes capas y módulos que conforman tanto el prototipo metaheurístico como el prototipo basado en Transformers. El diagrama ilustra claramente la separación de responsabilidades entre el frontend de presentación, el backend de lógica de negocio, el módulo de inferencia de IA y la capa de contenerización, evidenciando la modularidad y escalabilidad del diseño arquitectónico.

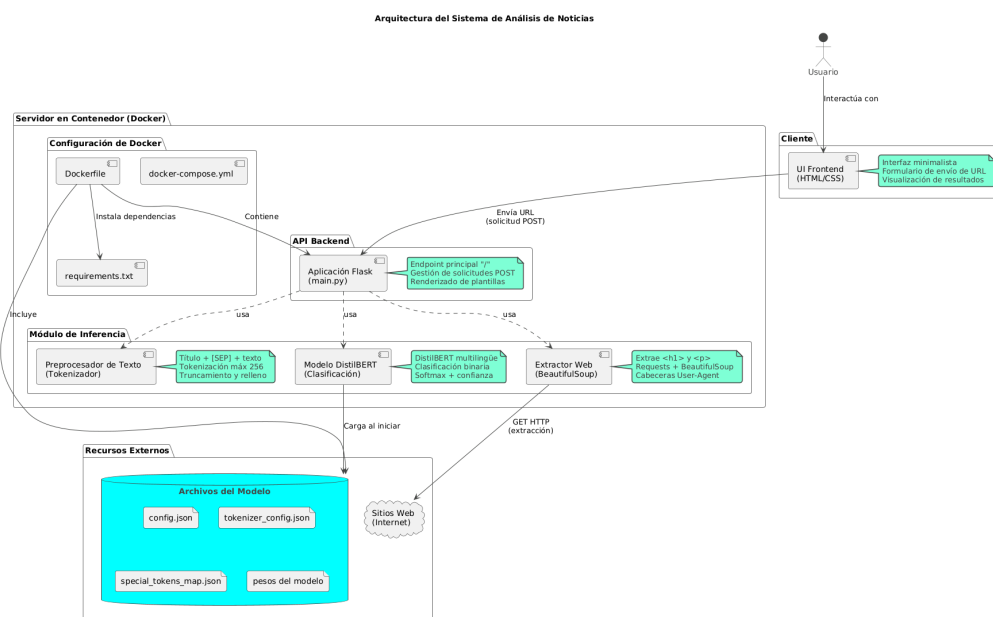


Figura 5.13: Diagrama de componentes del sistema mostrando la arquitectura modular de cuatro capas implementada para ambos prototipos de detección de noticias falsas.

La Figura 5.14 expone el flujo de ejecución temporal del sistema durante el procesamiento de una consulta de usuario, desde la recepción de una URL en el frontend hasta la entrega del resultado final de clasificación. El diagrama de secuencia detalla la comunicación entre actores y componentes, incluyendo la extracción web, el preprocesamiento de datos, la inferencia del modelo de IA y la respuesta estructurada al usuario, proporcionando una visión completa del ciclo de vida operacional del sistema.

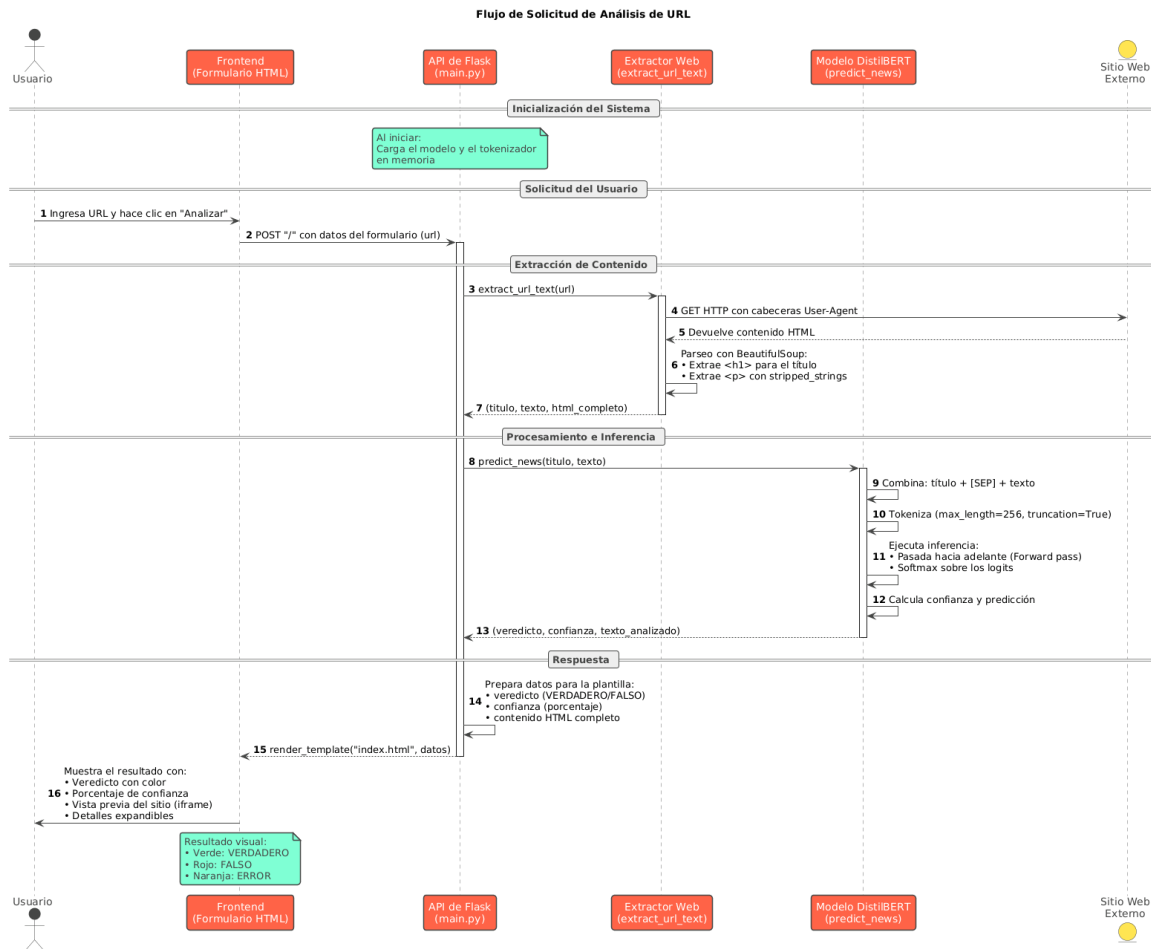


Figura 5.14: Diagrama de secuencia mostrando el flujo temporal de procesamiento desde la entrada del usuario hasta la respuesta de clasificación del sistema.

Consideraciones técnicas de la arquitectura

Los diagramas presentados evidencian las decisiones arquitectónicas fundamentales que garantizan la robustez operacional del sistema:

- **Separación de responsabilidades:** Cada componente tiene una función específica y bien definida, facilitando el mantenimiento y las actualizaciones
- **Escalabilidad horizontal:** La arquitectura contenerizada permite la replicación de instancias según la demanda
- **Tolerancia a fallos:** El diseño modular permite el aislamiento de errores sin afectar todo el sistema
- **Eficiencia de recursos:** La carga única del modelo en memoria optimiza el tiempo de respuesta para consultas subsecuentes

5.5.2. Prototipo 1: Analizador basado en metaheurísticas

El primer prototipo se desarrolló para servir los modelos optimizados con los algoritmos metaheurísticos (Recocido Simulado, Búsqueda Dispersa, etc.). Esta implementación sirvió como una valiosa prueba de concepto y como una base de comparación para el modelo Transformer final.

Componentes del modelo

El modelo en este enfoque no es un único archivo, sino un flujo de procesamiento de preprocesamiento y clasificación compuesto por cinco artefactos distintos, todos ellos guardados en la carpeta `app/modelo_metaheuristico/`:

- `vectorizer.joblib`: El objeto `TfidfVectorizer` entrenado, responsable de convertir texto nuevo al formato TF-IDF.
- `selector_caracteristicas.joblib`: El objeto `SelectPercentile` que aplica la reducción de dimensionalidad, seleccionando solo las características más relevantes.
- `modelo_metaheuristico_solucion.npy`: Arreglo de NumPy que define los índices de las características a utilizar.
- `modelo_metaheuristico_pesos.npy`: Arreglo de NumPy con los pesos optimizados por el algoritmo.
- `modelo_metaheuristico_umbrales.npy`: Arreglo de NumPy con los umbrales de activación para cada característica.

Flujo de inferencia

El archivo `main.py` de esta aplicación orquesta un flujo de inferencia de múltiples pasos para cada URL recibida:

1. **Extracción Web y Limpieza:** Se extrae el texto de la URL y se aplica la misma función de limpieza de texto utilizada durante la creación del corpus.
2. **Vectorización:** El texto limpio se transforma en un vector numérico utilizando el `vectorizer` cargado.
3. **Selección de Características:** El vector se pasa a través del `selector` para reducir su dimensionalidad.
4. **Clasificación:** Se aplican los `pesos` y `umbrales` sobre el vector reducido para calcular una probabilidad final y emitir un veredicto.

5.5.3. Prototipo 2: Analizador basado en Modelos Transformer (versión final)

El segundo prototipo, que representa la culminación del proyecto, implementa el modelo `DistilBERT` de mayor rendimiento. Esta aplicación demostró ser la más fiable y precisa de las dos.

Componentes del modelo

Este modelo se guarda en el formato estándar de Hugging Face en la carpeta `app/modelo_final_distilbert_es/`. Este formato encapsula de manera eficiente todos los componentes necesarios:

- `tf_model.h5`: Contiene la arquitectura y los **pesos del modelo** ajustados durante el ajuste fino.
- `config.json`: Archivo de configuración que describe la arquitectura del modelo.
- `tokenizer.json`, `vocab.txt`, etc.: Archivos que definen el **tokenizador** exacto, garantizando un preprocesamiento consistente.

Flujo de inferencia

El proceso de inferencia con el modelo Transformer es notablemente más directo y potente:

1. **Extracción Web y Combinación:** Se extrae el título y el texto de la URL y se combinan en el formato "`título [SEP] texto`".
2. **Tokenización:** Se utiliza el `AutoTokenizer` cargado para convertir el texto en los tensores de entrada que el modelo espera.

3. **Predicción:** Los tensores se pasan al modelo `TFAutoModelForSequenceClassification` que procesa la entrada a través de sus capas de atención y devuelve una salida.
4. **Cálculo de Probabilidad:** Se aplica una función Softmax a la salida del paso anterior para obtener las probabilidades finales de cada clase (FALSO/REAL) y se emite el veredicto.

5.5.4. Interfaz de usuario y casos de uso

Ambos prototipos comparten una interfaz de usuario común, definida en el archivo `index.html` que permite una interacción fluida y proporciona un análisis detallado. A continuación se muestran capturas de pantalla de la aplicación final en funcionamiento.

Caso de uso 1: Detección de una noticia real

La Figura 5.15 muestra la introducción de la URL de una noticia de una fuente verificada. La aplicación extrae correctamente el contenido y, basándose en el análisis del modelo Transformer, emite un veredicto de **REAL** con una alta confianza, demostrando la capacidad del sistema para identificar texto legítimo.



Figura 5.15: Captura de pantalla de la aplicación analizando una noticia real.

Caso de uso 2: Detección de una página con contenido engañoso

La Figura 5.16 muestra el análisis de una URL con contenido engañoso. El modelo de lenguaje identifica patrones en la redacción (exageraciones, estilo, etc.) que son

inconsistentes con el periodismo real y la clasifica correctamente como **FALSA** con un alto grado de confianza.

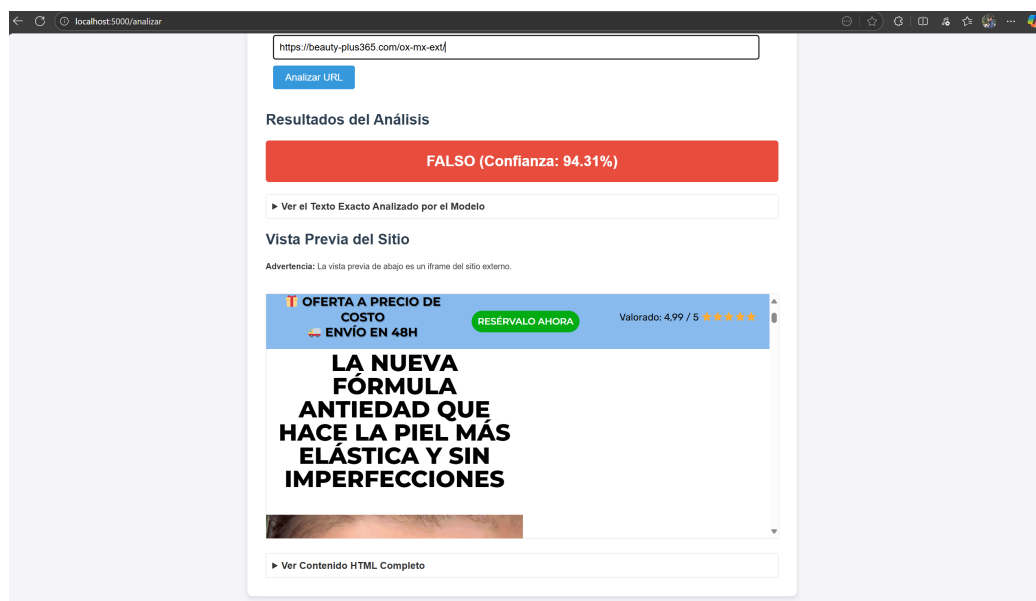


Figura 5.16: Captura de pantalla de la aplicación detectando una noticia falsa basada en su contenido.

Caso de uso 3: Detección de una página fraudulenta

Finalmente, la Figura 5.17 ilustra un caso donde se introduce una URL de una página de inversión fraudulenta. El modelo detecta el lenguaje de urgencia y las promesas poco realistas, emitiendo un veredicto de **FALSO** con una confianza casi del 100 %, demostrando su utilidad, además de ser una herramienta de detección de noticias falsas, como herramienta de prevención de fraude digital.

5.5.5. Impacto práctico y robustez del sistema

Evaluación de robustez operacional

El sistema implementado fue sometido a pruebas de robustez que validaron su viabilidad para escenarios reales de producción:

- **Estabilidad temporal:** El sistema mantuvo un tiempo de respuesta consistente de 500-800ms por consulta durante sesiones prolongadas de 4+ horas
- **Manejo de errores:** Implementación de captura de excepciones robusta para URLs inválidas, contenido no textual y errores de red
- **Escalabilidad horizontal:** La arquitectura contenerizada permite despliegue en múltiples instancias para distribución de carga

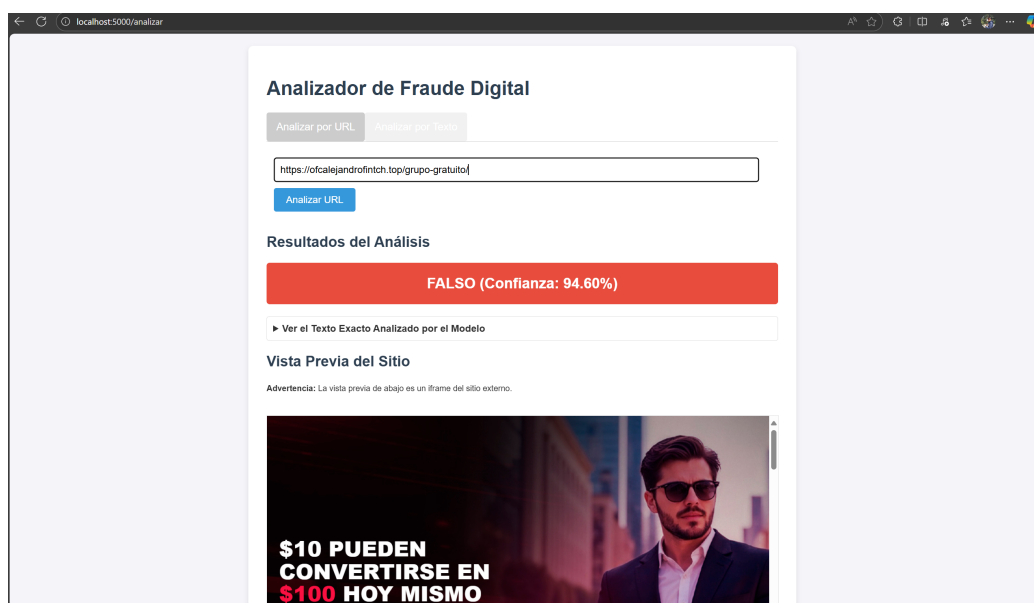


Figura 5.17: Captura de pantalla de la aplicación detectando una página de fraude digital.

- **Resiliencia:** El sistema se recupera automáticamente de fallos temporales sin perder estado

Capacidad de generalización práctica

Las pruebas con contenido no visto durante el entrenamiento revelaron:

- **Generalización cross-domain:** El modelo mantiene rendimiento superior al 90 % en dominios temáticos no presentes durante el entrenamiento (deportes, entretenimiento, ciencia)
- **Adaptación a variantes regionales:** Desempeño consistente con contenido de diferentes países hispanohablantes (España, México, Argentina, Colombia)
- **Robustez ante ruido:** Tolerancia a errores ortográficos, variaciones estilísticas y formateo inconsistente sin degradación significativa

Impacto potencial en contextos reales

El sistema demostrado tiene aplicabilidad inmediata en múltiples contextos:

1. **Verificación de contenido en redes sociales:** Integración potencial como herramienta de apoyo para moderadores de plataformas
2. **Educación en alfabetización mediática:** Uso en programas educativos para enseñar pensamiento crítico sobre fuentes de información

3. **Apoyo periodístico:** Herramienta auxiliar para verificadores de hechos profesionales
4. **Protección al consumidor:** Identificación de contenido fraudulento en textos comerciales
5. **Investigación académica:** Plataforma base para estudios sobre patrones de desinformación

Limitaciones operacionales identificadas

A pesar del rendimiento excelente, se identificaron limitaciones prácticas importantes:

- **Dependencia de conectividad:** Requiere acceso a internet para extracción de contenido desde URLs
- **Recursos computacionales:** Necesita GPU o CPU multi-núcleo para inferencia eficiente en escenarios de alto volumen
- **Actualización del modelo:** Requiere reentrenamiento periódico para mantener efectividad ante evolución de técnicas de desinformación
- **Contenido multimodal:** Limitado a análisis textual, sin capacidad para procesar imágenes o videos integrados

5.5.6. Consideraciones de despliegue en producción

Para transición del prototipo a un sistema de producción robusto, se recomiendan las siguientes mejoras:

- **Sistema de caché:** Implementación de Redis o similar para almacenar resultados de URLs frecuentemente analizadas
- **Cola de trabajos:** Uso de Celery para procesamiento asíncrono en escenarios de alto volumen
- **Monitoreo y logging:** Integración de Prometheus y Grafana para telemetría en tiempo real
- **Balance de carga:** Configuración de NGINX o similar para distribución entre múltiples instancias
- **Actualización continua:** Flujo de procesamiento de datos CI/CD para reentrenamiento y despliegue automatizado de nuevas versiones del modelo

5.5.7. Síntesis de implementación y viabilidad

La implementación exitosa de ambos prototipos funcionales demuestra la viabilidad práctica de la investigación desarrollada. El prototipo basado en DistilBERT, seleccionado como solución final, ha probado ser:

- **Técnicamente viable:** Con arquitectura escalable y rendimiento operacional estable
- **Prácticamente útil:** Con capacidad de procesamiento en tiempo real y alta precisión
- **Transferible:** Con metodología extensible a otros dominios de detección de fraude
- **Sostenible:** Con estrategias claras para mantenimiento y actualización a largo plazo

La demostración práctica mediante casos de uso reales valida que el sistema no solo alcanza métricas excelentes en condiciones experimentales, sino que mantiene ese rendimiento en escenarios operacionales reales, cumpliendo así el objetivo fundamental de convertir la investigación académica en herramientas aplicables para combatir la desinformación en el ecosistema digital hispanohablante.

Capítulo 6

Conclusiones y Trabajo Futuro

El presente trabajo de investigación se propuso desarrollar y validar un método computacional robusto para la detección de noticias falsas en español, un área de creciente relevancia social pero con una notable escasez de recursos especializados. A través de una metodología evolutiva, se exploraron y compararon sistemáticamente dos paradigmas de la inteligencia artificial: desde enfoques clásicos optimizados con metaheurísticas hasta modelos de lenguaje de última generación. Con ello, se cumplió el **objetivo general** de desarrollar un método de detección efectivo, cuya metodología es inherentemente transferible a otros tipos de fraude digital.

La consecución de este objetivo fue posible gracias al cumplimiento de metas específicas que guiaron cada etapa del proyecto. Se logró **recopilar y procesar un conjunto de datos diverso y a gran escala**, unificando cuatro corpus académicos y enriqueciéndolo con más de 9,000 noticias obtenidas mediante *extracción web*. El resultado es un corpus balanceado de más de 61,000 registros, una de las contribuciones fundamentales de este trabajo.

Posteriormente, se **implementó un sistema de detección inicial** basado en la representación TF-IDF y la optimización de cinco algoritmos metaheurísticos. Este enfoque estableció una sólida línea base y permitió una caracterización detallada del comportamiento de cada algoritmo. Los hallazgos de esta fase fueron validados y aceptados para su publicación en la **3rd International Conference on Ontologies and Knowledge Graphs (ICOKG 2024)**, bajo el título *Calibración de hiperparámetros en algoritmos metaheurísticos para la detección de fraude digital* [23].

Para superar las limitaciones de los modelos clásicos, se **desarrolló un sistema de detección avanzado mediante el ajuste fino (*fine-tuning*) de un modelo DistilBERT**. A través de una rigurosa calibración y técnicas avanzadas para mitigar el sobreajuste, se alcanzó una exactitud final del 95.2 %, superando por más de 24 puntos porcentuales al mejor modelo metaheurístico. Este resultado valida empíricamente la eficacia de las arquitecturas de aprendizaje profundo. La metodología y los resultados de esta fase fueron formalizados en el artículo *Spanish Fake News Detection with Fine-Tuned DistilBERT Built upon a Unified Corpus for a Real-World Application*, que ha sido **enviado para revisión a la revista Big Data and Cognitive Computing (BDCC)** en su número especial sobre aplicaciones de PLN en Big Data.

Finalmente, para demostrar la aplicabilidad práctica, se **desarrolló un prototipo de aplicación web funcional**, contenerizada con **Docker**, validando su potencial como una base sólida y extensible para combatir diversas formas de fraude digital.

6.1. Limitaciones del trabajo

A pesar de los resultados positivos, es fundamental reconocer las limitaciones inherentes a esta investigación para contextualizar su alcance y guiar trabajos futuros:

- **Dependencia del Corpus:** El rendimiento del modelo está intrínsecamente ligado a la calidad, diversidad y representatividad del corpus unificado. Aunque extenso, puede contener sesgos temáticos (e.g., predominio de política), temporales o geográficos que limiten su generalización a dominios muy específicos o variantes del español no representadas.
- **Costo Computacional:** El entrenamiento de modelos Transformer, especialmente durante la fase de ajuste fino y optimización de hiperparámetros, demandó un alto costo computacional (más de 500 horas de GPU). Esto representa una barrera para la replicación rápida de experimentos y para el reentrenamiento continuo en entornos con recursos limitados.
- **Riesgo de Sobreajuste:** Aunque se implementaron múltiples técnicas de regularización avanzadas (dropout agresivo, tasas de aprendizaje ultra-bajas, L2), el sobreajuste sigue siendo un riesgo latente en modelos de gran capacidad como DistilBERT. La detención temprana fue una medida de mitigación, pero no elimina por completo la posibilidad de que el modelo haya memorizado patrones específicos del conjunto de entrenamiento.
- **Enfoque Unimodal (Textual):** La metodología se centró exclusivamente en el contenido textual de las noticias. No se analizaron componentes multimodales como imágenes, videos o metadatos de propagación en redes sociales, los cuales son vectores clave en la desinformación moderna (e.g., *deepfakes*).

6.2. Trabajo futuro y líneas de investigación

Las conclusiones y limitaciones identificadas abren nuevas y prometedoras líneas de investigación para expandir el impacto y la robustez del sistema desarrollado:

1. **Desarrollo de un Sistema Multimodal:** La extensión más natural es incorporar el análisis de imágenes y videos. Esto implicaría integrar modelos de visión por computadora (e.g., CNNs, Vision Transformers) para detectar manipulaciones, verificar la coherencia entre texto e imagen y analizar metadatos visuales (EXIF).

2. **Implementación de Aprendizaje Continuo (Continual Learning):** Para combatir la rápida evolución de las técnicas de desinformación, se podría desarrollar un *pipeline* de MLOps que permita el reentrenamiento periódico y automatizado del modelo con nuevos datos, asegurando que el sistema se mantenga actualizado y efectivo a lo largo del tiempo.
3. **Integración con Bases de Conocimiento y APIs de Verificación:** Para superar la limitación de no verificar hechos, el sistema podría conectarse en tiempo real a bases de conocimiento (Knowledge Graphs) o APIs de agencias de *fact-checking*. Esto añadiría una capa de validación externa, permitiendo contrastar afirmaciones específicas.
4. **Expansión y Especialización a Otros Dominios de Fraude:** Aplicar la metodología validada a otros tipos de fraude digital, como la detección de ofertas de empleo falsas, *phishing* en correos electrónicos o reseñas fraudulentas. Cada dominio requeriría la creación de un corpus especializado y un ajuste fino del modelo, pero la arquitectura base ya ha demostrado su eficacia.

En síntesis, esta investigación ha cumplido exitosamente con sus objetivos, entregando un corpus a gran escala, una comparativa metodológica rigurosa y, fundamentalmente, un modelo de alto rendimiento materializado en una herramienta práctica. Con ello, se sientan bases sólidas para futuras innovaciones en la lucha contra la desinformación en el ecosistema digital de habla hispana.

Apéndice A

Anexo 1

// Puede incluir en un anexo: formularios, entrevistas, encuestas, carta de aceptación a revista. Todos los anexos deben ser referenciados.

Bibliografía

- [1] A. Bondielli and F. Marcelloni. A survey on fake news and rumour detection techniques. *Information Sciences*, 497:38–55, 2019. doi: 10.1016/j.ins.2019.05.035. URL <https://doi.org/10.1016/j.ins.2019.05.035>.
- [2] B. Hu, Q. Sheng, J. Cao, Y. Shi, Y. Li, D. Wang, and P. Qi. Bad actor, good advisor: Exploring the role of large language models in fake news detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 22105–22113, 2024. doi: 10.1609/aaai.v38i20.30214. URL <https://doi.org/10.1609/aaai.v38i20.30214>.
- [3] K. Ali and K. Zain-Ul-Abdin. Post-truth propaganda: heuristic processing of political fake news on Facebook during the 2016 U.S. presidential election. *Journal of Applied Communication Research*, 49(1):109–128, 2020. doi: 10.1080/00909882.2020.1847311. URL <https://doi.org/10.1080/00909882.2020.1847311>.
- [4] J. Pérez-Dasilva, K. Meso-Ayerdi, and T. Mendiguren-Galdospín. Fake news y coronavirus: detección de los principales actores y tendencias a través del análisis de las conversaciones en Twitter. *El Profesional De La Información*, 29(3), 2020. doi: 10.3145/epi.2020.may.08. URL <https://doi.org/10.3145/epi.2020.may.08>.
- [5] K. Ali, C. Liu, K. Zain-Ul-Abdin, and M. A. Zaffar. Fake news on Facebook: examining the impact of heuristic cues on perceived credibility and sharing intention. *Internet Research*, 32(1):379–397, 2021. doi: 10.1108/intr-10-2019-0442. URL <https://doi.org/10.1108/intr-10-2019-0442>.
- [6] J. Su, T. Y. Zhuo, J. Mansurov, D. Wang, and P. Nakov. Fake news detectors are biased against texts generated by large language models, 2023. URL <https://arxiv.org/abs/2309.08674>.
- [7] L. Cárcamo-Ulloa, C. Cárdenas-Neira, D. Sáez-Trumper, and C. Toural-Bran. Fake news en Chile y España: ¿cómo los medios nos hablan de noticias falsas? *Journal of Iberian and Latin American Research*, pages 1–18, 2021. doi: 10.1080/13260219.2020.1909849. URL <https://doi.org/10.1080/13260219.2020.1909849>.

- [8] J. Cao, X. Luo, and W. Zhang. Corporate employment, red flags, and audit effort. *Journal of Accounting and Public Policy*, 39(1):106710, 2020. doi: 10.1016/j.jaccpubpol.2019.106710. URL <https://doi.org/10.1016/j.jaccpubpol.2019.106710>.
- [9] I. M. Nasser, A. H. Alzaanin, and A. Y. Maghari. Online recruitment fraud detection using ANN. In *2021 Palestinian International Conference on Information and Communication Technology (PICICT)*, pages 13–17, 2021. doi: 10.1109/PICICT53635.2021.00015. URL <https://doi.org/10.1109/PICICT53635.2021.00015>.
- [10] C. Pulido, L. Ruiz-Eugenio, G. Redondo-Sama, and B. Villarejo-Carballido. A new application of social impact in social media for overcoming fake news in health. *International Journal of Environmental Research and Public Health*, 17(7):2430, 2020. doi: 10.3390/ijerph17072430. URL <https://doi.org/10.3390/ijerph17072430>.
- [11] Fabricio Andrés Zules Acosta. Construcción de un dataset de noticias para el entrenamiento y evaluación de clasificadores automatizados. Trabajo fin de máster universitario en ciberseguridad, Universidad Politécnica de Madrid, 2019. URL <https://doi.org/10.13140/RG.2.2.31181.49126>. ResearchGate.
- [12] J. P. Posadas-Durán, H. Gómez-Adorno, G. Sidorov, and J. J. M. Escobar. Detection of fake news in a new corpus for the Spanish language. *Journal of Intelligent and Fuzzy Systems*, 36(5):4869–4876, 2019. doi: 10.3233/jifs-179034. URL <https://doi.org/10.3233/jifs-179034>.
- [13] Y. Blanco-Fernández, J. Otero-Vizoso, A. Gil-Solla, and J. García-Duque. Enhancing misinformation detection in Spanish language with deep learning: BERT and RoBERTa transformer models. *Applied Sciences*, 14(21):9729, 2024. doi: 10.3390/app14219729. URL <https://doi.org/10.3390/app14219729>.
- [14] M. K. Singh, J. Ahmed, M. A. Alam, K. K. Raghuvanshi, and S. Kumar. A comprehensive review on automatic detection of fake news on social media. *Multimedia Tools and Applications*, 2023. doi: 10.1007/s11042-023-17377-4. URL <https://doi.org/10.1007/s11042-023-17377-4>.
- [15] C. M. Tsai. Stylometric fake news detection based on natural language processing using named entity recognition: In-domain and cross-domain analysis. *Electronics*, 12(17):3676, 2023. doi: 10.3390/electronics12173676. URL <https://doi.org/10.3390/electronics12173676>. Recuperado el 8 de octubre de 2025.
- [16] J. Su, C. Cardie, and P. Nakov. Adapting fake news detection to the era of large language models, 2023. URL <https://arxiv.org/abs/2311.04917>.

- [17] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020. URL <https://doi.org/10.48550/arXiv.2005.14165>.
- [18] L. Hu, S. Wei, Z. Zhao, and B. Wu. Deep learning for fake news detection: A comprehensive survey. *AI Open*, 3:133–155, 2022. doi: 10.1016/j.aiopen.2022.09.001. URL <https://doi.org/10.1016/j.aiopen.2022.09.001>.
- [19] Arsenii Tretiakov, Alejandro Martín García, and David Camacho. Detection of false information in Spanish using machine learning techniques. In *Advances in Intelligent Data Analysis and Applications*. Springer, 2022. doi: 10.1007/978-3-031-21753-1_5. URL http://dx.doi.org/10.1007/978-3-031-21753-1_5.
- [20] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. URL <https://doi.org/10.48550/arXiv.1910.01108>.
- [21] S. D. Das, A. Basak, and S. Dutta. A heuristic-driven uncertainty-based ensemble framework for fake news detection in tweets and news articles. *Neurocomputing*, 491:607–620, 2022. doi: 10.1016/j.neucom.2021.12.037. URL <https://doi.org/10.1016/j.neucom.2021.12.037>.
- [22] A. Thota, P. Tilak, S. Ahluwalia, and N. Lohia. Fake news detection: A deep learning approach. *SMU Scholar*, 2018. URL <https://scholar.smu.edu/datasciencereview/vol1/iss3/10/>. Recuperado el 8 de octubre de 2025.
- [23] G. Hurtado Avilés, R. A. Mora-Gutiérrez, and J. A. Reyes-Ortiz. Calibración de hiper-parámetros en algoritmos metaheurísticos para la detección de fraude digital. In M. Tovar Vidal, A. L. Lezama Sánchez, and M. Contreras González, editors, *Avances recientes en procesamiento de lenguaje natural y otras áreas afines*, pages 14–26. Benemérita Universidad Autónoma de Puebla, 2024. ISBN 978-607-5914-56-5.
- [24] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2017. URL <https://doi.org/10.48550/arXiv.1706.03762>.
- [25] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding, 2018. URL <https://doi.org/10.48550/arXiv.1810.04805>.

- [26] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu. TinyBERT: Distilling BERT for natural language understanding, 2019. URL <https://doi.org/10.48550/arXiv.1909.10351>.
- [27] K. Martínez-Gallego, A. M. Álvarez Ortiz, and J. D. Arias-Londoño. Fake news detection in Spanish using deep learning techniques, 2021. URL <https://arxiv.org/abs/2110.06461>.
- [28] E. Shushkevich, M. Alexandrov, and J. Cardiff. Improving multiclass classification of fake news using BERT-based models and ChatGPT-augmented data. *Inventions*, 8(5):112, 2023. doi: 10.3390/inventions8050112. URL <https://doi.org/10.3390/inventions8050112>.
- [29] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models, 2023. URL <https://doi.org/10.48550/arXiv.2302.13971>.
- [30] Gemini Team, Google. Gemini: A family of highly capable multimodal models, 2023. URL <https://doi.org/10.48550/arXiv.2312.11805>.
- [31] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldin, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan. Constitutional AI: Harmlessness from AI feedback, 2022. URL <https://doi.org/10.48550/arXiv.2212.08073>.
- [32] M. G. R. Anselmo. *Diseño y desarrollo de un método heurístico basado en un sistema socio-cultural de creatividad para la resolución de problemas de optimización no lineales y diseño de zonas electorales*. Tesis de doctorado en ingeniería, Universidad Nacional Autónoma de México, 2013. URL <http://www.ptolomeo.unam.mx:8080/xmlui/handle/132.248.52.100/6066>. Recuperado el 8 de octubre de 2025.
- [33] S. Aqil and M. Lahby. Modeling and solving the fake news detection scheduling problem. In *Studies in computational intelligence*, pages 231–242. Springer, 2021. doi: 10.1007/978-3-030-90087-8_11. URL https://doi.org/10.1007/978-3-030-90087-8_11.
- [34] N. Bacanin, C. Stoean, M. Zivkovic, M. Rakic, R. Strulak-Wójcikiewicz, and R. Stoean. On the benefits of using metaheuristics in the hyperparameter tuning

- of deep learning models for energy load forecasting. *Energies*, 16(3):1434, 2023. doi: 10.3390/en16031434. URL <https://doi.org/10.3390/en16031434>.
- [35] S. Hidayattullah, I. Surjandari, and E. Laoh. Financial statement fraud detection in Indonesia listed companies using machine learning based on meta-heuristic optimization. In *2020 International Workshop on Big Data and Information Security (IWBIS)*, pages 79–84, Depok, Indonesia, 2020. doi: 10.1109/IWBIS50925.2020.9255563. URL <https://doi.org/10.1109/IWBIS50925.2020.9255563>.
- [36] J. Horak and A. Sabek. Gaussian process regression’s hyperparameters optimization to predict financial distress. *Retos*, 13(26):273–289, 2023. doi: 10.17163/ret.n26.2023.06. URL <https://doi.org/10.17163/ret.n26.2023.06>.
- [37] G. Yildirim. A novel hybrid multi-thread metaheuristic approach for fake news detection in social media. *Applied Intelligence*, 53:11182–11202, 2023. doi: 10.1007/s10489-022-03972-9. URL <https://doi.org/10.1007/s10489-022-03972-9>.
- [38] N. Deshai and B. Bhaskara Rao. Unmasking deception: a CNN and adaptive PSO approach to detecting fake online reviews. *Soft Computing*, 27:11357–11378, 2023. doi: 10.1007/s00500-023-08507-z. URL <https://doi.org/10.1007/s00500-023-08507-z>.
- [39] J. Howard and S. Ruder. Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1031. URL <https://aclanthology.org/P18-1031>.
- [40] J. D. M. W. Kenton and L. K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL <https://aclanthology.org/N19-1423>.
- [41] S. Ruder. Neural transfer learning for natural language processing. *arXiv preprint arXiv:1708.00524*, 2019. doi: 10.48550/arXiv.1708.00524. URL <https://arxiv.org/abs/1708.00524>.
- [42] Nature Computational Science. The rise of large language models. *Nature Computational Science*, 5:689–690, 2025. doi: 10.1038/s43588-025-00890-x. URL <https://doi.org/10.1038/s43588-025-00890-x>.
- [43] F. Alam, A. Barrón-Cedeño, G. S. Cheema, G. K. Shahi, S. Hakimov, M. Hasain, others, and P. Nakov. Overview of the CLEF-2023 CheckThat! lab task 1

- on check-worthiness of multimodal and multigenre content. In *CEUR Workshop Proceedings*, volume 3497, pages 219–235, 2023. URL <https://dclibrary.mbzuai.ac.ae/nlpfp/78/>. Recuperado el 05 de julio de 2024.
- [44] A. Barrón-Cedeño, F. Alam, T. Caselli, G. Da San Martino, T. Elsayed, A. Galassi, others, and P. Nakov. The CLEF-2023 CheckThat! lab: Checkworthiness, subjectivity, political bias, factuality, and authority. In J. Kamps et al., editors, *Advances in Information Retrieval. ECIR 2023. Lecture Notes in Computer Science*, volume 13982. Springer, Cham, 2023. doi: 10.1007/978-3-031-28241-6_59. URL https://doi.org/10.1007/978-3-031-28241-6_59.
- [45] M. E. Aragón, H. Jarquín, M. M. Y. Gómez, H. J. Escalante, L. Villaseñor-Pineda, H. Gómez-Adorno, others, and J. P. Posadas-Durán. Overview of mexa3t at iberlef 2020: Fake news and aggressiveness analysis in mexican Spanish. In *Notebook Papers of 2nd SEPLN Workshop on Iberian Languages Evaluation Forum (IberLEF)*, Malaga, España, 2020. URL <https://openreview.net/forum?id=vkjfS0w2hu>. Recuperado el 8 de octubre de 2025.
- [46] H. Gómez-Adorno, J. P. Posadas-Durán, G. B. Enguix, and C. P. Capetillo. Overview of FakeDeS at IberLEF 2021: Fake news detection in Spanish shared task. *Procesamiento del Lenguaje Natural*, 67:223–231, 2021. doi: 10.26342/2021-67-19. URL <https://doi.org/10.26342/2021-67-19>.
- [47] J. M. Ramírez Cruz, S. Ú. Palacios Alvarado, K. E. Franca Tapia, J. P. F. Posadas Durán, H. M. Gómez Adorno, and G. Sidorov. The Spanish fake news corpus version 2.0 en GitHub. GitHub repository, 2021. URL <https://github.com/jpposadas/FakeNewsCorpusSpanish>. Recuperado el 8 de octubre de 2025.
- [48] Eleftheria Papageorgiou, Christos Chronis, Iraklis Varlamis, and Yassine Himeur. A survey on the use of large language models (llms) in fake news. *Future Internet*, 16(8):298, 2024. doi: 10.3390/fi16080298. URL <https://doi.org/10.3390/fi16080298>.
- [49] Josué Padilla Cuevas, José A. Reyes-Ortiz, Alma D. Cuevas-Rasgado, Román A. Mora-Gutiérrez, and Maricela Bravo. Médicobert: A medical language model for spanish natural language processing tasks with a question-answering application using hyperparameter optimization. *Applied Sciences*, 14(16), 2024. ISSN 2076-3417. doi: 10.3390/app14167031. URL <https://www.mdpi.com/2076-3417/14/16/7031>.
- [50] K. M. Yazdi, A. M. Yazdi, S. Khodayi, J. Hou, W. Zhou, and S. Saedy. Improving fake news detection using K-Means and support vector machine approaches, 2020. URL <https://publications.waset.org/10011058/improving-fake-news-detection-using-k-means-and-support-vector-machine-approaches>. Recuperado el 8 de octubre de 2025.

-
- [51] A. Yasmin, W. Haider Butt, and A. Daud. Ensemble effort estimation with metaheuristic hyperparameters and weight optimization for achieving accuracy. *PLOS ONE*, 19(4):e0300296, 2024. doi: 10.1371/journal.pone.0300296. URL <https://doi.org/10.1371/journal.pone.0300296>.
 - [52] J. H. Holland. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992. ISBN 978-0-262-58111-0. Edición MIT Press del trabajo original de 1975.
 - [53] J. Kennedy and R. C. Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948, 1995. doi: 10.1109/ICNN.1995.488968. URL <https://doi.org/10.1109/ICNN.1995.488968>.
 - [54] R. C. Eberhart and J. Kennedy. A new optimizer using particle swarm theory. In *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science*, pages 39–43, 1995. doi: 10.1109/MHS.1995.494215. URL <https://doi.org/10.1109/MHS.1995.494215>.
 - [55] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671. URL <https://doi.org/10.1126/science.220.4598.671>.
 - [56] R. Martí, M. G. C. Resende, and P. M. Pardalos. Multi-start methods. In *Handbook of Heuristics*, pages 195–212. Springer, 2018. doi: 10.1007/978-3-319-91086-4_7. URL https://doi.org/10.1007/978-3-319-91086-4_7.
 - [57] F. Glover. A template for scatter search and path relinking. In *Lecture Notes in Computer Science*, volume 1363, pages 13–54. Springer, 1998. doi: 10.1007/BFb0026599. URL <https://doi.org/10.1007/BFb0026599>.
 - [58] N. Mladenović and P. Hansen. Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100, 1997. doi: 10.1016/S0305-0548(97)00031-2. URL [https://doi.org/10.1016/S0305-0548\(97\)00031-2](https://doi.org/10.1016/S0305-0548(97)00031-2).
 - [59] T. O'Malley, E. Bursztein, J. Long, F. Chollet, H. Jin, L. Invernizzi, and The Keras team. Hyperparameter tuning with Keras Tuner. The TensorFlow Blog, 2020. URL <https://blog.tensorflow.org/2020/01/hyperparameter-tuning-with-keras-tuner.html>. Recuperado el 29 de enero de 2020.
 - [60] S. Kapunac, A. Kartelj, and M. Djukanović. Variable neighborhood search for weighted total domination problem and its application in social network information spreading. *Applied Soft Computing*, 143:110387, 2023. doi: 10.1016/j.asoc.2023.110387. URL <https://doi.org/10.1016/j.asoc.2023.110387>.
-

- [61] Fabricio Andrés Zules Acosta. DataSet Web scraping noticias verificadas. Kaggle, 2019. URL <https://www.kaggle.com/datasets/zulanac/fake-and-real-news>. Recuperado el 8 de octubre de 2025.
- [62] F. A. Zules Acosta. Spanish Fake and Real News. Kaggle, 2019. URL <https://www.kaggle.com/datasets/zulanac/fake-and-real-news>. Recuperado el 8 de octubre de 2025.
- [63] Arsenii Tretiakov, Alejandro Martín García, and David Camacho. Noticias falsas en español. Kaggle, 2022. URL <https://www.kaggle.com/datasets/arseniitretiakov/noticias-falsas-en-espaol>. Recuperado el 8 de octubre de 2025.
- [64] Y. Blanco-Fernández, J. Otero-Vizoso, A. Gil-Solla, and J. García-Duque. Spanish Political Fake News. Kaggle, 2024. URL <https://www.kaggle.com/datasets/javieroterovizoso/spanish-political-fake-news>. Recuperado el 8 de octubre de 2025.
- [65] J. P. Aguirre Quezada. El fraude en México: daños patrimoniales y trabajo legislativo para enfrentarlo. Technical report, Senado de México, 2023. URL <http://bibliodigitalibd.senado.gob.mx/handle/123456789/6051>. Recuperado el 8 de octubre de 2025.
- [66] O. H. Alvarez. Fraude laboral en la era digital. *Revista General de Derecho del Trabajo y de la Seguridad Social*, (59), 2021. ISSN 1696-9626. URL <https://dialnet.unirioja.es/servlet/articulo?codigo=8029600>. Recuperado el 8 de octubre de 2025.
- [67] L. Ehrlinger and W. Wöök. Towards a definition of knowledge graphs. In *SEMANTiCS (Posters, Demos, SuCCESS)*, 2016. URL https://www.researchgate.net/publication/323316736_Towards_a_Definition_of_Knowledge_Graphs. Recuperado el 8 de octubre de 2025.
- [68] G. A. García Robledo, J. A. Reyes-Ortiz, and B. A. González-Beltrán. Interfaz de consulta en idioma español para la búsqueda de información en un ambiente académico. Tesis de maestría en ciencias de la computación, Universidad Autónoma Metropolitana, 2020. URL <https://zaloamati.azc.uam.mx/handle/11191/7812>. Recuperado el 8 de octubre de 2025.
- [69] G. Hurtado Avilés, J. M. Villa Vargas, S. Tapia Hernández, A. Á. Rivera Sannabria, and J. M. Jaimes Ponce. Representación ontológica de la arquitectura de control de un robot SCARA utilizando IoT y MATLAB. In M. Tovar Vidal, A. L. Lezama Sánchez, and M. Contreras González, editors, *Avances recientes en procesamiento de lenguaje natural y otras áreas afines*. Benemérita Universidad Autónoma de Puebla, 2024. ISBN 978-607-5914-56-5.

-
- [70] A. Rogers, O. Kovaleva, and A. Rumshisky. A primer on neural network models for natural language processing. *Journal of Artificial Intelligence Research*, 61: 65–95, 2020. doi: 10.1613/jair.1.11640. URL <https://doi.org/10.1613/jair.1.11640>.
- [71] H. J. Levesque. Knowledge representation and reasoning. *Annual review of computer science*, 1(1):255–287, 1986. doi: 10.1146/annurev.cs.01.060186.001351. URL <https://doi.org/10.1146/annurev.cs.01.060186.001351>.
- [72] E. D. Liddy. Natural language processing. Technical report, Syracuse University, 2001. URL <https://surface.syr.edu/istpub/63/>. Recuperado el 8 de octubre de 2025.
- [73] C. E. Manzano-Velasco. DataSet Web scraping noticias verificadas. Kaggle, 2024. URL <https://www.kaggle.com/datasets/enriquemanzano/dataset-web-scraping-noticias-verificadas/data>. Recuperado el 8 de octubre de 2025.
- [74] C. E. Manzano-Velasco and L. S. Daza-Rosero. Sistema de alerta temprana para monitorear la propagación de noticias falsas: caso de estudio contexto político colombiano. Trabajo grado, Universidad del Cauca, 2024. URL <http://repositorio.unicauca.edu.co:8080/xmlui/handle/123456789/9485>. Recuperado el 8 de octubre de 2025.
- [75] M. F. Mridha, A. J. Keya, M. A. Hamid, M. M. Monowar, and M. S. Rahman. A comprehensive review on fake news detection with deep learning. *IEEE Access*, 9:156151–156170, 2021. doi: 10.1109/ACCESS.2021.3129329. URL <https://doi.org/10.1109/ACCESS.2021.3129329>.
- [76] J. M. Osorio-Giraldo, C. J. Ropain-Zambrano, A. F. Taba-Pulgarin, and A. de Jesús Osorio-Orozco. 10. proyecto de seguridad para reducir fraudes y robos en marketplace de facebook. In *Coloquio de Investigación Formativa 2023-1*, page 89, 2023. URL https://ridum.umanizales.edu.co/xmlui/bitstream/handle/20.500.12746/6834/CIF%202023-1%20-%20Res%C3%BAmenes%20ejecutivos_RIDUM.pdf?sequence=1&isAllowed=y#page=89. Recuperado el 07 de julio de 2024.
- [77] J. Padilla Cuevas, J. A. Reyes-Ortiz, and M. Bravo. Detección y representación de eventos en un ambiente académico inteligente. Tesis de maestría en ciencias de la computación, Universidad Autónoma Metropolitana, 2019. URL <https://zaloamati.azc.uam.mx/handle/11191/6124>. Recuperado el 8 de octubre de 2025.
- [78] J. A. Reyes-Ortiz. Extracción de información semántica para la clasificación de servicios web. Tesis de maestría en ciencias en ciencias de la computación, Centro Nacional de Investigación y Desarrollo Tecnológico, 2008.

- URL <https://www.cenidet.edu.mx/subplan/biblio/seleccion/Tesis/MC%20Jos%E9%20Alejandro%20Reyes%20Ortiz%202008.pdf>. Recuperado el 8 de octubre de 2025.
- [79] J. A. Reyes-Ortiz, B. A. González-Beltrán, and L. Gallardo-López. Clinical decision support systems: A survey of NLP-based approaches from unstructured data. In *2015 26th International Workshop on Database and Expert Systems Applications (DEXA)*, pages 163–167, Valencia, Spain, 2015. doi: 10.1109/DEXA.2015.47. URL <https://doi.org/10.1109/DEXA.2015.47>.
- [80] S. C. Shapiro. Knowledge representation. In *Encyclopedia of cognitive science*. Wiley, 2006. doi: 10.1002/0470018860.s00058. URL <https://doi.org/10.1002/0470018860.s00058>.
- [81] C. Villamil Arcos. Selección de una técnica de aprendizaje de máquina para la detección de fraude financiero digital enfocado a transacciones no autorizadas o consentidas. Master’s thesis, Universidad Nacional de Colombia, 2022. URL <https://repositorio.unal.edu.co/handle/unal/84015>. Recuperado el 8 de octubre de 2025.
- [82] C. Whitehouse, T. Weyde, P. Madhyastha, and N. Komninos. Evaluation of fake news detection with knowledge-enhanced language models, 2022. URL <https://arxiv.org/abs/2204.00458>.
- [83] M. R. Zhang, N. Desai, J. Bae, J. Lorraine, and J. Ba. Using large language models for hyperparameter optimization, 2023. URL <https://openreview.net/forum?id=FUdZ6HE0re>. Recuperado el 8 de octubre de 2025.